

Corso di Laurea in Ingegneria Informatica

Corso di Ingegneria del Software

Prof.ssa A. R. Fasolino — A.A. 2025/26

PROGETTO DI INGEGNERIA DEL SOFTWARE

“Piattaforma per la gestione e la condivisione di materiale didattico”

Gruppo 18

Giovanni Aliperta

Attilio Loris Maria Bontempi

Carlo Cotino

Clemente Antonio

Traccia n. 17 — Consegna: settembre 2026

Indice

1. Specifiche informali	5
1.1 Testo della traccia	5
1.2 Perimetro dello sviluppo	6
2. Analisi e specifica dei requisiti	7
2.1 Analisi nomi-verbi	7
2.2 Revisione dei requisiti	7
2.3 Glossario dei termini	10
2.4 Classificazione dei requisiti	11
2.4.1 Requisiti funzionali	11
2.4.2 Requisiti sui dati	13
2.4.3 Requisiti non funzionali di qualità	14
2.4.4 Vincoli	15
2.5 Modellazione dei casi d'uso	17
2.5.1 Attori e casi d'uso	17
2.5.2 Diagramma dei casi d'uso	18
2.5.3 Scenari dei casi d'uso	19
2.6 Diagramma delle classi di analisi	26
2.6.1 Distribuzione delle responsabilità in analisi	27
2.7 Diagrammi di sequenza di analisi	28
2.7.1 Registrazione (UC1)	28
2.7.2 Accesso (UC2)	29
2.7.3 IscrivitiCorso (UC6)	30
2.7.4 PubblicaContenuto (UC7)	30
2.7.5 VisualizzaContenutiCorso (UC13)	32
2.7.6 MonitoraAndamentoCorso (UC15)	34
2.7.7 Diagramma di stato del Contenuto	36
2.8 Distribuzione delle responsabilità dopo i diagrammi di sequenza	39
2.9 Diagramma delle classi di analisi raffinato	39
2.9.1 La trasformazione della classe Piattaforma	40
3. Piano di test funzionale	43
3.1 Registrazione (UC1)	43
3.2 Accesso (UC2)	44
3.3 IscrivitiCorso (UC6)	45
3.4 PubblicaContenuto (UC7)	46
3.5 AttivaContenuto (UC9) e RimuoviContenuto (UC11)	47
3.6 VisualizzaContenutiCorso (UC13)	48
3.7 MonitoraAndamentoCorso (UC15)	49
3.8 VisualizzaNotifiche (UC22)	50
4. Progettazione	51

4.1 Architettura: il pattern BCED	51
4.1.1 Il package dto.....	52
4.2 Diagramma delle classi di progettazione.....	53
4.2.1 Traduzione delle responsabilità di analisi.....	53
4.2.2 Vista d'insieme.....	54
4.2.3 Package boundary	54
4.2.4 Package control	57
4.2.5 Package entity	59
4.2.6 Package database	62
4.2.7 Schema relazionale.....	63
4.3 Design pattern applicati.....	66
4.3.1 Singleton	66
4.3.2 Façade.....	67
4.3.3 Factory Method	67
4.3.4 State.....	67
4.3.5 Observer	68
4.3.6 Adapter	69
4.4 Verifica dei principi di progettazione.....	70
4.5 Diagrammi di sequenza di progettazione.....	71
4.5.1 Registrazione (UC1)	71
4.5.2 Accesso (UC2)	73
4.5.3 IscrivitiCorso (UC6)	74
4.5.4 PubblicaContenuto (UC7)	74
4.5.5 VisualizzaContenutiCorso (UC13)	76
4.5.6 MonitoraAndamentoCorso (UC15)	76
4.5.7 VisualizzaNotifiche (UC22).....	78
5. Implementazione	79
5.1 Struttura del progetto e tecnologie.....	79
5.1.1 Configurazione della persistenza.....	80
5.2 Package database	81
5.2.1 JpaUtil	81
5.2.2 GestorePersistenza	81
5.3 Package entity.....	84
5.3.1 Mappatura delle entità.....	84
5.3.2 Le regole di business nelle entità	84
5.3.3 Il pattern State nel codice.....	85
5.3.4 I Registri	87
5.4 Package control	88
5.4.1 PiattaformaFacade	88
5.4.2 L'autenticazione.....	88
5.4.3 La validazione dei dati	89
5.4.4 Observer e Adapter nel codice	90

5.5 Package boundary	92
5.5.1 La classe StileGUI	92
5.5.2 Le schermate realizzate	92
5.6 Package dto	99
5.7 Diagramma di deployment	99
6. Verifica e validazione	101
6.1 Test strutturale	101
6.1.1 Grafo di flusso di controllo	101
6.1.2 Complessità ciclomatica	102
6.1.3 Cammini indipendenti e corrispondenza con i casi di test	103
6.2 Test di unità con JUnit	104
6.2.1 Isolamento dell'ambiente di test.....	104
6.2.2 Elenco dei test e loro esito	104
6.3 Esiti del test funzionale.....	108
6.4 Verifica del rispetto dell'architettura	110
6.5 Registro dei difetti individuati e corretti	110
Appendice A — Indice dei diagrammi e collegamento con il modello	112
Appendice B — Installazione ed esecuzione.....	114
B.1 Prerequisiti	114
B.2 Preparazione del database	114
B.3 Compilazione ed esecuzione	114
B.4 Credenziali di dimostrazione	115
Appendice C — Registro delle revisioni.....	116

1. Specifiche informali

Si riporta di seguito il testo della traccia assegnata al gruppo (Traccia n. 17), che costituisce il documento di partenza dell'intera attività di analisi. Il testo è riportato integralmente e senza modifiche: le riformulazioni necessarie a renderlo verificabile sono oggetto del capitolo 2.

1.1 Testo della traccia

Si intende sviluppare un sistema software per la gestione e la condivisione di materiale didattico pubblicato dai docenti e consultato dagli studenti.

Il sistema consente l'accesso di docenti e studenti a una piattaforma digitale mediante autenticazione tramite credenziali personali. Al momento della registrazione, ciascun utente deve specificare il proprio ruolo (studente o docente), oltre a fornire nome, cognome ed indirizzo email istituzionale.

Ogni docente ha la possibilità di creare e gestire uno o più corsi. Ciascun corso è identificato da un titolo, una descrizione, un codice univoco e, opzionalmente, dall'anno accademico di riferimento. L'iscrizione di uno studente a un corso può avvenire in due modalità: il docente può iscrivere direttamente lo studente al corso, oppure lo studente stesso può iscriversi autonomamente inserendo il codice univoco del corso.

All'interno di ogni corso, il docente può pubblicare materiale didattico di vario tipo mediante un'apposita interfaccia grafica. Per ciascun contenuto pubblicato devono essere specificati almeno un titolo, una breve descrizione, una categoria e la data di pubblicazione. Le categorie possono comprendere, ad esempio, slide, dispense, esercizi, soluzioni, avvisi o materiale integrativo. Per semplicità si assume che ogni contenuto appartenga a una sola categoria.

Il docente può inoltre organizzare i materiali in sezioni o moduli didattici, ciascuno identificato da un titolo. Deve essere possibile associare ogni contenuto a una determinata sezione del corso, in modo da agevolarne la consultazione. I materiali possono essere resi visibili immediatamente agli studenti oppure mantenuti temporaneamente non pubblicati fino a una successiva attivazione da parte del docente.

Ogni studente autenticato dispone di una propria interfaccia nella quale può visualizzare l'elenco dei corsi a cui è iscritto. Selezionando un corso, può consultare l'elenco completo dei materiali didattici disponibili, eventualmente filtrandoli per categoria, data o sezione. Deve inoltre essere possibile accedere al dettaglio di ciascun contenuto e visualizzarne le informazioni principali.

Il sistema mantiene traccia dei materiali pubblicati per ciascun corso e consente al docente di modificarne le informazioni o rimuoverli. Per semplicità, si può supporre che un materiale eliminato non sia più visibile agli studenti. Il docente può inoltre visualizzare l'elenco completo dei contenuti pubblicati per ciascun corso e monitorarne la distribuzione per categoria.

Ogni studente ha accesso a una sezione personale in cui può consultare i corsi seguiti e i materiali recentemente pubblicati. Il docente, attraverso un'interfaccia dedicata, può monitorare l'andamento complessivo dei propri corsi, visualizzando il numero totale di materiali pubblicati, il numero di studenti iscritti a ciascun corso e le categorie di contenuti maggiormente utilizzate.

Il sistema può includere una funzione di ricerca dei materiali per titolo, descrizione o categoria. Deve inoltre essere possibile visualizzare il profilo del docente associato a ciascun corso, comprensivo almeno del nome e dell'elenco dei corsi gestiti.

La piattaforma dovrà essere accessibile via web sia da desktop che da dispositivi mobili, con un'interfaccia grafica intuitiva e responsiva. È previsto un sistema di notifiche che avvisi gli studenti della pubblicazione di nuovo materiale o dell'inserimento di eventuali avvisi da parte del docente. Il sistema dovrà inoltre garantire la protezione dei dati personali, la sicurezza delle informazioni e la corretta gestione dei permessi di accesso tra docenti e studenti.

1.2 Perimetro dello sviluppo

La traccia descrive un sistema più ampio di quanto sia realizzabile nel tempo assegnato. Coerentemente con le indicazioni del corso, l'analisi è stata condotta sull'intero dominio — tutti i requisiti sono stati estratti, classificati e modellati come casi d'uso — mentre la progettazione di dettaglio e l'implementazione sono state sviluppate su un sottoinsieme significativo di casi d'uso, scelto in modo che attraversi tutti e quattro i livelli dell'architettura e coinvolga tutte le entità del dominio.

I casi d'uso sviluppati fino al codice funzionante sono nove: Registrazione (UC1), Accesso (UC2), IscrivitiCorso (UC6), PubblicaContenuto (UC7), AttivaContenuto (UC9), RimuoviContenuto (UC11), VisualizzaContenutiCorso (UC13), MonitoraAndamentoCorso (UC15) e VisualizzaNotifiche (UC22). I primi due costituiscono il presupposto di tutti gli altri: senza autenticazione la pre-condizione «l'Utente ha effettuato l'accesso», che compare in ogni scenario, resterebbe un'affermazione non verificata dal sistema.

Scelta di perimetro. *Registrazione e Accesso non erano stati sviluppati nella prima versione dell'elaborato perché non rientravano fra i quattro casi d'uso assegnati ai membri del gruppo. Questa versione li include, e con essi la gestione della sessione di lavoro: è la sessione autenticata a garantire le pre-condizioni degli altri casi d'uso, e a rendere verificabili i controlli di titolarità del Corso e di iscrizione dello Studente che prima erano soltanto dichiarati.*

Due indicazioni della traccia sono state deliberatamente interpretate in modo restrittivo, per ragioni che vengono argomentate al capitolo 4: l'accessibilità «via web da desktop e da dispositivi mobili» è stata realizzata come applicazione desktop Java Swing, secondo il vincolo V03 imposto dal corso; il «servizio di messaggistica» per le notifiche è stato modellato come servizio esterno raggiunto attraverso un adattatore, ma la sua realizzazione concreta è simulata.

2. Analisi e specifica dei requisiti

Il capitolo documenta il percorso che porta dal testo informale della traccia a un modello di analisi verificabile: individuazione dei concetti candidati (§2.1), riscrittura dei requisiti in frasi atomiche (§2.2), glossario (§2.3), classificazione (§2.4), modellazione dei casi d'uso (§2.5), modello statico (§2.6), modello dinamico (§2.7) e infine il raffinamento che chiude l'analisi e apre la progettazione (§2.8 e §2.9).

2.1 Analisi nomi-verbi

Sul testo della traccia è stata condotta l'analisi grammaticale suggerita dal corso: i sostantivi ricorrenti sono candidati a diventare classi o attributi, i verbi candidati a diventare funzionalità, i soggetti che interagiscono con il sistema candidati a diventare attori. L'esito è riassunto nella tabella seguente; il termine «materiale didattico» presente nella traccia è stato uniformato in «Contenuto», che è il nome adottato in tutto il resto del documento.

Categoria	Termini individuati nel testo della traccia
Classi candidate	Utente, Studente, Docente, Corso, Contenuto (materiale didattico), Sezione (modulo didattico), Categoria, Iscrizione, Notifica, Piattaforma
Attributi candidati	nome, cognome, email istituzionale, ruolo, credenziali; titolo, descrizione, codice univoco, anno accademico; titolo, descrizione, categoria, data di pubblicazione, stato di pubblicazione
Funzionalità candidate	registrarsi, accedere, creare, gestire, modificare, iscrivere, iscriversi, pubblicare, organizzare, associare, rendere visibile, attivare, rimuovere, visualizzare, filtrare, consultare, monitorare, cercare, notificare
Attori candidati	Docente, Studente (specializzazioni di Utente)
Termini scartati	«piattaforma digitale» e «sistema» designano il sistema stesso, non una classe del dominio; «interfaccia grafica» appartiene alla soluzione, non al problema

Tabella 1 — Esito dell'analisi nomi-verbi sul testo della traccia

Sul termine «Piattaforma». In questa fase Piattaforma è stata mantenuta come classe di analisi, con il ruolo di punto d'ingresso alle funzionalità di sistema (registrazione, accesso, invio delle notifiche). Non è un oggetto del dominio: è la sede provvisoria delle responsabilità che non appartengono naturalmente a nessuna entità. Il §2.9 mostra dove queste responsabilità vengono ricollocate in progettazione, e la classe scompare.

2.2 Revisione dei requisiti

Ogni frase della traccia è stata riscritta in forma atomica, non ambigua e verificabile, secondo lo schema «Il sistema deve...» per le funzionalità e «Di ogni X si vuole memorizzare...» per i dati. Le frasi sono numerate: il numero costituisce il riferimento di origine usato nelle tabelle di classificazione del §2.4, e permette di risalire da ogni requisito codificato alla porzione di traccia da cui deriva.

N.	Requisito revisionato
1	Il sistema deve offrire all'Utente una funzionalità per registrarsi, specificando ruolo, nome, cognome ed email istituzionale.
2	Il sistema deve offrire all'Utente una funzionalità per accedere mediante credenziali personali.
3	Di ogni Utente si vuole memorizzare nome, cognome, email istituzionale, ruolo e credenziali di accesso.
4	Il sistema deve offrire al Docente una funzionalità per creare un Corso.
5	Il sistema deve offrire al Docente una funzionalità per modificare i dati di un Corso di cui è titolare.
6	Di ogni Corso si vuole memorizzare titolo, descrizione, codice univoco, anno accademico (opzionale) e Docente titolare.
7	Il sistema deve offrire al Docente una funzionalità per iscrivere direttamente uno Studente a un Corso.
8	Il sistema deve offrire allo Studente una funzionalità per iscriversi autonomamente a un Corso inserendo il codice univoco.
9	Di ogni Iscrizione si vuole memorizzare lo Studente e il Corso associati.
10	Il sistema deve offrire al Docente una funzionalità per pubblicare un Contenuto didattico all'interno di un Corso.
11	Di ogni Contenuto si vuole memorizzare titolo, descrizione, categoria, data di pubblicazione e stato di pubblicazione.
12	Ogni Contenuto deve appartenere a una sola Categoria (slide, dispense, esercizi, soluzioni, avvisi o materiale integrativo).
13	Il sistema deve offrire al Docente una funzionalità per organizzare i Contenuti di un Corso in Sezioni.
14	Di ogni Sezione si vuole memorizzare titolo e Corso di appartenenza.
15	Il sistema deve offrire al Docente una funzionalità per associare un Contenuto a una Sezione del Corso.
16	Il sistema deve offrire al Docente una funzionalità per rendere visibile o mantenere non pubblicato un Contenuto, attivandolo successivamente.
17	Il sistema deve offrire al Docente una funzionalità per modificare un Contenuto pubblicato.
18	Il sistema deve offrire al Docente una funzionalità per rimuovere un Contenuto pubblicato.
19	Il sistema deve offrire allo Studente una funzionalità per visualizzare l'elenco dei Corsi a cui è iscritto.
20	Il sistema deve offrire allo Studente una funzionalità per visualizzare l'elenco dei Contenuti di un Corso.
21	Il sistema deve offrire allo Studente una funzionalità per filtrare i Contenuti di un Corso per categoria, data o Sezione.
22	Il sistema deve offrire allo Studente una funzionalità per visualizzare il dettaglio di un Contenuto.
23	Il sistema deve offrire al Docente una funzionalità per visualizzare l'elenco dei Contenuti pubblicati di un Corso.
24	Il sistema deve offrire al Docente una funzionalità per visualizzare la distribuzione dei Contenuti per Categoria in un Corso.
25	Il sistema deve offrire al Docente una funzionalità per monitorare l'andamento di un Corso, visualizzando numero di Contenuti pubblicati, numero di Studenti iscritti e Categorie più utilizzate.
26	Il sistema deve offrire all'Utente una funzionalità per cercare Contenuti per titolo, descrizione o categoria.

N.	Requisito revisionato
27	Il sistema deve offrire all'Utente una funzionalità per visualizzare il profilo di un Docente, comprensivo di nome ed elenco dei Corsi gestiti.
28	Il sistema deve inviare una Notifica allo Studente quando viene pubblicato un nuovo Contenuto in un Corso a cui è iscritto.
29	Il sistema deve inviare una Notifica allo Studente quando il Docente pubblica un avviso nel Corso.
30	L'email istituzionale di ogni Utente deve essere univoca nel sistema e costituisce l'identificativo con cui l'Utente si autentica.
31	Di ogni Utente si vuole memorizzare il numero di tentativi di accesso falliti consecutivi e l'eventuale istante fino al quale l'account risulta bloccato.
32	Il sistema deve offrire allo Studente una funzionalità per visualizzare le Notifiche ricevute.
33	Di ogni Notifica si vuole memorizzare il testo, la data di invio, lo Studente destinatario e lo stato di lettura.

Tabella 2 — Requisiti revisionati in forma atomica

Fraasi 30-33. Le ultime quattro frasi non compaiono letteralmente nella traccia ma ne esplicitano contenuti impliciti: l'unicità dell'email discende dal fatto che essa costituisce la credenziale di accesso; il conteggio dei tentativi falliti discende dalla richiesta di «sicurezza delle informazioni»; la consultazione delle notifiche discende dall'esistenza stessa del sistema di notifiche, che sarebbe inutile se lo Studente non potesse leggerle. Renderle esplicite è ciò che permette di verificarle.

2.3 Glossario dei termini

Il glossario fissa il significato dei termini del dominio usati nel resto del documento, con i sinonimi incontrati nella traccia. Serve a garantire che analisti, progettisti e sviluppatori attribuiscono lo stesso significato alle stesse parole.

Termine	Descrizione	Sinonimi
Utente	Persona registrata alla piattaforma, con ruolo Docente o Studente; si autentica con la propria email istituzionale e una password	—
Docente	Utente che crea e gestisce Corsi e vi pubblica Contenuti	—
Studente	Utente che si iscrive a Corsi e ne consulta i Contenuti	—
Corso	Insieme organizzato di materiale didattico erogato da un Docente, identificato da un codice univoco	—
Contenuto	Unità di materiale didattico pubblicata dal Docente all'interno di un Corso	Materiale didattico, materiale
Sezione	Modulo in cui il Docente organizza i Contenuti di un Corso per agevolarne la consultazione	Modulo didattico
Categoria	Tipo di un Contenuto: slide, dispense, esercizi, soluzioni, avvisi, materiale integrativo	Tipologia
Iscrizione	Associazione fra uno Studente e un Corso a cui è abilitato ad accedere	—
Notifica	Messaggio inviato allo Studente per informarlo della pubblicazione di nuovo materiale o di un avviso del Docente	Avviso di sistema

Termine	Descrizione	Sinonimi
Stato di pubblicazione	Condizione di visibilità di un Contenuto: bozza (non visibile), pubblicato (visibile), rimosso (non più visibile)	Visibilità
Sessione	Periodo di lavoro di un Utente autenticato, dal momento dell'accesso a quello dell'uscita	—

Tabella 3 — Glossario dei termini del dominio

2.4 Classificazione dei requisiti

I requisiti revisionati sono stati classificati in requisiti funzionali (RF), requisiti sui dati (RD), requisiti non funzionali di qualità (RQ) e vincoli (V). La colonna «Origine» riporta il numero della frase del §2.2 da cui il requisito deriva, così che ogni riga sia tracciabile fino al testo della traccia.

2.4.1 Requisiti funzionali

ID	Requisito	Origine
RF01	Il sistema deve offrire all'Utente una funzionalità per registrarsi, specificando ruolo, nome, cognome ed email istituzionale	1
RF02	Il sistema deve offrire all'Utente una funzionalità per accedere mediante credenziali personali	2
RF03	Il sistema deve offrire al Docente una funzionalità per creare un Corso	4
RF04	Il sistema deve offrire al Docente una funzionalità per modificare i dati di un Corso di cui è titolare	5
RF05	Il sistema deve offrire al Docente una funzionalità per iscrivere direttamente uno Studente a un Corso	7
RF06	Il sistema deve offrire allo Studente una funzionalità per iscriversi autonomamente a un Corso inserendo il codice univoco	8
RF07	Il sistema deve offrire al Docente una funzionalità per pubblicare un Contenuto didattico all'interno di un Corso	10
RF08	Il sistema deve offrire al Docente una funzionalità per organizzare i Contenuti di un Corso in Sezioni	13
RF09	Il sistema deve offrire al Docente una funzionalità per associare un Contenuto a una Sezione del Corso	15
RF10	Il sistema deve offrire al Docente una funzionalità per rendere visibile o mantenere non pubblicato un Contenuto, attivandolo successivamente	16
RF11	Il sistema deve offrire al Docente una funzionalità per modificare un Contenuto pubblicato	17
RF12	Il sistema deve offrire al Docente una funzionalità per rimuovere un Contenuto pubblicato	18
RF13	Il sistema deve offrire allo Studente una funzionalità per visualizzare l'elenco dei Corsi a cui è iscritto	19
RF14	Il sistema deve offrire allo Studente una funzionalità per visualizzare l'elenco dei Contenuti di un Corso	20
RF15	Il sistema deve offrire allo Studente una funzionalità per filtrare i Contenuti di un Corso per categoria, data o Sezione	21

ID	Requisito	Origine
RF16	Il sistema deve offrire allo Studente una funzionalità per visualizzare il dettaglio di un Contenuto	22
RF17	Il sistema deve offrire al Docente una funzionalità per visualizzare l'elenco dei Contenuti pubblicati di un Corso	23
RF18	Il sistema deve offrire al Docente una funzionalità per visualizzare la distribuzione dei Contenuti per Categoria in un Corso	24
RF19	Il sistema deve offrire al Docente una funzionalità per monitorare l'andamento di un Corso (numero di Contenuti pubblicati, numero di Studenti iscritti, Categorie più utilizzate)	25
RF20	Il sistema deve offrire all'Utente una funzionalità per cercare Contenuti per titolo, descrizione o categoria	26
RF21	Il sistema deve offrire all'Utente una funzionalità per visualizzare il profilo di un Docente, comprensivo di nome ed elenco dei Corsi gestiti	27
RF22	Il sistema deve inviare una Notifica allo Studente quando viene pubblicato un nuovo Contenuto in un Corso a cui è iscritto	28
RF23	Il sistema deve inviare una Notifica allo Studente quando il Docente pubblica un avviso nel Corso	29
RF24	Il sistema deve offrire allo Studente una funzionalità per visualizzare le Notifiche ricevute	32

Tabella 4 — Requisiti funzionali

2.4.2 Requisiti sui dati

ID	Requisito	Origine
RD01	Di ogni Utente si vuole memorizzare nome, cognome, email istituzionale, ruolo e credenziali di accesso	3
RD02	Di ogni Corso si vuole memorizzare titolo, descrizione, codice univoco, anno accademico (opzionale) e Docente titolare	6
RD03	Di ogni Iscrizione si vuole memorizzare lo Studente e il Corso associati	9
RD04	Di ogni Contenuto si vuole memorizzare titolo, descrizione, categoria, data di pubblicazione e stato di pubblicazione	11
RD05	Ogni Contenuto deve appartenere a una sola Categoria (slide, dispense, esercizi, soluzioni, avvisi, materiale integrativo)	12
RD06	Di ogni Sezione si vuole memorizzare titolo e Corso di appartenenza	14
RD07	L'email istituzionale di ogni Utente deve essere univoca nel sistema e costituisce l'identificativo con cui l'Utente si autentica	30
RD08	Di ogni Utente si vuole memorizzare il numero di tentativi di accesso falliti consecutivi e l'eventuale istante fino al quale l'account risulta bloccato	31
RD09	Di ogni Notifica si vuole memorizzare il testo, la data di invio, lo Studente destinatario e lo stato di lettura	33

Tabella 5 — Requisiti sui dati

2.4.3 Requisiti non funzionali di qualità

I requisiti di qualità sono espressi secondo le caratteristiche dello standard ISO/IEC 25010 e, come richiesto, sono corredati di una metrica, di un valore atteso e della modalità di verifica: un requisito di qualità privo di metrica non è verificabile e non è quindi un requisito.

ID	Categoria	Descrizione	Metrica	Valore atteso	Verifica
RQ-QUAL-01	Usabilità	L'interfaccia deve essere intuitiva e utilizzabile senza formazione specifica	Tempo di completamento di un'operazione base	≤ 3 minuti	Test con utenti
RQ-QUAL-02	Sicurezza	Il sistema deve proteggere i dati personali e gestire correttamente i permessi fra Docenti e Studenti	N. accessi non autorizzati a risorse riservate	0	Test di controllo dei permessi
RQ-QUAL-03	Affidabilità	Il sistema deve essere disponibile agli Utenti registrati	Uptime mensile	≥ 99%	Monitoraggio
RQ-QUAL-04	Portabilità	La piattaforma deve essere eseguibile su diverse piattaforme desktop grazie alla portabilità della JVM	Corretto funzionamento sui sistemi testati	100% piattaforme target	Test cross-platform
RQ-QUAL-05	Efficienza	Il sistema deve mostrare rapidamente l'elenco dei materiali richiesti	Tempo di risposta	≤ 2 s (95% richieste)	Test di carico
RQ-SEC-01	Sicurezza	Il sistema deve bloccare temporaneamente l'account dopo un numero massimo di tentativi di accesso falliti consecutivi	N. tentativi prima del blocco / durata	5 tentativi / 15 minuti	Test di unità su GestoreUtenti
RQ-SEC-02	Sicurezza	In caso di autenticazione fallita il sistema non deve rivelare se l'email inserita corrisponda a un account esistente	N. messaggi di errore distinti fra «email inesistente» e «password errata»	0	Ispezione del codice e test di unità

Tabella 6 — Requisiti non funzionali di qualità (ISO/IEC 25010)

2.4.4 Vincoli

I vincoli non derivano dalla traccia ma dal contesto del progetto didattico: fissano tecnologie e pattern architetturali imposti dal corso. Rispetto alla prima versione dell'elaborato il vincolo V02 è stato riscritto e sono stati aggiunti V05 e V06.

ID	Vincolo	Motivazione
V01	Il sistema deve essere sviluppato in linguaggio Java, rispettando il pattern architetturale BCED (Boundary-Control-Entity-Database)	Imposto dal corso (lezione 14)
V02	Il livello di persistenza deve essere realizzato mediante il framework ORM Hibernate (JPA) su DBMS MySQL; l'accesso alla persistenza deve essere incapsulato in un'unica classe Façade del package Database	Imposto dal corso (lezione 16, pag. 72): «non scriveremo direttamente tutto il codice JDBC: useremo Hibernate»

ID	Vincolo	Motivazione
V03	L'interfaccia grafica deve essere realizzata mediante la libreria Java Swing	Imposto dal corso (lezione 16)
V04	Per l'invio delle Notifiche agli Studenti deve essere disponibile un servizio di messaggistica, la cui realizzazione concreta è esterna al sistema	Derivato dalle frasi 28-29 della traccia
V05	L'accesso alle funzionalità del sistema deve essere consentito esclusivamente a Utenti autenticati mediante credenziali personali	Derivato dalla frase 2 e dalla richiesta di «corretta gestione dei permessi»
V06	La gestione delle dipendenze e la compilazione del progetto devono essere realizzate con Apache Maven	Imposto dal corso (lezione 16)

Tabella 7 — Vincoli e altri requisiti

Perché V02 è cambiato. Nella prima versione dell'elaborato il vincolo V02 imponeva la persistenza «mediante il pattern DAO, utilizzando il DBMS H2». Era una scelta del gruppo, non una prescrizione del corso, e contraddiceva l'indicazione esplicita della lezione 16. La riscrittura di V02 è la causa prima di tutta la revisione architetturale documentata al capitolo 4: rimosse le classi DAO, introdotto Hibernate, sostituito H2 con MySQL.

2.5 Modellazione dei casi d'uso

2.5.1 Attori e casi d'uso

Gli attori primari sono il Docente e lo Studente, entrambi specializzazioni dell'attore generico Utente: registrazione, accesso e ricerca sono funzionalità comuni, ereditate da entrambi. La generalizzazione fra attori evita di duplicare tre casi d'uso su due attori distinti ed è la traduzione, sul diagramma dei casi d'uso, della gerarchia Utente → {Studente, Docente} che ritroveremo nel diagramma delle classi.

Caso d'uso	Attore primario	Attore secondario	Relazioni	Requisiti
UC1: Registrazione	Utente	—	—	RF01
UC2: Accesso	Utente	—	—	RF02
UC3: CreaCorso	Docente	—	—	RF03
UC4: ModificaCorso	Docente	—	—	RF04
UC5: IscrivitiStudente	Docente	—	—	RF05
UC6: IscrivitiCorso	Studente	—	—	RF06
UC7: PubblicaContenuto	Docente	—	«include» UC21	RF07
UC8: OrganizzaSezioni	Docente	—	—	RF08, RF09
UC9: AttivaContenuto	Docente	—	«include» UC21	RF10
UC10: ModificaContenuto	Docente	—	—	RF11
UC11: RimuoviContenuto	Docente	—	—	RF12
UC12: VisualizzaCorsiIscritti	Studente	—	—	RF13
UC13: VisualizzaContenutiCorso	Studente	—	«include» UC18	RF14
UC14: VisualizzaDettaglioContenuto	Studente	—	—	RF16
UC15: MonitoraAndamentoCorso	Docente	—	«include» UC19, UC20	RF19
UC16: CercaContenuti	Utente	—	—	RF20
UC17: VisualizzaProfiloDocente	Utente	—	—	RF21
UC18: FiltraContenuti	Studente	—	«included by» UC13	RF15
UC19: VisualizzaContenutiPubblicati	Docente	—	«included by» UC15	RF17
UC20: VisualizzaDistribuzioneCategoria	Docente	—	«included by» UC15	RF18
UC21: InviaNotifica	—	Servizio di messaggistica	«included by» UC7, UC9	RF22, RF23
UC22: VisualizzaNotifiche	Studente	—	—	RF24

Tabella 8 — Casi d'uso, attori e requisiti corrispondenti

Che cosa è cambiato rispetto alla prima versione. Sono stati aggiunti UC22 (VisualizzaNotifiche), che dà allo Studente il modo di leggere le notifiche prodotte da UC21, e la relazione «include» fra

UC9 e UC21: anche l'attivazione differita di un Contenuto rende il materiale visibile, e deve quindi notificare gli iscritti esattamente come la pubblicazione immediata. Nella prima versione la notifica era legata al solo UC7, il che lasciava scoperto lo scenario previsto dalla frase 16 della traccia.

2.5.2 Diagramma dei casi d'uso

Il diagramma riporta i ventidue casi d'uso, i due attori primari con la loro generalizzazione da Utente e l'attore secondario che rappresenta il servizio di messaggistica esterno. Le relazioni «include» sono usate dove un comportamento è parte obbligatoria di più casi d'uso: la notifica, il filtraggio, i due conteggi che compongono il monitoraggio.

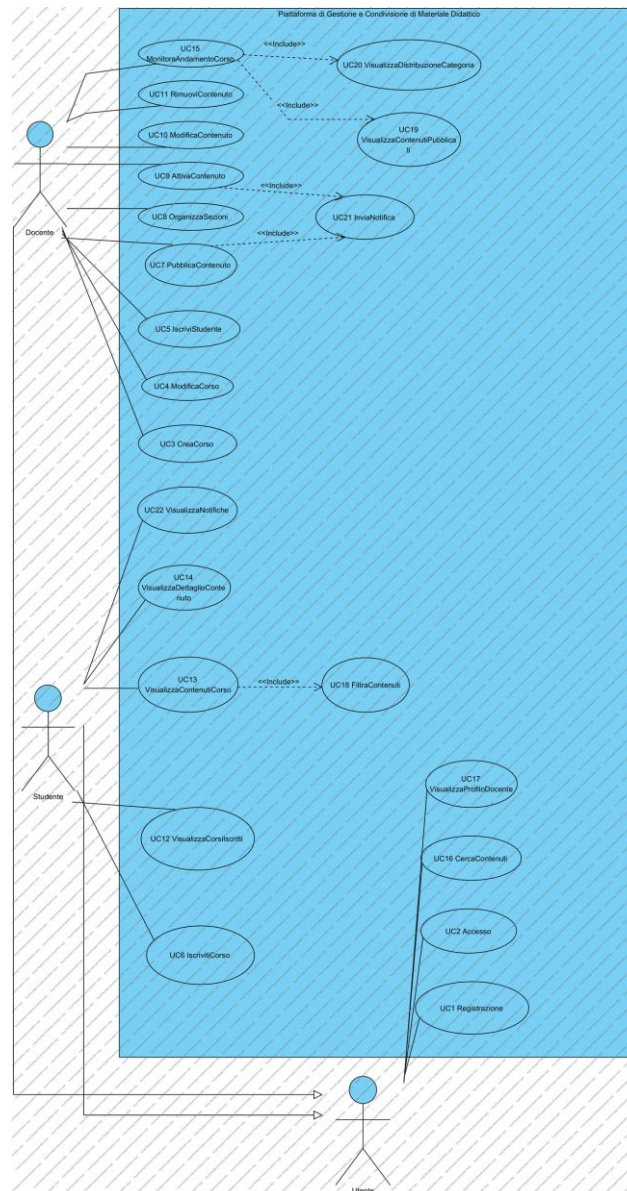


Figura 1 — Diagramma dei casi d'uso [\[apri il modello\]](#)

2.5.3 Scenari dei casi d'uso

Si riportano gli scenari dei nove casi d'uso portati fino all'implementazione. Per ciascuno sono indicati attori, descrizione, pre-condizioni, sequenza principale degli eventi, post-condizioni, casi d'uso

correlati e sequenze alternative. Gli scenari di UC9 e UC11 sono riportati in forma sintetica all'interno di §2.5.3.7, perché condividono con UC7 pre-condizioni e struttura.

2.5.3.1 Registrazione (UC1)

Caso d'uso	Registrazione (UC1)
Attore primario	Utente
Attore secondario	—
Descrizione	Un nuovo Utente si registra alla piattaforma specificando il proprio ruolo (Studente o Docente), i dati anagrafici, l'email istituzionale e una password.
Pre-condizioni	L'Utente non è ancora registrato alla piattaforma.
Sequenza di eventi principale	1. Il caso d'uso inizia quando l'Utente seleziona la funzionalità «Registrati» 2. Il sistema richiede ruolo, nome, cognome, email istituzionale e password 3. L'Utente inserisce i dati richiesti 4. Il sistema verifica che i dati siano stati specificati nel formato corretto 5. if i dati sono nel formato corretto 5.1. Il sistema verifica che l'email istituzionale non sia già associata a un account 5.2. if l'email non è già associata a un account 5.2.1. Il sistema crea il nuovo Utente con il ruolo indicato 5.2.2. Il sistema registra le credenziali dell'Utente 5.2.3. Il sistema conferma all'Utente l'avvenuta registrazione
Post-condizioni	Il nuovo Utente risulta registrato nel sistema con il ruolo indicato e può autenticarsi con le proprie credenziali.
Casi d'uso correlati	—
Sequenza di eventi alternativi	Al punto 4, se uno dei dati non rispetta il formato richiesto, il sistema restituisce un messaggio di errore e richiede all'Utente di correggerlo (torna al punto 3). Al punto 5.1, se l'email istituzionale risulta già associata a un account, il sistema restituisce un messaggio di errore e termina il caso d'uso.

Tabella 9 — Scenario del caso d'uso Registrazione (UC1)

2.5.3.2 Accesso (UC2)

Caso d'uso	Accesso (UC2)
Attore primario	Utente
Attore secondario	—
Descrizione	Un Utente registrato accede alla piattaforma inserendo le proprie credenziali personali; il sistema lo riconosce e gli presenta l'interfaccia corrispondente al proprio ruolo.
Pre-condizioni	L'Utente è già registrato alla piattaforma.
Sequenza di eventi principale	1. Il caso d'uso inizia quando l'Utente seleziona la funzionalità «Accedi» 2. Il sistema richiede email istituzionale e password 3. L'Utente inserisce le credenziali 4. Il sistema verifica che le credenziali siano state specificate nel formato corretto 5. if le credenziali sono nel formato corretto 5.1. Il sistema verifica che l'account non risulti temporaneamente bloccato 5.2. if l'account non è bloccato 5.2.1. Il sistema verifica la corrispondenza fra le credenziali inserite e quelle registrate 5.2.2. if le credenziali corrispondono 5.2.2.1. Il sistema azzerà il contatore dei tentativi falliti dell'Utente 5.2.2.2. Il sistema apre la sessione di lavoro e mostra l'interfaccia corrispondente al ruolo

Caso d'uso	Accesso (UC2)
Post-condizioni	L'Utente risulta autenticato e la sessione di lavoro è associata al suo ruolo (Studente o Docente).
Casi d'uso correlati	—
Sequenza di eventi alternativi	Al punto 4, se le credenziali non rispettano il formato richiesto, il sistema restituisce un messaggio di errore e richiede di reinserirle (torna al punto 3); il tentativo non viene conteggiato. Al punto 5.1, se l'account risulta temporaneamente bloccato, il sistema informa l'Utente del blocco e termina il caso d'uso. Al punto 5.2.2, se le credenziali non corrispondono, il sistema incrementa il contatore dei tentativi falliti e restituisce un messaggio di errore che non rivela se l'email sia registrata (RQ-SEC-02); se il contatore raggiunge il numero massimo di tentativi, il sistema blocca temporaneamente l'account (RQ-SEC-01) e ne informa l'Utente.

Tabella 10 — Scenario del caso d'uso Accesso (UC2)

2.5.3.3 IscrivitiCorso (UC6)

Caso d'uso	IscrivitiCorso (UC6)
Attore primario	Studente
Attore secondario	—
Descrizione	Uno Studente si iscrive autonomamente a un Corso inserendo il codice univoco fornito dal Docente.
Pre-condizioni	Lo Studente ha effettuato l'accesso alla piattaforma (UC2).
Sequenza di eventi principale	1. Il caso d'uso inizia quando lo Studente seleziona la funzionalità «Iscriviti a un corso» 2. Il sistema richiede allo Studente di inserire il codice univoco del Corso 3. Lo Studente inserisce il codice del Corso 4. Il sistema verifica che il codice corrisponda a un Corso esistente 5. if esiste un Corso con quel codice 5.1. Il sistema verifica che lo Studente non sia già iscritto al Corso 5.2. if lo Studente non è già iscritto al Corso 5.2.1. Il sistema crea una nuova Iscrizione fra lo Studente e il Corso 5.2.2. Il sistema aggiunge il Corso all'elenco dei corsi seguiti dallo Studente 5.2.3. Il sistema conferma allo Studente l'avvenuta iscrizione
Post-condizioni	Lo Studente risulta iscritto al Corso e il Corso compare nell'elenco dei corsi seguiti.
Casi d'uso correlati	—
Sequenza di eventi alternativi	Al punto 4, se il codice non corrisponde a nessun Corso esistente, il sistema restituisce un messaggio di errore e richiede allo Studente di reinserire il codice (torna al punto 2). Al punto 5.1, se lo Studente risulta già iscritto al Corso, il sistema restituisce un messaggio informativo e termina il caso d'uso.

Tabella 11 — Scenario del caso d'uso IscrivitiCorso (UC6)

2.5.3.4 PubblicaContenuto (UC7)

Caso d'uso	PubblicaContenuto (UC7)
Attore primario	Docente
Attore secondario	—
Descrizione	Il Docente pubblica un nuovo Contenuto didattico all'interno di un Corso, specificandone titolo, descrizione, categoria e sezione di appartenenza.

Caso d'uso	PubblicaContenuto (UC7)
Pre-condizioni	Il Docente ha effettuato l'accesso (UC2) ed è titolare del Corso.
Sequenza di eventi principale	1. Il caso d'uso inizia quando il Docente seleziona la funzionalità «Pubblica materiale» all'interno di un proprio Corso 2. Il sistema richiede titolo, descrizione, categoria, sezione (opzionale) e stato di pubblicazione del Contenuto 3. Il Docente inserisce i dati richiesti 4. Il sistema verifica che i dati obbligatori siano stati correttamente specificati 5. if i dati sono validi 5.1. Il sistema crea il nuovo Contenuto in stato di bozza e lo associa al Corso (e alla Sezione, se indicata) 5.2. if il Contenuto è impostato come visibile immediatamente 5.2.1. Il sistema porta il Contenuto nello stato «pubblicato» 5.2.2. «include» InviaNotifica 5.3. Il sistema conferma al Docente l'avvenuta pubblicazione
Post-condizioni	Il nuovo Contenuto è stato creato e associato al Corso (e alla Sezione, se indicata); se reso visibile immediatamente, gli Studenti iscritti al Corso sono stati notificati.
Casi d'uso correlati	UC21 InviaNotifica
Sequenza di eventi alternativi	Al punto 4, se i dati obbligatori non sono stati specificati correttamente, il sistema restituisce un messaggio di errore e richiede al Docente di correggerli (torna al punto 3). Al punto 5.1, se la Sezione indicata non appartiene al Corso, il sistema restituisce un messaggio di errore e termina il caso d'uso.

Tabella 12 — Scenario del caso d'uso PubblicaContenuto (UC7)

2.5.3.5 VisualizzaContenutiCorso (UC13)

Caso d'uso	VisualizzaContenutiCorso (UC13)
Attore primario	Studente
Attore secondario	—
Descrizione	Lo Studente consulta l'elenco dei materiali didattici pubblicati in un Corso a cui è iscritto, eventualmente filtrandoli per categoria, data o sezione.
Pre-condizioni	Lo Studente ha effettuato l'accesso (UC2) ed è iscritto al Corso.
Sequenza di eventi principale	1. Il caso d'uso inizia quando lo Studente seleziona un Corso a cui è iscritto 2. Il sistema verifica che lo Studente sia effettivamente iscritto al Corso 3. Il sistema mostra l'elenco dei Contenuti pubblicati nel Corso 4. Lo Studente può impostare uno o più filtri (categoria, data, sezione) 5. if lo Studente ha impostato dei filtri 5.1. «include» FiltraContenuti 5.2. Il sistema mostra l'elenco dei Contenuti filtrato 6. Lo Studente seleziona un Contenuto per consultarne i dettagli 7. Il sistema mostra titolo, descrizione, categoria e data di pubblicazione del Contenuto selezionato
Post-condizioni	Lo Studente ha visualizzato l'elenco (eventualmente filtrato) dei Contenuti del Corso e, se richiesto, il dettaglio di un Contenuto selezionato.
Casi d'uso correlati	UC18 FiltraContenuti, UC14 VisualizzaDettaglioContenuto
Sequenza di eventi alternativi	Al punto 2, se lo Studente non risulta iscritto al Corso, il sistema restituisce un messaggio di errore e termina il caso d'uso. Al punto 5.2, se nessun Contenuto soddisfa i filtri impostati, il sistema mostra un messaggio informativo e un elenco vuoto.

Tabella 13 — Scenario del caso d'uso VisualizzaContenutiCorso (UC13)

2.5.3.6 MonitoraAndamentoCorso (UC15)

Caso d'uso	MonitoraAndamentoCorso (UC15)
Attore primario	Docente
Attore secondario	—
Descrizione	Il Docente consulta le statistiche di andamento di un proprio Corso: numero di materiali pubblicati, numero di Studenti iscritti e categorie di Contenuti più utilizzate.
Pre-condizioni	Il Docente ha effettuato l'accesso (UC2) ed è titolare del Corso.
Sequenza di eventi principale	1. Il caso d'uso inizia quando il Docente seleziona un proprio Corso e richiede la funzionalità «Andamento corso» 2. Il sistema verifica che il Docente sia titolare del Corso 3. «include» VisualizzaContenutiPubblicati 4. Il sistema calcola il numero totale di Contenuti pubblicati nel Corso 5. Il sistema calcola il numero di Studenti iscritti al Corso 6. «include» VisualizzaDistribuzioneCategoria 7. Il sistema calcola la distribuzione dei Contenuti per Categoria 8. Il sistema mostra al Docente il riepilogo dell'andamento del Corso
Post-condizioni	Il Docente ha visualizzato il riepilogo aggiornato dell'andamento del Corso.
Casi d'uso correlati	UC19 VisualizzaContenutiPubblicati, UC20 VisualizzaDistribuzioneCategoria
Sequenza di eventi alternativi	Al punto 2, se il Docente non è titolare del Corso, il sistema restituisce un messaggio di errore e termina il caso d'uso. Ai punti 4-7, se il Corso non ha ancora Contenuti pubblicati né Studenti iscritti, i valori calcolati risultano pari a zero; il sistema mostra comunque il riepilogo al punto 8.

Tabella 14 — Scenario del caso d'uso MonitoraAndamentoCorso (UC15)

2.5.3.7 VisualizzaNotifiche (UC22)

Caso d'uso	VisualizzaNotifiche (UC22)
Attore primario	Studente
Attore secondario	—
Descrizione	Lo Studente consulta le Notifiche ricevute a seguito della pubblicazione di nuovo materiale nei Corsi a cui è iscritto.
Pre-condizioni	Lo Studente ha effettuato l'accesso (UC2).
Sequenza di eventi principale	1. Il caso d'uso inizia quando lo Studente seleziona la funzionalità «Notifiche» 2. Il sistema recupera le Notifiche destinate allo Studente, ordinate dalla più recente 3. Il sistema mostra allo Studente l'elenco delle Notifiche con testo, Corso di riferimento e data 4. Il sistema marca come lette le Notifiche mostrate
Post-condizioni	Lo Studente ha visualizzato le proprie Notifiche, che risultano marcate come lette.
Casi d'uso correlati	—
Sequenza di eventi alternativi	Al punto 2, se non esistono Notifiche destinate allo Studente, il sistema mostra un messaggio informativo e un elenco vuoto.

Tabella 15 — Scenario del caso d'uso VisualizzaNotifiche (UC22)

2.5.3.8 AttivaContenuto (UC9) e RimuoviContenuto (UC11)

I due casi d'uso condividono con UC7 attore, pre-condizioni e struttura, e agiscono sullo stesso oggetto: lo stato di pubblicazione di un Contenuto. Se ne riporta lo scenario in forma compatta, rimandando al diagramma di stato del §2.7.7 per la descrizione completa delle transizioni ammesse.

	AttivaContenuto (UC9)	RimuoviContenuto (UC11)
Attore primario	Docente	Docente
Descrizione	Il Docente rende visibile agli Studenti un Contenuto precedentemente mantenuto in bozza	Il Docente rimuove un Contenuto, che cessa di essere visibile agli Studenti
Pre-condizioni	Il Docente ha effettuato l'accesso, è titolare del Corso e il Contenuto è in stato di bozza	Il Docente ha effettuato l'accesso, è titolare del Corso e il Contenuto non è già rimosso
Sequenza principale	1. Il Docente seleziona un Contenuto in bozza e richiede «Attiva» 2. Il sistema verifica che la transizione sia ammessa dallo stato corrente 3. Il sistema porta il Contenuto nello stato «pubblicato» 4. «include» InviaNotifica 5. Il sistema conferma l'avvenuta attivazione	1. Il Docente seleziona un Contenuto e richiede «Rimuovi» 2. Il sistema verifica che la transizione sia ammessa dallo stato corrente 3. Il sistema porta il Contenuto nello stato «rimosso» 4. Il sistema conferma l'avvenuta rimozione
Post-condizioni	Il Contenuto è visibile agli Studenti iscritti, che sono stati notificati	Il Contenuto non è più visibile agli Studenti
Alternativi	Al punto 2, se il Contenuto è già pubblicato o rimosso, il sistema segnala che l'operazione non è ammessa e termina	Al punto 2, se il Contenuto è già rimosso, il sistema segnala che l'operazione non è ammessa e termina

Tabella 16 — Scenari sintetici dei casi d'uso UC9 e UC11

2.6 Diagramma delle classi di analisi

Il diagramma delle classi di analisi rappresenta i concetti del dominio individuati al §2.1 e le loro relazioni, senza alcuna scelta implementativa: non compaiono né tipi di dato del linguaggio, né classi di supporto, né decisioni sulla persistenza. Gli attributi derivano dai requisiti sui dati RD01-RD09.

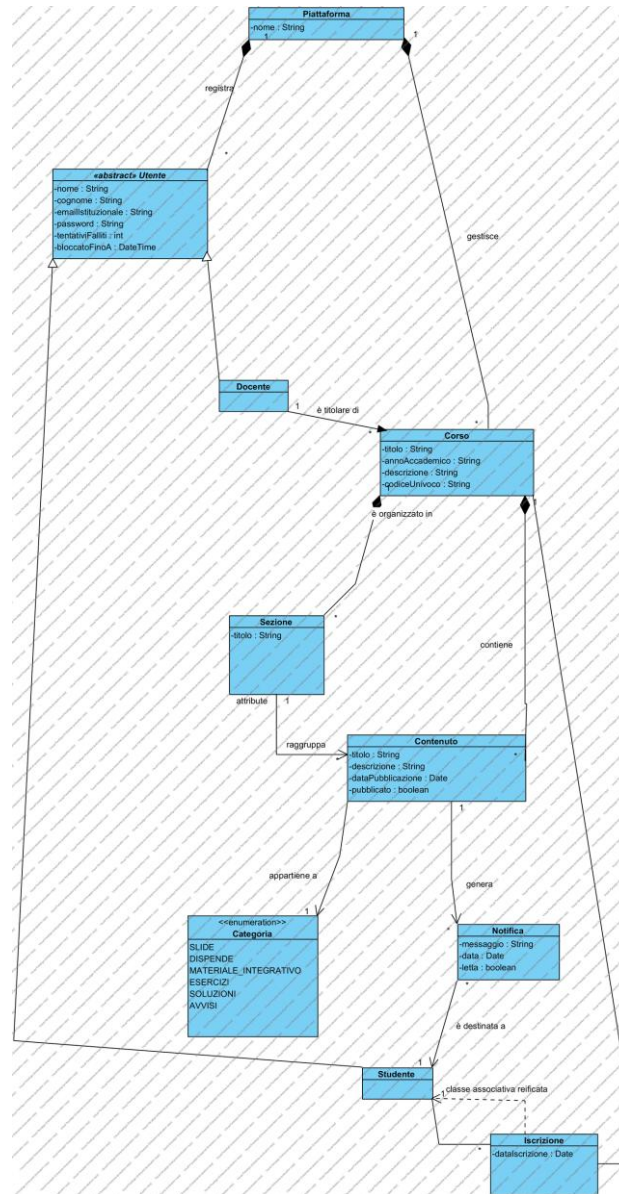


Figura 2 — Diagramma delle classi di analisi [\[apri il modello\]](#)

La gerarchia Utente → {Studente, Docente} traduce RD01 e la generalizzazione fra attori del §2.5.2: nome, cognome, email, password e ruolo sono comuni, mentre le associazioni sono specifiche di ciascuna specializzazione. Gli attributi tentativiFalliti e bloccatoFinoA su Utente traducono RD08 e sono ciò che rende verificabile RQ-SEC-01.

L'Iscrizione è modellata come classe e non come semplice associazione molti-a-molti perché RD03 chiede di memorizzarne dati propri (lo Studente e il Corso associati, e la data in cui l'iscrizione è avvenuta): un'associazione senza attributi non sarebbe in grado di rappresentarla. Analogamente la Notifica è una classe, perché RD09 le attribuisce testo, data e stato di lettura.

Nota UML apposta al diagramma. «La classe *Piattaforma* non è un concetto del dominio: raccoglie le responsabilità di sistema che in questa fase non appartengono a nessuna entità. Si veda §2.9 per la sua trasformazione in progettazione.» La nota è riportata anche sul modello *Visual Paradigm*, accanto alla classe.

2.6.1 Distribuzione delle responsabilità in analisi

Prima di passare ai diagrammi di sequenza si assegna a ogni funzionalità la classe che ne è responsabile, applicando il pattern GRASP Information Expert: la responsabilità va alla classe che possiede le informazioni necessarie a soddisfarla. Le responsabilità che non trovano un esperto naturale nel dominio restano temporaneamente su *Piattaforma*.

Responsabilità	Classe responsabile	Motivazione (Information Expert)
Registrazione di un nuovo Utente	Piattaforma	Nessuna entità del dominio possiede l'insieme degli Utenti registrati
Accesso alla piattaforma	Piattaforma	Come sopra: occorre cercare l'Utente fra tutti quelli registrati
Verifica delle proprie credenziali	Utente	L'Utente possiede la propria password: è l'esperto naturale
Creazione di un Corso	Docente	Il Docente possiede l'elenco dei Corsi di cui è titolare
Iscrizione a un Corso	Studente	Lo Studente possiede l'elenco delle proprie Iscrizioni
Pubblicazione di un Contenuto	Corso	Il Corso possiede l'elenco dei propri Contenuti e delle proprie Sezioni
Organizzazione in Sezioni	Corso	Il Corso possiede le proprie Sezioni
Consultazione dei Contenuti di un Corso	Corso	Il Corso possiede i propri Contenuti
Monitoraggio dell'andamento di un Corso	Corso	I dati da aggregare (Contenuti, Iscrizioni) appartengono al Corso
Invio di una Notifica	Piattaforma	Il recapito del messaggio è un servizio di sistema, non una funzione del dominio

Tabella 17 — Distribuzione delle responsabilità nel modello di analisi

2.7 Diagrammi di sequenza di analisi

Per ciascun caso d'uso sviluppato si riporta il diagramma di sequenza che ne rappresenta lo scenario principale, con l'attore, l'oggetto di sistema e gli oggetti del dominio coinvolti. In questa fase i messaggi sono espressi nel linguaggio del dominio, non ancora come firme di metodo: la loro traduzione in operazioni è oggetto del §2.8.

2.7.1 Registrazione (UC1)

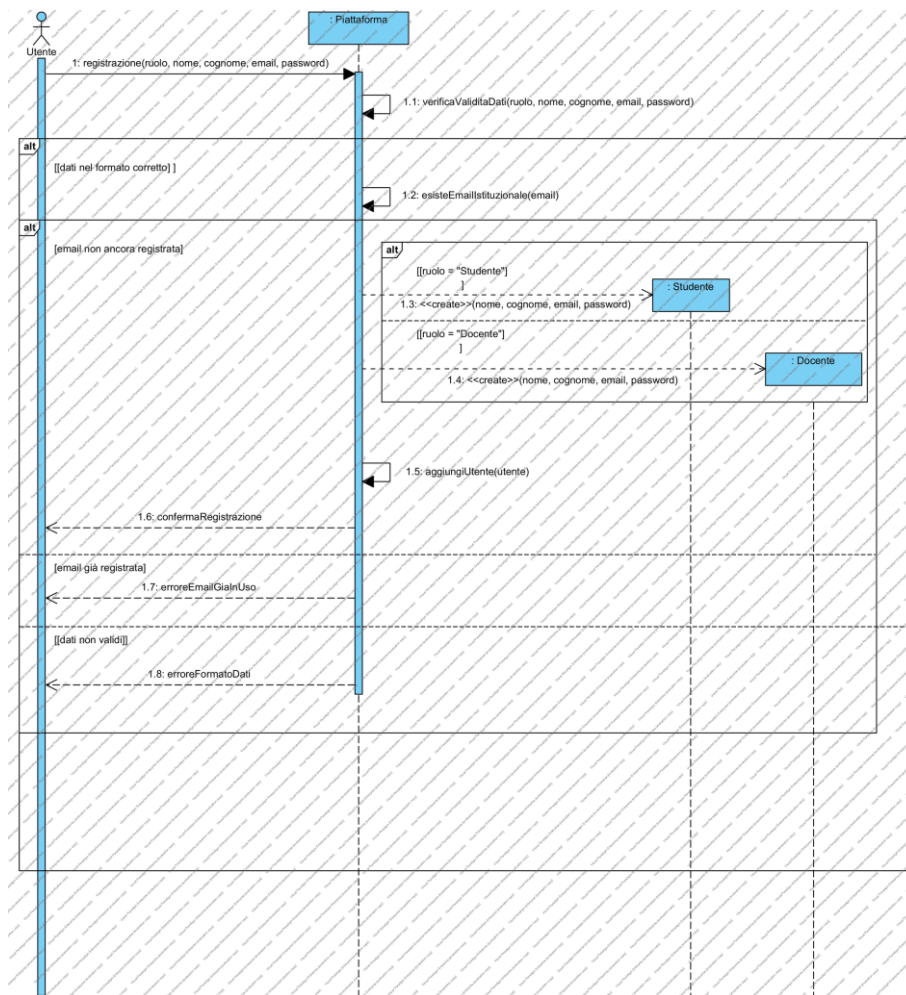


Figura 3 — Diagramma di sequenza di analisi — Registrazione (UC1) [\[apri il modello\]](#)

Il diagramma mostra il controllo di unicità dell'email (RD07) prima della creazione dell'Utente: il frame alt distingue il caso in cui l'email risulti già registrata, che termina il caso d'uso, dal caso nominale. La creazione dell'oggetto Utente è rappresentata con un messaggio «create» diretto sulla linea di vita, secondo la notazione UML per l'istanziamento.

2.7.2 Accesso (UC2)

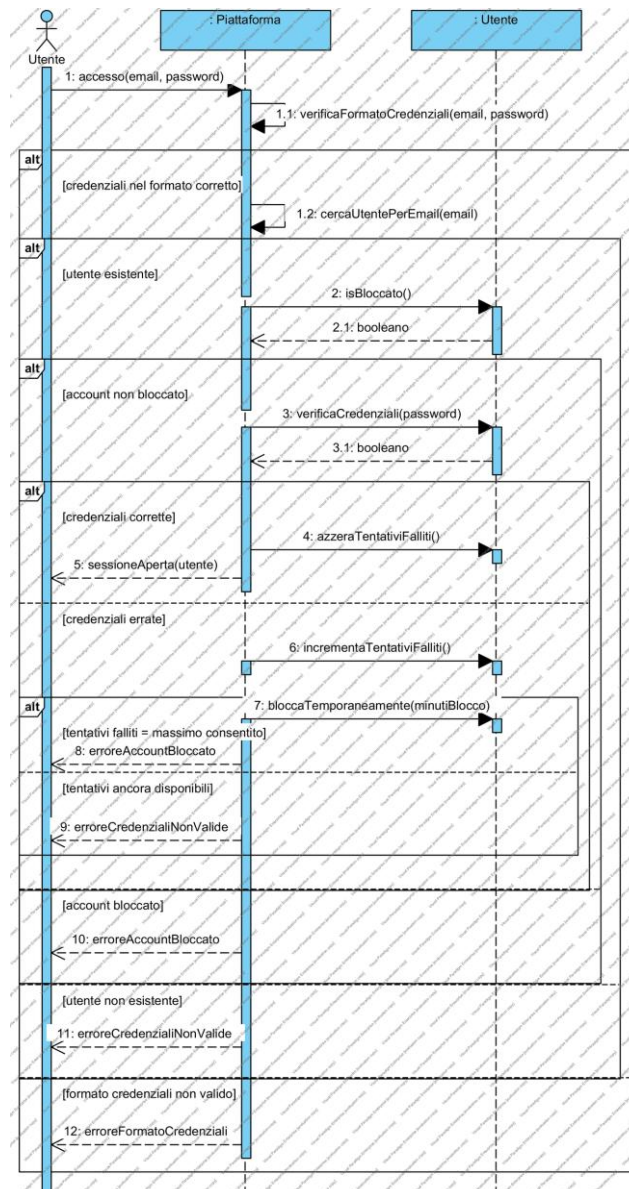


Figura 4 — Diagramma di sequenza di analisi — Accesso (UC2) [\[apri il modello\]](#)

È il diagramma su cui si legge l'applicazione dell'Information Expert: la verifica delle credenziali non è svolta da Piattaforma ma delegata all'oggetto Utente, che è l'unico a possedere la propria password. Lo stesso vale per il controllo del blocco e per l'aggiornamento del contatore dei tentativi falliti. I tre frame alt annidati corrispondono ai tre punti di decisione dello scenario: formato delle credenziali, account bloccato, credenziali corrispondenti.

2.7.3 IscrivitiCorso (UC6)

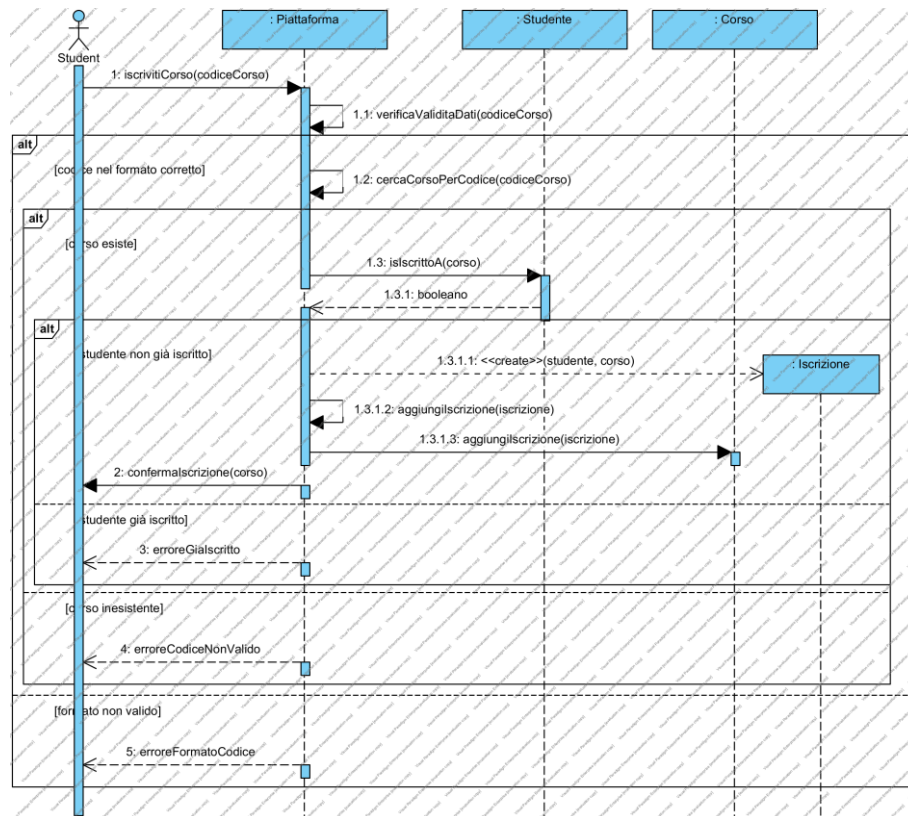


Figura 5 — Diagramma di sequenza di analisi — *IscrivitiCorso (UC6)* [\[apri il modello\]](#)

La ricerca del Corso per codice è responsabilità di Piattaforma, che è l'unica a conoscere l'insieme dei Corsi; la verifica di non-duplicazione dell'iscrizione è invece delegata allo Studente, che possiede l'elenco delle proprie Iscrizioni. La creazione dell'Iscrizione avviene solo dopo che entrambi i controlli hanno avuto esito positivo.

2.7.4 PubblicaContenuto (UC7)

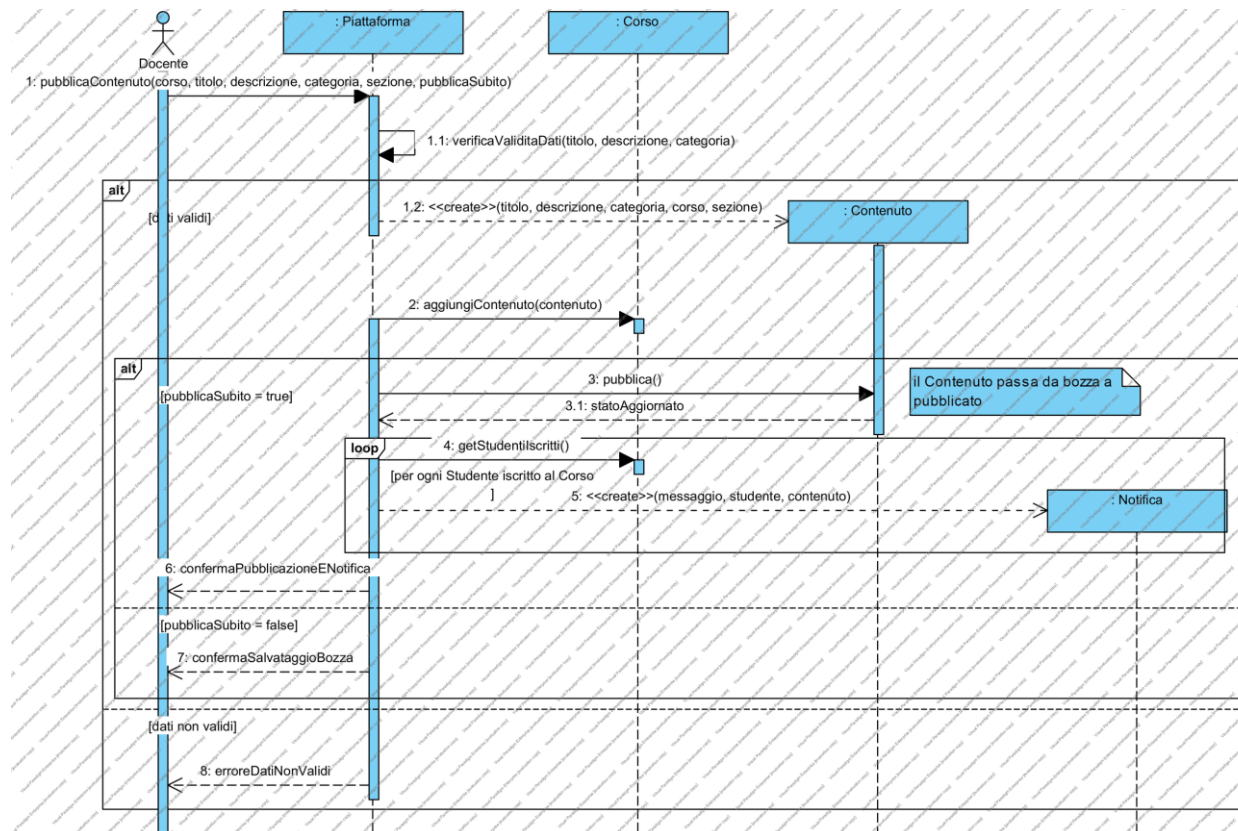


Figura 6 — Diagramma di sequenza di analisi — *PubblicaContenuto (UC7)* [\[apri il modello\]](#)

Il diagramma rende esplicito che la pubblicazione è composta da due passi distinti: la creazione del Contenuto, che nasce sempre in stato di bozza, e l'eventuale passaggio allo stato «pubblicato», che è ciò che innesca la notifica agli iscritti. Questa separazione è la ragione per cui UC9 (attivazione differita) può riutilizzare lo stesso meccanismo di notifica senza duplicarlo.

2.7.5 VisualizzaContenutiCorso (UC13)

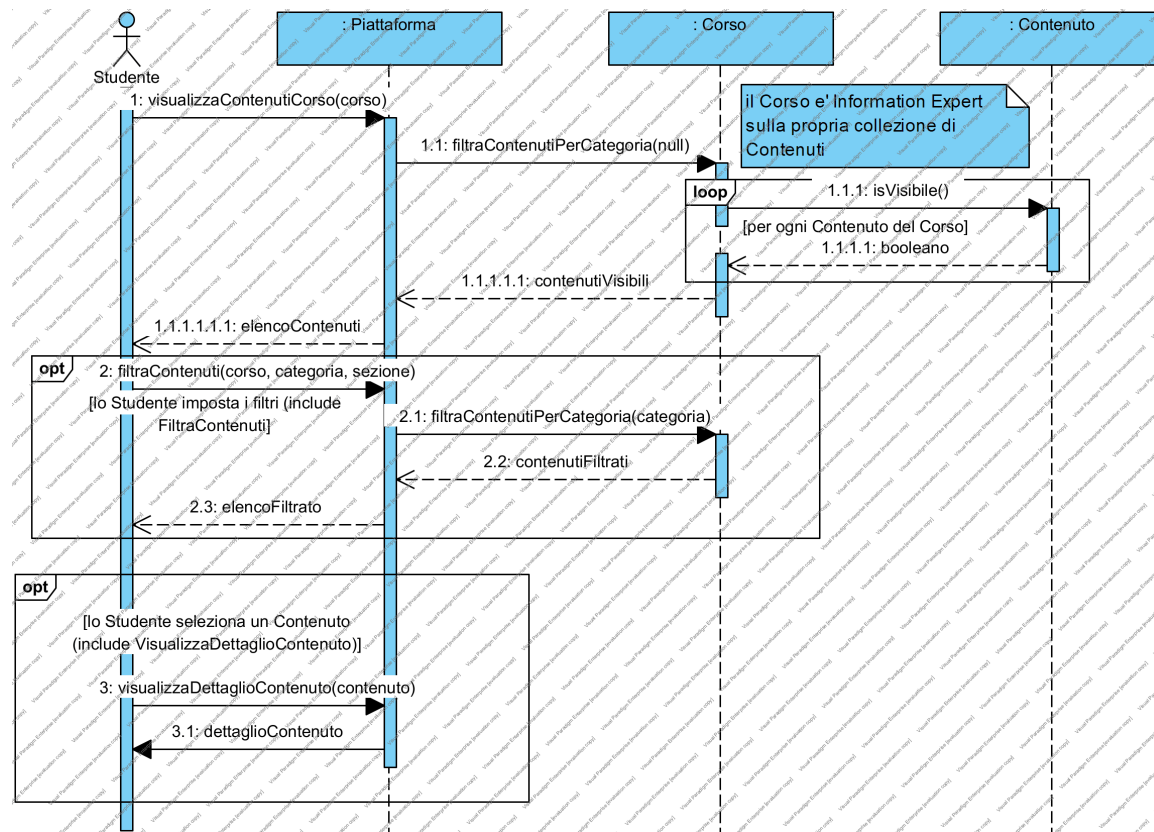


Figura 7 — Diagramma di sequenza di analisi — VisualizzaContenutiCorso (UC13) [\[apri il modello\]](#)

La verifica dell'iscrizione precede qualunque lettura: è il controllo di autorizzazione richiesto da RQ-QUAL-02, che nella prima versione dell'elaborato non era realizzabile perché il sistema non sapeva chi fosse l'Utente collegato. Il filtro è rappresentato come frame opt, coerentemente con la relazione «include» condizionata verso UC18.

2.7.6 MonitoraAndamentoCorso (UC15)

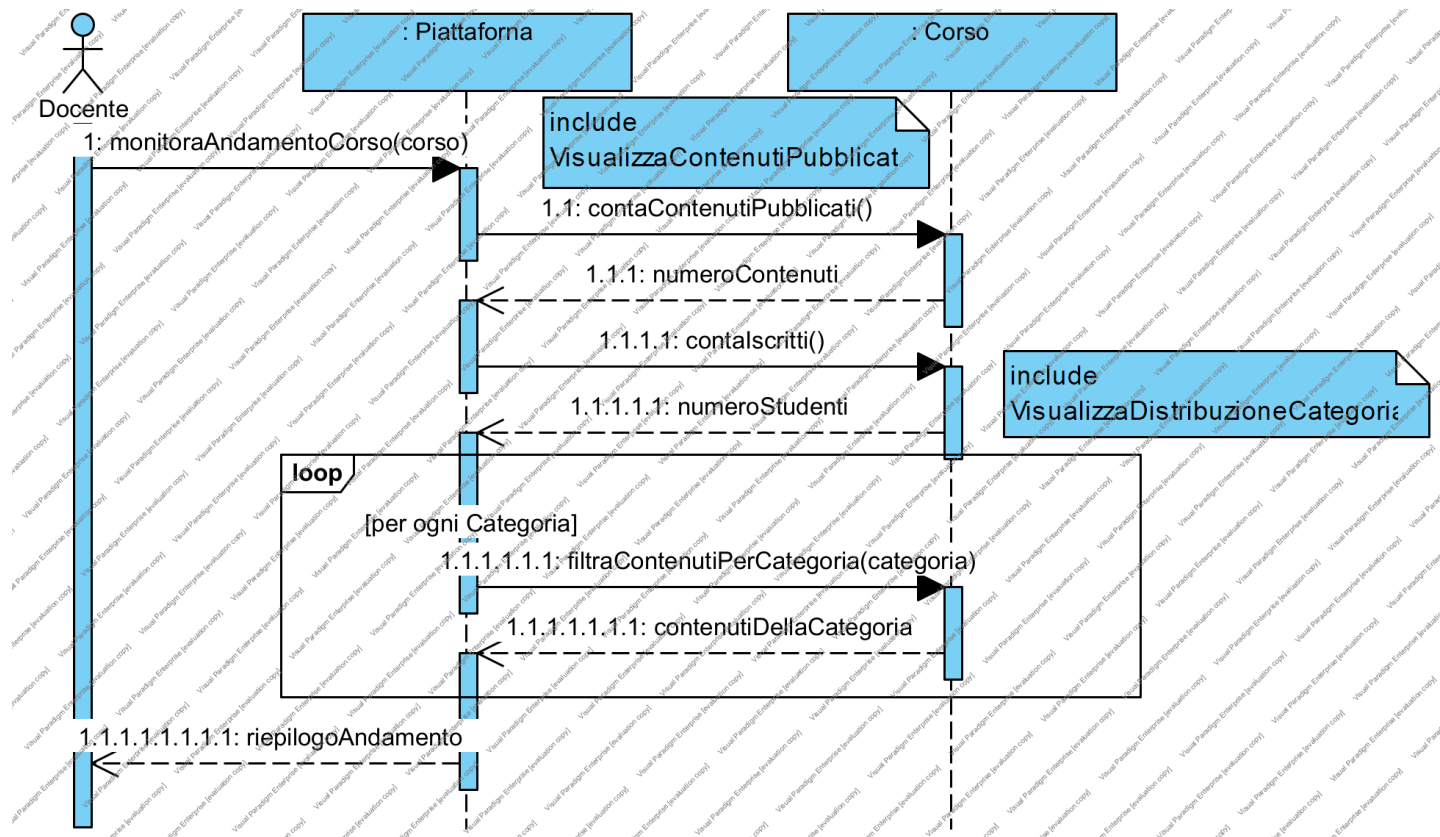


Figura 8 — Diagramma di sequenza di analisi — MonitoraAndamentoCorso (UC15) [\[apri il modello\]](#)

I tre conteggi sono tutti delegati al Corso, che è l'esperto delle informazioni da aggregare. Il risultato è un riepilogo unico restituito all'attore: la scelta di restituire un solo oggetto invece di tre valori separati anticipa il DTO di riepilogo che si trova in progettazione.

2.7.7 Diagramma di stato del Contenuto

Lo stato di pubblicazione richiesto da RD04 non è un semplice attributo booleano: le frasi 16 e 18 della traccia descrivono un ciclo di vita con transizioni ammesse e transizioni vietate. Il diagramma di stato lo formalizza.

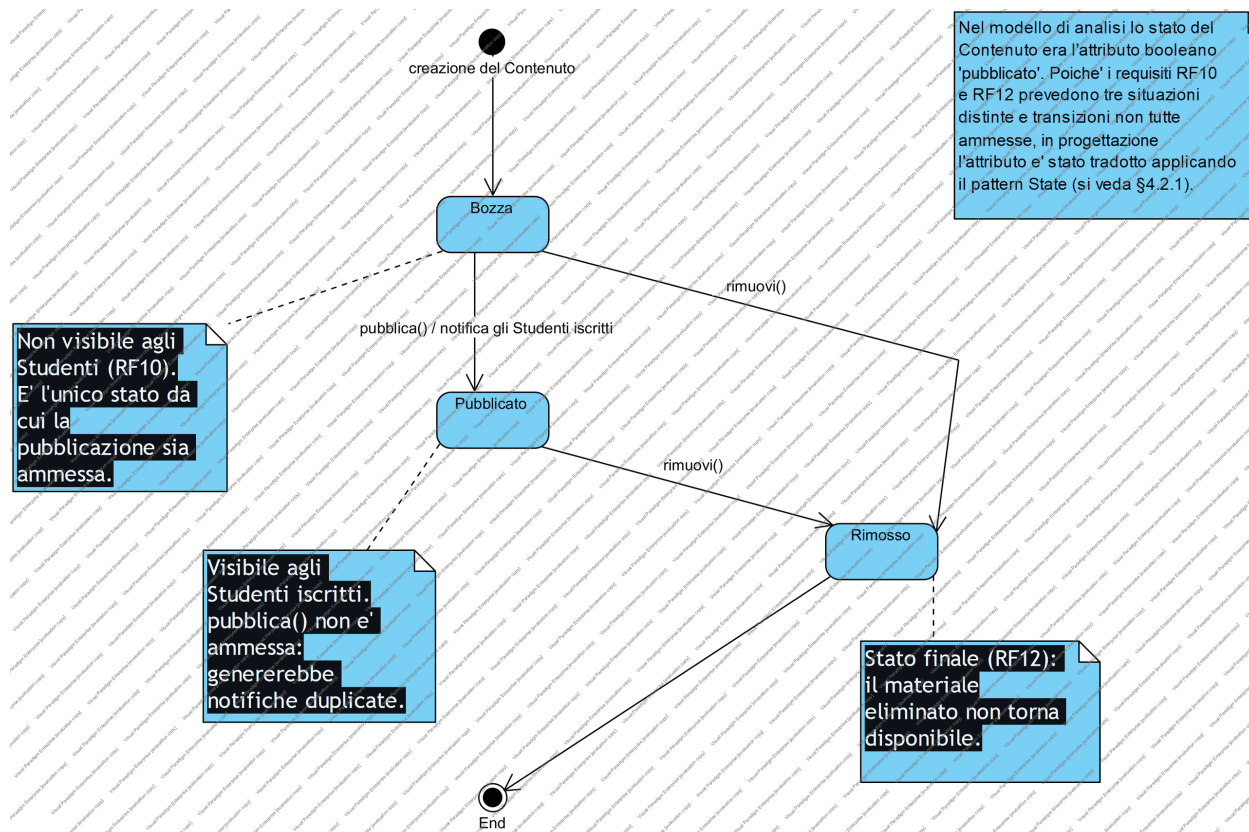


Figura 9 — Diagramma di stato del Contenuto [\[apri il modello\]](#)

Un Contenuto nasce in stato Bozza. La transizione pubblica() lo porta in Pubblicato ed è l'evento che rende il materiale visibile agli Studenti; la transizione rimuovi() porta in Rimosso, che è uno stato finale da cui non si esce. Sono vietate: pubblicare un Contenuto già pubblicato, pubblicare un Contenuto rimosso, rimuovere un Contenuto già rimosso.

Nota UML apposta al diagramma. *«Il ciclo di vita qui rappresentato è realizzato in progettazione con il design pattern State: si vedano §4.3.4 e il package entity.stato.» È questo diagramma a giustificare la scelta del pattern, e non viceversa.*

2.8 Distribuzione delle responsabilità dopo i diagrammi di sequenza

I diagrammi di sequenza hanno fatto emergere, per ogni classe, le operazioni che essa deve saper eseguire per portare a termine gli scenari. La tabella seguente aggiorna quella del §2.6.1 con i metodi individuati: è il passaggio che trasforma il diagramma delle classi di analisi in un modello completo, e senza il quale il salto verso la progettazione risulterebbe ingiustificato.

Classe	Responsabilità confermate	Operazioni emerse dai diagrammi di sequenza
Piattaforma	Registrazione, Accesso, ricerca di Utenti e Corsi, invio delle Notifiche	registraUtente(), autentica(), cercaUtentePerEmail(), cercaCorsoPerCodice(), inviaNotifica()
Utente	Verifica delle proprie credenziali e del proprio stato di blocco	verificaCredenziali(), isBloccato(), incrementaTentativiFalliti(), azzeraTentativiFalliti(), bloccaTemporaneamente()
Studente	Gestione delle proprie Iscrizioni	isIscrittoA(), aggiungiIscrizione()
Docente	Titolarità dei Corsi	isTitolareDi(), aggiungiCorso()
Corso	Gestione dei propri Contenuti e Sezioni, statistiche	aggiungiContenuto(), contenutiPubblicati(), contalscritti(), distribuzionePerCategoria(), contieneSezione()
Contenuto	Gestione del proprio stato di pubblicazione	pubblica(), rimuovi(), isVisible()
Sezione	Appartenenza al Corso	appartieneA()
Iscrizione	Legame Studente-Corso	— (classe associativa, priva di comportamento proprio)
Notifica	Stato di lettura	marcaComeLetta()

Tabella 18 — Responsabilità e operazioni emerse dai diagrammi di sequenza

Perché questa tabella è importante. È il punto in cui si vede che la verifica delle credenziali è passata da *Piattaforma* a *Utente*, e che *Piattaforma* è rimasta con sole responsabilità di ricerca e di coordinamento. Chi legge il diagramma delle classi di progettazione e non trova più *Piattaforma*, trova qui l'elenco esatto di ciò che quella classe faceva, e al §4.2.1 la destinazione di ciascuna voce.

2.9 Diagramma delle classi di analisi raffinato

Il diagramma delle classi di analisi viene ora aggiornato con le operazioni emerse al §2.8. È l'ultimo artefatto dell'analisi e il primo ingresso della progettazione: contiene tutto ciò che il sistema deve saper fare, ancora espresso nel linguaggio del dominio e ancora privo di qualunque scelta tecnologica.

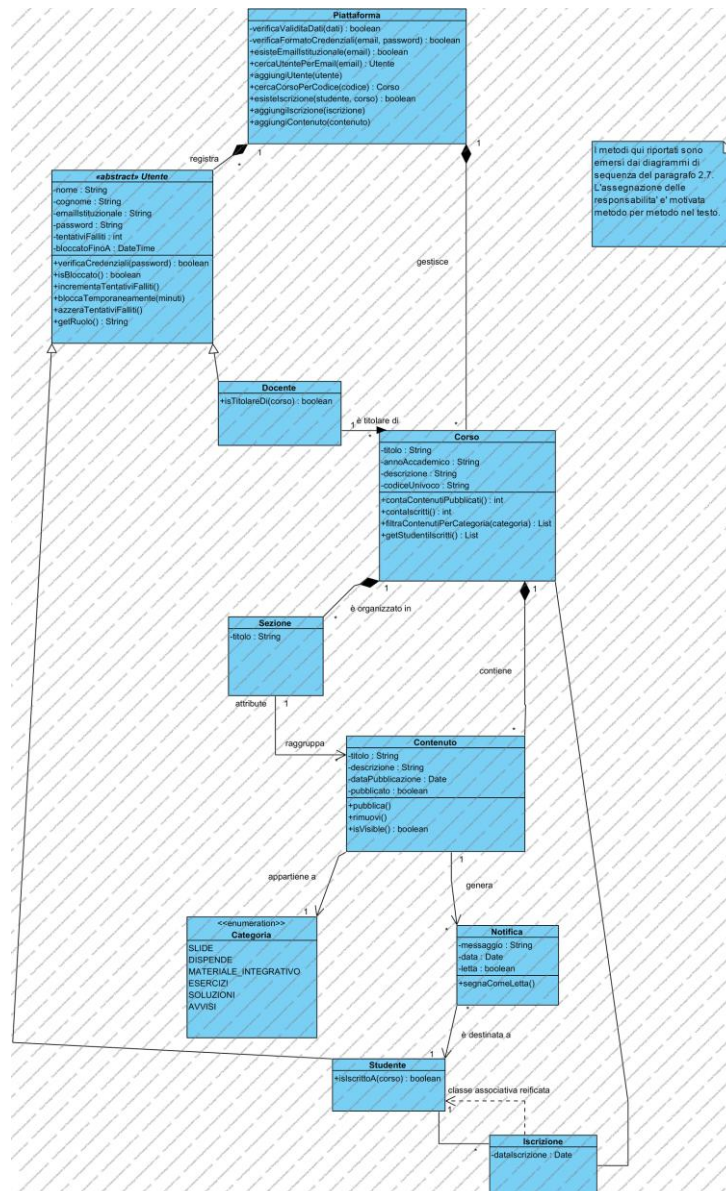


Figura 10 — Diagramma delle classi di analisi raffinato [\[apri il modello\]](#)

Rispetto al diagramma del §2.6 sono cambiate tre cose. Le classi hanno ora un comportamento, e non soltanto dati. La verifica delle credenziali è collocata su Utente, dove il §2.7.2 l'ha portata. L'attributo booleano che rappresentava lo stato di pubblicazione è stato sostituito dalle tre operazioni pubblica(), rimuovi() e isVisibile(), coerenti con il diagramma di stato del §2.7.7.

2.9.1 La trasformazione della classe Piattaforma

La classe Piattaforma è l'unico elemento del modello di analisi che non sopravvive alla progettazione, ed è quindi l'elemento che richiede la spiegazione più esplicita: un lettore che confronti il diagramma delle classi di analisi con quello di progettazione non deve chiedersi dove sia finita. Il diagramma seguente, allegato in Visual Paradigm come sotto-diagramma della classe stessa, mostra la destinazione di ciascuna delle sue responsabilità.

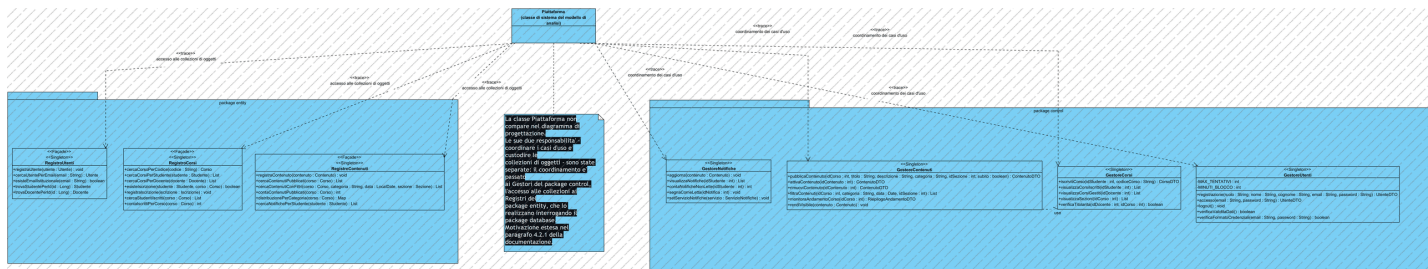


Figura 11 — Trasformazione della classe Piattaforma nelle classi di progettazione [\[apri il modello\]](#)

Le responsabilità di coordinamento dei casi d'uso migrano nel package control, distribuite fra quattro Gestori secondo l'area funzionale; le responsabilità di ricerca e di garanzia dell'unicità migrano nel package entity, sui tre Registri; la responsabilità di recapito del messaggio, che nel dominio non ha un titolare, diventa un'interfaccia realizzata da un adattatore verso un servizio esterno. Nessuna responsabilità viene persa e nessuna viene inventata: la corrispondenza è biunivoca ed è tabulata al §4.2.1.

Nota UML apposta al diagramma. *«Questo diagramma è allegato come sotto-diagramma alla classe Piattaforma del diagramma delle classi di analisi. Documenta la ricollocazione delle sue responsabilità nel modello di progettazione (§4.2.1).»*

3. Piano di test funzionale

Il piano di test funzionale è stato costruito con la tecnica della Category Partition (lezione 18): per ogni caso d'uso si individuano i parametri che ne determinano il comportamento, per ciascun parametro si partiziona il dominio dei valori in classi di equivalenza, e da ogni classe si estrae un rappresentante. Le classi che descrivono situazioni di errore sono marcate [ERROR] e generano un caso di test dedicato; le classi valide vengono combinate fra loro per produrre i casi nominali.

La strategia adottata è quella dell'«una scelta per volta»: un caso di test nominale che combina tutte le classi valide, e poi un caso per ogni classe [ERROR], mantenendo validi i restanti parametri. È la strategia che minimizza il numero di casi mantenendo la copertura di tutte le classi, ed evita l'esplosione combinatoria del prodotto cartesiano.

Gli esiti dell'esecuzione dei casi qui progettati sono riportati al §6.3.

3.1 Registrazione (UC1)

RUOLO	NOME E COGNOME	EMAIL	PASSWORD
Ruolo selezionato fra Studente e Docente Nessun ruolo selezionato [ERROR]	Entrambi non vuoti e di lunghezza ≤ 50 Almeno uno vuoto [ERROR]	Formato valido e dominio unina.it Formato non valido [ERROR] Dominio diverso da unina.it [ERROR] Formato valido ma già registrata [ERROR]	Lunghezza ≥ 8 , almeno una maiuscola e una cifra Lunghezza < 8 [ERROR] Priva di maiuscola o di cifra [ERROR]

Tabella 19 — Classi di equivalenza — Registrazione

TC	Descrizione	Classi coperte	Input	Output atteso	Post-condizioni attese
TC1	Tutti input validi, ruolo Studente	Ruolo valido, dati validi, email nuova, password valida	{ruolo: STUDENTE, nome: "Mario", cognome: "Rossi", email: "m.rossi@studenti.unina.it", password: "Password01"}	Registrazione conclusa; messaggio di conferma	Il nuovo Studente risulta registrato e può autenticarsi
TC2	Tutti input validi, ruolo Docente	Ruolo valido (Docente), restanti validi	{ruolo: DOCENTE, email: "l.verdi@unina.it", password: "Password01"}	Registrazione conclusa; messaggio di conferma	Il nuovo Docente risulta registrato
TC3	Nome vuoto	Nome/cognome [ERROR]	{nome: "", restanti validi}	Messaggio di errore: dati obbligatori mancanti	Nessun Utente viene creato
TC4	Email di formato non valido	Email formato [ERROR]	{email: "mario.rossi", restanti validi}	Messaggio di errore: formato email non valido	Nessun Utente viene creato
TC5	Email di dominio non istituzionale	Email dominio [ERROR]	{email: "m.rossi@gmail.com", restanti validi}	Messaggio di errore: è richiesta un'email istituzionale	Nessun Utente viene creato

TC	Descrizione	Classi coperte	Input	Output atteso	Post-condizioni attese
TC6	Email già registrata	Email duplicata [ERROR]	{email: "a.fasolino@unina.it" (già presente), restanti validi}	Messaggio di errore: email già associata a un account	Nessun Utente viene creato (RD07 rispettato)
TC7	Password troppo corta	Password lunghezza [ERROR]	{password: "Pass01", restanti validi}	Messaggio di errore: password non conforme	Nessun Utente viene creato
TC8	Password priva di cifra	Password composizione [ERROR]	{password: "Passwordd", restanti validi}	Messaggio di errore: password non conforme	Nessun Utente viene creato

Tabella 20 — Casi di test — Registrazione (UC1)

3.2 Accesso (UC2)

FORMATO DELLE CREDENZIALI	ESISTENZA DELL'ACCOUNT	STATO DELL'ACCOUNT	CORRISPONDENZA PASSWORD
Email e password non vuote e di formato valido Email vuota o di formato non valido [ERROR] Password vuota [ERROR]	Email corrispondente a un Utente registrato Email non corrispondente ad alcun Utente [ERROR]	Account non bloccato Account temporaneamente bloccato [ERROR]	Password corrispondente Password non corrispondente [ERROR]

Tabella 21 — Classi di equivalenza — Accesso

TC	Descrizione	Classi coperte	Input	Output atteso	Post-condizioni attese
TC1	Accesso valido di uno Studente	Formato valido, account esistente, non bloccato, password corretta	{email: "g.aliperta@studenti.unina.it", password: "Password01"}	Sessione aperta; viene mostrata l'area Studente	L'Utente risulta autenticato con ruolo Studente
TC2	Accesso valido di un Docente	Come TC1 con ruolo Docente	{email: "a.fasolino@unina.it", password: "Password01"}	Sessione aperta; viene mostrata l'area Docente	L'Utente risulta autenticato con ruolo Docente
TC3	Email vuota	Formato [ERROR]	{email: "", password: "Password01"}	Messaggio di errore: credenziali non valide	Nessuna sessione aperta; il tentativo non è conteggiato
TC4	Password vuota	Formato [ERROR]	{email: "g.aliperta@studenti.unina.it", password: ""}	Messaggio di errore: credenziali non valide	Nessuna sessione aperta; il tentativo non è conteggiato
TC5	Email non registrata	Esistenza [ERROR]	{email: "nessuno@unina.it", password: "Password01"}	Messaggio generico «credenziali non	Nessuna sessione aperta

TC	Descrizione	Classi coperte	Input	Output atteso	Post-condizioni attese
				valide», identico a TC6 (RQ-SEC-02)	
TC6	Password errata	Corrispondenza [ERROR]	{email: "g.aliperta@studenti.unina.it", password: "Sbagliata01"}	Messaggio generico «credenziali non valide», identico a TC5	Contatore dei tentativi falliti incrementato di 1
TC7	Quinto tentativo fallito consecutivo	Corrispondenza [ERROR] ripetuta	5 tentativi consecutivi con password errata	Messaggio: account bloccato per 15 minuti	L'account risulta bloccato (RQ-SEC-01)
TC8	Accesso con account bloccato	Stato account [ERROR]	{credenziali corrette su account bloccato}	Messaggio: account temporaneamente bloccato	Nessuna sessione aperta anche con password corretta

Tabella 22 — Casi di test — Accesso (UC2)

3.3 IscrivitiCorso (UC6)

CODICE CORSO	CORSO CORRISPONDENTE	STATO ISCRIZIONE
Stringa alfanumerica di lunghezza 8-12 Stringa vuota [ERROR] Stringa con caratteri non alfanumerici [ERROR] Stringa di lunghezza fuori intervallo [ERROR]	Il codice corrisponde a un Corso esistente Il codice non corrisponde ad alcun Corso [ERROR]	Lo Studente non è ancora iscritto al Corso Lo Studente è già iscritto al Corso [ERROR]

Tabella 23 — Classi di equivalenza — IscrivitiCorso

TC	Descrizione	Classi coperte	Input	Output atteso	Post-condizioni attese
TC1	Tutti input validi	Codice valido, Corso esistente, non iscritto	{codiceCorso: "CINF2024B"}	Iscrizione creata; messaggio di conferma	Lo Studente risulta iscritto al Corso
TC2	Codice vuoto	Codice [ERROR]	{codiceCorso: ""}	Messaggio di errore: codice non valido	Nessuna Iscrizione creata
TC3	Codice con caratteri non alfanumerici	Codice [ERROR]	{codiceCorso: "Cl@NF#"}	Messaggio di errore: codice non valido	Nessuna Iscrizione creata
TC4	Codice di lunghezza eccessiva	Codice [ERROR]	{codiceCorso: "CINF2024A0000000"}	Messaggio di errore: codice non valido	Nessuna Iscrizione creata
TC5	Codice non corrispondente ad alcun Corso	Corso corrispondente [ERROR]	{codiceCorso: "ZZZZ9999"}	Messaggio di errore: nessun Corso trovato	Nessuna Iscrizione creata

TC	Descrizione	Classi coperte	Input	Output atteso	Post-condizioni attese
TC6	Studente già iscritto	Stato iscrizione [ERROR]	{codiceCorso: "CINF2024A"} con Studente già iscritto	Messaggio informativo: già iscritto	Lo stato del sistema resta invariato

Tabella 24 — Casi di test — IscrivitiCorso (UC6)

3.4 PubblicaContenuto (UC7)

TITOLO	DESCRIZIONE	CATEGORIA	SEZIONE	PUBBLICAZIONE
Non vuoto, lunghezza ≤ 100 Vuoto [ERROR] Lunghezza > 100 [ERROR]	Non vuota, lunghezza ≤ 500 Vuota [ERROR] Lunghezza > 500 [ERROR]	Valore dell'enumerazione Categoria Non specificata [ERROR]	Nessuna Sezione Sezione del Corso Sezione di un altro Corso [ERROR]	Immediata Differita (bozza)

Tabella 25 — Classi di equivalenza — PubblicaContenuto

TC	Descrizione	Classi coperte	Input	Output atteso	Post-condizioni attese
TC1	Validi, pubblicazione immediata, senza Sezione	Titolo, descrizione, categoria validi; nessuna Sezione; immediata	{titolo: "Slide Lezione 1", descrizione: "Introduzione", categoria: SLIDE, sezione: null, pubblicaSubito: true}	Contenuto creato in stato PUBBLICATO; Studenti iscritti notificati	Il Contenuto è visibile agli Studenti; le Notifiche esistono
TC2	Validi, pubblicazione differita, con Sezione	Sezione del Corso; differita	{titolo: "Esercizi Modulo 2", categoria: ESERCIZI, sezione: "Modulo 2", pubblicaSubito: false}	Contenuto creato in stato BOZZA; nessuna Notifica	Il Contenuto non è visibile agli Studenti
TC3	Titolo vuoto	Titolo [ERROR]	{titolo: "", restanti validi}	Messaggio di errore: dati obbligatori mancanti	Nessun Contenuto creato
TC4	Titolo oltre 100 caratteri	Titolo [ERROR]	{titolo: stringa di 120 caratteri, restanti validi}	Messaggio di errore: titolo troppo lungo	Nessun Contenuto creato
TC5	Descrizione vuota	Descrizione [ERROR]	{descrizione: "", restanti validi}	Messaggio di errore: dati obbligatori mancanti	Nessun Contenuto creato
TC6	Descrizione oltre 500 caratteri	Descrizione [ERROR]	{descrizione: stringa di 600 caratteri, restanti validi}	Messaggio di errore: descrizione troppo lunga	Nessun Contenuto creato
TC7	Categoria non specificata	Categoria [ERROR]	{categoria: null, restanti validi}	Messaggio di errore: categoria obbligatoria	Nessun Contenuto creato

TC	Descrizione	Classi coperte	Input	Output atteso	Post-condizioni attese
TC8	Sezione di un altro Corso	Sezione [ERROR]	{sezione: Sezione appartenente a un Corso diverso}	Messaggio di errore: Sezione non valida per questo Corso	Nessun Contenuto creato
TC9	Docente non titolare del Corso	Titolarità [ERROR]	{corso: Corso di un altro Docente, restanti validi}	Messaggio di errore: operazione non consentita	Nessun Contenuto creato

Tabella 26 — Casi di test — *PubblicaContenuto* (UC7)

Nota su TC9. Nella prima versione dell'elaborato questo caso di test era classificato «non applicabile», perché senza autenticazione il sistema non sapeva quale Docente stesse operando. Con la sessione introdotta da UC2 il controllo di titolarità è realmente eseguibile, ed è oggi verificato anche da un test JUnit (§6.2).

3.5 AttivaContenuto (UC9) e RimuoviContenuto (UC11)

I due casi d'uso agiscono sulla stessa variabile — lo stato del Contenuto — e le loro classi di equivalenza coincidono con gli stati del diagramma del §2.7.7. Le transizioni non ammesse dal diagramma di stato sono altrettante classi [ERROR].

STATO CORRENTE DEL CONTENUTO	OPERAZIONE RICHIESTA	TITOLARITÀ DEL CORSO
BOZZA PUBBLICATO RIMOSSO	Attiva (pubblica) Rimuovi	Il Docente è titolare del Corso Il Docente non è titolare [ERROR]

Tabella 27 — Classi di equivalenza — *AttivaContenuto* / *RimuoviContenuto*

TC	Descrizione	Classi coperte	Input	Output atteso	Post-condizioni attese
TC1	Attivazione di un Contenuto in bozza	BOZZA + Attiva + titolare	{contenuto in BOZZA, operazione: attiva}	Stato portato a PUBBLICATO; Studenti notificati	Il Contenuto è visibile; esistono le Notifiche
TC2	Rimozione di un Contenuto pubblicato	PUBBLICATO + Rimuovi + titolare	{contenuto in PUBBLICATO, operazione: rimuovi}	Stato portato a RIMOSSO	Il Contenuto non è più visibile agli Studenti
TC3	Rimozione di un Contenuto in bozza	BOZZA + Rimuovi + titolare	{contenuto in BOZZA, operazione: rimuovi}	Stato portato a RIMOSSO	Il Contenuto non è più visibile
TC4	Attivazione di un Contenuto già pubblicato	PUBBLICATO + Attiva [ERROR]	{contenuto in PUBBLICATO, operazione: attiva}	Messaggio: operazione non ammessa nello stato corrente	Lo stato resta PUBBLICATO
TC5	Attivazione di un Contenuto rimosso	RIMOSSO + Attiva [ERROR]	{contenuto in RIMOSSO, operazione: attiva}	Messaggio: operazione non ammessa nello stato corrente	Lo stato resta RIMOSSO

TC	Descrizione	Classi coperte	Input	Output atteso	Post-condizioni attese
TC6	Rimozione di un Contenuto già rimosso	RIMOSSO + Rimuovi [ERROR]	{contenuto in RIMOSSO, operazione: rimuovi}	Messaggio: operazione non ammessa nello stato corrente	Lo stato resta RIMOSSO
TC7	Docente non titolare	Titolarità [ERROR]	{contenuto di un Corso di un altro Docente}	Messaggio di errore: operazione non consentita	Lo stato resta invariato

Tabella 28 — Casi di test — AttivaContenuto (UC9) e RimuoviContenuto (UC11)

3.6 VisualizzaContenutiCorso (UC13)

ISCRIZIONE DELLO STUDENTE	FILTRO IMPOSTATO	CONTENUTO SELEZIONATO	VISIBILITÀ DEI CONTENUTI
Lo Studente è iscritto al Corso Lo Studente non è iscritto al Corso [ERROR]	Nessun filtro Filtro con almeno un Contenuto corrispondente Filtro senza Contenuti corrispondenti	Contenuto esistente e appartenente al Corso Contenuto inesistente o di altro Corso [ERROR]	Il Corso contiene Contenuti pubblicati Il Corso contiene solo Contenuti in bozza o rimossi

Tabella 29 — Classi di equivalenza — VisualizzaContenutiCorso

TC	Descrizione	Classi coperte	Input	Output atteso	Post-condizioni attese
TC1	Nessun filtro impostato	Iscritto, nessun filtro, Contenuti pubblicati	{corso: "Ingegneria del Software", filtro: nessuno}	Elenco completo dei soli Contenuti pubblicati	Lo Studente visualizza l'elenco completo
TC2	Filtro con risultati	Filtro con corrispondenze	{corso: IS, filtro: {categoria: SLIDE}}	Elenco filtrato non vuoto	Lo Studente visualizza l'elenco filtrato
TC3	Filtro senza risultati	Filtro senza corrispondenze	{corso: IS, filtro: {categoria: AVVISI}}	Messaggio informativo ed elenco vuoto	Lo Studente visualizza un elenco vuoto
TC4	Studente non iscritto	Iscrizione [ERROR]	{corso a cui lo Studente non è iscritto}	Messaggio di errore: accesso non autorizzato	L'elenco non viene mostrato
TC5	Corso con soli Contenuti in bozza	Visibilità: nessun pubblicato	{corso con 2 Contenuti in BOZZA}	Elenco vuoto: le bozze non sono visibili allo Studente	Lo Studente non vede i Contenuti non pubblicati
TC6	Dettaglio di un Contenuto valido	Contenuto selezionato valido	{contenuto: "Dispensa Requisiti"}	Titolo, descrizione, categoria e data del Contenuto	Il dettaglio è mostrato correttamente

Tabella 30 — Casi di test — VisualizzaContenutiCorso (UC13)

3.7 MonitoraAndamentoCorso (UC15)

CONTENUTI DEL CORSO	ISCRIZIONI AL CORSO	TITOLARITÀ DEL CORSO
Il Corso ha almeno un Contenuto pubblicato Il Corso non ha Contenuti pubblicati	Il Corso ha almeno uno Studente iscritto Il Corso non ha Studenti iscritti	Il Docente richiedente è titolare del Corso Il Docente richiedente non è titolare [ERROR]

Tabella 31 — Classi di equivalenza — MonitoraAndamentoCorso

TC	Descrizione	Classi coperte	Input	Output atteso	Post-condizioni attese
TC1	Corso con Contenuti e iscritti	Contenuti presenti, iscrizioni presenti, titolarità valida	{corso: "Ingegneria del Software"} con 5 Contenuti e 2 iscritti	Riepilogo: 5 Contenuti, 2 Studenti, distribuzione per Categoria	Il Docente visualizza il riepilogo aggiornato
TC2	Corso senza Contenuti né iscritti	Contenuti assenti, iscrizioni assenti	{corso appena creato}	Riepilogo con valori pari a zero e distribuzione vuota	Il riepilogo viene comunque mostrato
TC3	Corso con Contenuti ma senza iscritti	Contenuti presenti, iscrizioni assenti	{corso con 3 Contenuti e 0 iscritti}	Riepilogo: 3 Contenuti, 0 Studenti	Il riepilogo viene mostrato correttamente
TC4	Docente non titolare	Titolarietà [ERROR]	{corso di un altro Docente}	Messaggio di errore: accesso non autorizzato	Il riepilogo non viene mostrato

Tabella 32 — Casi di test — MonitoraAndamentoCorso (UC15)

3.8 VisualizzaNotifiche (UC22)

NOTIFICHE DESTINATE ALLO STUDENTE	STATO DI LETTURA
Esiste almeno una Notifica Non esiste alcuna Notifica	Tutte non lette Alcune già lette

Tabella 33 — Classi di equivalenza — VisualizzaNotifiche

TC	Descrizione	Classi coperte	Input	Output atteso	Post-condizioni attese
TC1	Studente con Notifiche non lette	Notifiche presenti, non lette	{studente iscritto a un Corso in cui è stato pubblicato materiale}	Elenco delle Notifiche con testo, Corso e data	Le Notifiche mostrate risultano lette
TC2	Studente senza Notifiche	Notifiche assenti	{studente appena registrato}	Messaggio informativo ed elenco vuoto	Lo stato del sistema resta invariato
TC3	Riapertura dopo la lettura	Notifiche già lette	{riapertura della schermata dopo TC1}	Le stesse Notifiche, marcate come lette	Il contatore delle non lette è pari a zero

Tabella 34 — Casi di test — VisualizzaNotifiche (UC22)

4. Progettazione

La progettazione traduce il modello di analisi in un modello di soluzione, introducendo le decisioni che l'analisi aveva deliberatamente rimandato: la stratificazione dell'architettura, la collocazione delle responsabilità nelle classi software, la tecnologia di persistenza e i design pattern. Il capitolo segue quest'ordine: l'architettura (§4.1), il modello statico con la traduzione delle responsabilità (§4.2), i design pattern applicati con la loro giustificazione (§4.3), i principi di progettazione verificati (§4.4) e infine il modello dinamico (§4.5).

4.1 Architettura: il pattern BCED

Il vincolo V01 impone il pattern architetturale BCED, nella forma presentata alla lezione 14. L'architettura è organizzata in quattro livelli sovrapposti, ciascuno con una sola ragione di esistere e con dipendenze orientate esclusivamente dall'alto verso il basso.

Livello	Responsabilità	Che cosa NON può fare
Boundary	Interazione con l'attore: raccolta dell'input, presentazione dell'output, logica di presentazione	Non contiene logica di business; non conosce le classi entity né il database
Control	Coordinamento dei casi d'uso: sequenza dei passi, validazione dei dati in ingresso, gestione della sessione	Non contiene codice di interfaccia grafica; non accede direttamente alla persistenza
Entity	Dati e regole di business del dominio; ricerca e reperimento delle istanze persistenti	Non conosce il control né il boundary; non apre transazioni
Database	Apertura e chiusura delle connessioni, transazioni, commit e rollback, traduzione oggetto-tabella	Non conosce alcuna classe di dominio: opera su <code>Class<T></code> generico

Tabella 35 — Responsabilità dei quattro livelli dell'architettura BCED

La lezione 14 (pag. 39) prescrive che «il package Entity ottiene i dati dal package Data invocando le operazioni Carica e Salva». È la regola che determina la direzione delle dipendenze fra i due livelli inferiori: sono le entità a rivolgersi alla persistenza, non il contrario. Nella nostra realizzazione questo compito non è distribuito su ogni singola entità ma concentrato in tre classi Registro, che appartengono al package entity e sono le uniche a conoscere il package database.

Perché sono spariti i DAO. Nella prima versione dell'elaborato ogni entità aveva la propria classe DAO, e il vincolo V02 imponeva quel pattern. Con un ORM la traduzione oggetto-tabella è a carico del framework: scrivere una classe DAO per entità significherebbe reimplementare a mano ciò che Hibernate già fa, e introdurre un quinto livello non previsto dal BCED. La lezione 14 (pag. 40) elenca il persistence framework come una delle tre strategie ammesse, alternativa al DAO e non complementare ad esso.

La verifica del rispetto della stratificazione è meccanica e ripetibile: nessuna classe del package boundary importa classi dei package entity o database; nessuna classe del package entity importa classi dei package control o boundary; nessuna classe del package database importa classi di alcun altro package del progetto. Il controllo si esegue con una singola ispezione delle direttive di import ed è documentato al §6.4.

4.1.1 Il package dto

Fra boundary e control i dati non viaggiano come oggetti del dominio ma come Data Transfer Object: oggetti privi di comportamento, contenenti soltanto tipi primitivi e stringhe. È questa scelta a rendere possibile la regola per cui il boundary non conosce il package entity.

Senza DTO, una schermata che mostra l'elenco dei Contenuti di un Corso dovrebbe ricevere oggetti Contenuto, e il package boundary dipenderebbe dal package entity, violando la stratificazione. Il DTO non è quindi un livello aggiuntivo dell'architettura: è il tipo di dato con cui due livelli adiacenti comunicano, e per questo il package dto è disegnato di lato rispetto alla pila dei quattro livelli, raggiunto da dipendenze «use» sia dal boundary sia dal control.

Effetto sull'enumerazione Categoria. Anche l'enumerazione *Categoria* appartiene al package entity, e quindi non può attraversare il confine: nei DTO la categoria è rappresentata come stringa, e la conversione avviene nel control. Il metodo `getCategorieDisponibili()` della *Façade* restituisce infatti una `List<String>`, non una `List<Categoria>`. È un dettaglio piccolo ma è esattamente il punto in cui la regola verrebbe violata senza accorgersene.

4.2 Diagramma delle classi di progettazione

4.2.1 Traduzione delle responsabilità di analisi

La tabella seguente è il ponte fra il modello di analisi e il modello di progettazione: per ogni responsabilità individuata al §2.8 indica la classe di progettazione che la eredita, e la ragione della collocazione. Nessuna responsabilità dell'analisi resta senza destinazione, e nessuna classe di progettazione compare senza una responsabilità che la giustifichi.

Responsabilità in analisi	Classe in analisi	Classe/i in progettazione	Motivazione
Registrazione, Accesso	Piattaforma	GestoreUtenti (+ UtenteFactory)	Coordinamento di casi d'uso: appartiene al livello control
Ricerca degli Utenti, unicità dell'email	Piattaforma	RegistroUtenti	Reperimento di istanze persistenti: appartiene al livello entity (L14 p.39)
Verifica delle proprie credenziali e del blocco	Utente	Utente (invariata)	Regola di business su dati propri dell'entità: resta dov'era
Iscrizione a un Corso	Studente	GestoreCorsi	Il caso d'uso attraversa più entità: il coordinamento è del control
Ricerca dei Corsi, verifica dell'iscrizione	Piattaforma / Studente	RegistroCorsi	Interrogazioni sulla persistenza: livello entity
Pubblicazione e consultazione del materiale	Corso	GestoreContenuti	Coordinamento dei casi d'uso UC7, UC9, UC11, UC13
Ricerche, conteggi e filtri sui Contenuti	Corso	RegistroContenuti	Interrogazioni sulla persistenza: livello entity
Stato di pubblicazione (attributo booleano)	Contenuto	StatoContenuto + StatoBozza, StatoPubblicato, StatoRimosso	Il ciclo di vita del §2.7.7 richiede il pattern State (§4.3.4)
Invio delle Notifiche	Piattaforma	GestoreNotifiche (realizza Observer)	Reazione a un evento di dominio: pattern Observer (§4.3.5)
Recapito del messaggio	— (non modellata)	ServizioNotifiche + AdapterServizioNotifiche	Servizio esterno con firma incompatibile: pattern Adapter (§4.3.6)
Accesso ai dati persistenti	— (non modellata)	GestorePersistenza + JpaUtil	Il livello database non esiste in analisi: nasce con V02
Identità dell'Utente collegato	— (non modellata)	SessioneUtente	Nasce dal vincolo V05: l'analisi la dava per implicita nelle pre-condizioni

Tabella 36 — Traduzione delle responsabilità dall'analisi alla progettazione

Si notino le due direzioni della trasformazione. Alcune responsabilità si spostano verso l'alto: l'iscrizione a un Corso, che in analisi apparteneva allo Studente per Information Expert, in progettazione diventa un caso d'uso coordinato da GestoreCorsi, perché coinvolge Studente, Corso e Iscrizione e nessuno dei tre è l'esperto dell'intero flusso. Altre si spostano verso il basso: la ricerca dei

Corsi, che in analisi apparteneva a Piattaforma, in progettazione scende sui Registri, perché la lezione 14 assegna al livello entity il reperimento delle istanze persistenti.

Tre responsabilità sono invece nuove, e non hanno un corrispondente in analisi: l'accesso ai dati persistenti, che nasce dal vincolo V02; l'identità dell'Utente collegato, che nasce dal vincolo V05; il recapito del messaggio, che nasce dal vincolo V04. Tutte e tre derivano da vincoli tecnologici, ed è corretto che l'analisi — che descrive il problema e non la soluzione — non le contenesse.

4.2.2 Vista d'insieme

Il diagramma seguente mostra i quattro package con le loro dipendenze reciproche e il package dto di lato. È la vista che rende immediatamente verificabile la regola sulla direzione delle dipendenze: tutte le frecce «use» puntano verso il basso.

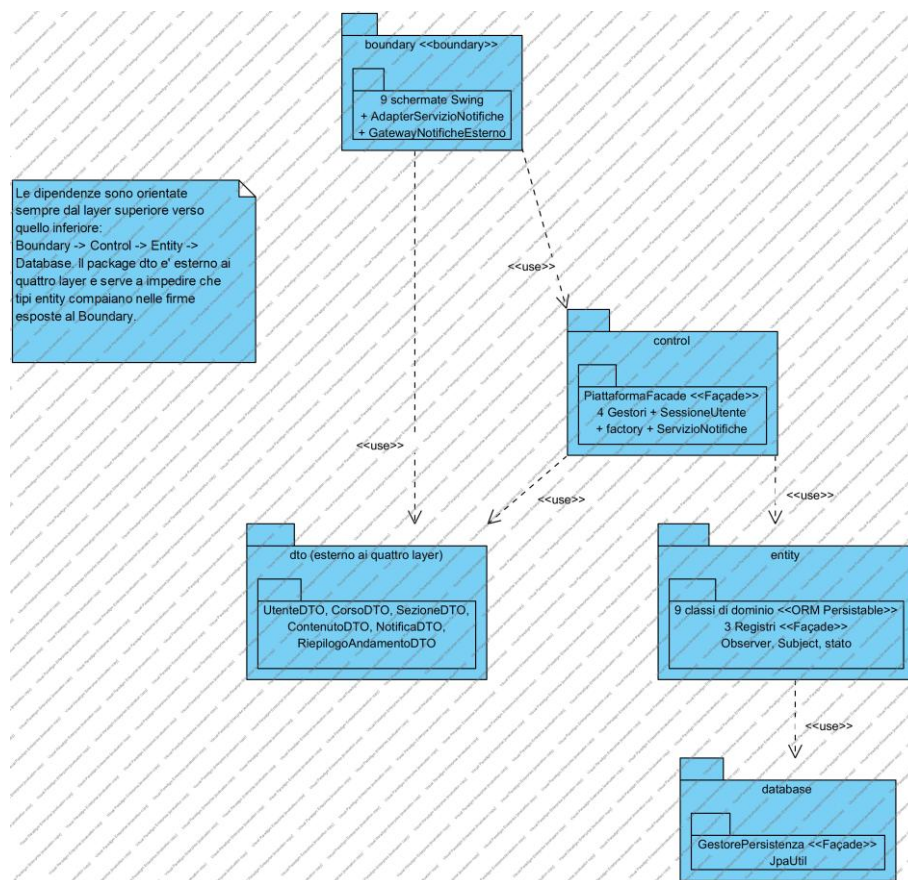


Figura 12 — Diagramma delle classi di progettazione — vista d'insieme dei package [\[apri il modello\]](#)

4.2.3 Package boundary

Il package boundary contiene le undici classi Swing che realizzano l'interfaccia utente, più le due classi che realizzano l'Adapter verso il servizio di notifiche esterno. Ogni schermata corrisponde a un caso d'uso o a un raggruppamento coerente di casi d'uso.

Classe	Caso d'uso realizzato	Ruolo
FormLogin	UC2 Accesso	Finestra iniziale dell'applicazione; raccoglie email e password
FormRegistrazione	UC1 Registrazione	Raccoglie ruolo, dati anagrafici, email e password
DashboardStudente	UC12, punto di accesso a UC6, UC13, UC22	Area personale dello Studente, con i Corsi seguiti e il contatore delle Notifiche
DashboardDocente	punto di accesso a UC7, UC9, UC11, UC15	Area personale del Docente, con i Corsi di cui è titolare
FormIscrivitiCorso	UC6 IscrivitiCorso	Finestra modale per l'inserimento del codice del Corso
FormPubblicaContenuto	UC7 PubblicaContenuto	Raccoglie i dati del Contenuto e la scelta di pubblicazione immediata
FormGestisciContenuti	UC9 AttivaContenuto, UC11 RimuoviContenuto	Elenco dei Contenuti con il loro stato e le azioni ammesse
FormVisualizzaContenutiCorso	UC13, UC14, UC18	Elenco dei Contenuti pubblicati, filtri e pannello di dettaglio
FormMonitoraAndamentoCorso	UC15, UC19, UC20	Riepilogo statistico del Corso
FormNotifiche	UC22 VisualizzaNotifiche	Elenco delle Notifiche ricevute dallo Studente
StileGUI	—	Classe di utilità di presentazione: colori, font, costruzione dei componenti e finestre di dialogo
AdapterServizioNotifiche	UC21 InviaNotifica	Adatta il gateway esterno all'interfaccia ServizioNotifiche dichiarata nel control
GatewayNotificheEsterno	—	«external» — simula il servizio di messaggistica previsto dal vincolo V04

Tabella 37 — Classi del package boundary

Tutte le schermate dipendono da una sola classe del livello control, *PiattaformaFacade*, e da nessun'altra: è la conseguenza diretta dell'applicazione del pattern *Façade* descritta al §4.3.2. La classe *StileGUI* non realizza alcun caso d'uso ma raccoglie la logica di presentazione — palette, tipografia, costruzione dei componenti ricorrenti, finestre di dialogo di conferma e di errore — che sarebbe altrimenti duplicata in ogni schermata.

Perché l'Adapter sta nel boundary. Il servizio di notifiche è un sistema esterno, e l'interazione con sistemi esterni appartiene al livello di confine: è ciò che la lettera *B* del pattern *BCED* indica, che non significa soltanto «interfaccia utente» ma «confine del sistema». L'interfaccia *ServizioNotifiche* è invece dichiarata nel control, che è il livello che ne ha bisogno: il control dipende da un'astrazione che sta al proprio livello, e la realizzazione concreta gli viene fornita dall'esterno.

4.2.4 Package control

Il package control contiene la *Façade*, i quattro Gestori, la classe di sessione, l'interfaccia del servizio di notifiche e il sotto-package factory.

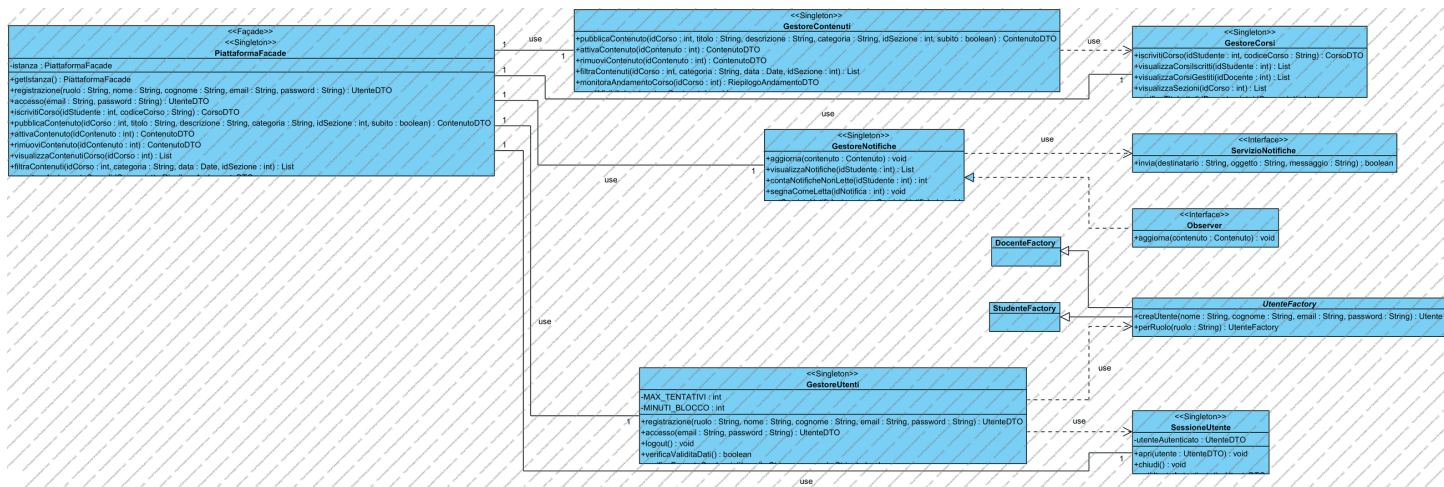


Figura 14 — Diagramma delle classi — package control [\[apri il modello\]](#)

Classe	Ruolo	Casi d'uso coordinati
PiattaformaFacade	«Façade», Singleton — unico punto di accesso del boundary al sistema; espone un metodo per caso d'uso, con parametri primitivi e DTO	tutti
GestoreUtenti	Registrazione, autenticazione, politica di blocco dell'account, validazione delle credenziali	UC1, UC2
GestoreCorsi	Iscrizione ai Corsi, consultazione dei Corsi seguiti e gestiti, gestione delle Sezioni	UC6, UC12, UC8
GestoreContenuti	Pubblicazione, attivazione, rimozione, consultazione e filtraggio dei Contenuti, riepilogo di andamento	UC7, UC9, UC11, UC13, UC14, UC15, UC16, UC18, UC19, UC20
GestoreNotifiche	Realizza l'interfaccia Observer: reagisce alla pubblicazione di un Contenuto creando e recapitando le Notifiche	UC21, UC22
SessioneUtente	Singleton — custodisce l'identità dell'Utente autenticato per la durata della sessione (V05)	—
ServizioNotifiche	«interface» — astrazione del canale di recapito, realizzata nel boundary dall'Adapter	—
UtenteFactory	«abstract» — Factory Method per la creazione dell'Utente del ruolo richiesto	UC1
StudenteFactory, DocenteFactory	Realizzazioni concrete della factory	UC1

Tabella 38 — Classi del package control

La suddivisione in quattro Gestori segue il criterio della coesione di livello raccomandato dalla lezione 13: ogni Gestore raggruppa i casi d'uso che insistono sulla stessa area di responsabilità, e ha quindi una sola ragione per cambiare. Un unico Gestore per tutti i casi d'uso sarebbe una classe con quattro ragioni di cambiamento; un Gestore per caso d'uso produrrebbe ventidue classi quasi vuote, con forte duplicazione.

Fra i Gestori esiste una dipendenza: GestoreContenuti utilizza GestoreCorsi per risolvere il Corso su cui operare. È accoppiamento all'interno dello stesso livello, ammesso dall'architettura, e non attraversa alcun confine. La dipendenza verso GestoreNotifiche, che nella prima versione esisteva ed era una vera dipendenza funzionale, è stata invece eliminata con l'introduzione del pattern Observer (§4.3.5).

4.2.5 Package entity

Il package entity contiene le classi del dominio, mappate sulle tabelle del database, le classi che realizzano il pattern State e i tre Registri.

Classe	Stereotipo	Ruolo
Utente	«ORM Persistable», abstract	Radice della gerarchia; dati anagrafici, credenziali, contatore dei tentativi falliti e istante di sblocco
Studente, Docente	«ORM Persistable»	Specializzazioni di Utente; realizzano il metodo astratto getRuolo()
Corso	«ORM Persistable»	Titolo, descrizione, codice univoco, anno accademico, Docente titolare
Sezione	«ORM Persistable»	Titolo e Corso di appartenenza
Contenuto	«ORM Persistable»	Titolo, descrizione, categoria, data e stato; estende Subject
Iscrizione	«ORM Persistable»	Classe associativa fra Studente e Corso, con data di iscrizione
Notifica	«ORM Persistable»	Testo, data, destinatario e stato di lettura
Categoria	«enumeration»	Slide, dispense, esercizi, soluzioni, avvisi, materiale integrativo
StatoContenutoEnum	«enumeration»	Bozza, pubblicato, rimosso: è il valore effettivamente persistito
StatoContenuto	«interface»	Astrazione dello stato; realizzata da StatoBozza, StatoPubblicato, StatoRimosso
Subject, Observer	«interface»	Infrastruttura del pattern Observer
RegistroUtenti	«Façade», Singleton	Registrazione e ricerca degli Utenti; garantisce l'unicità dell'email (RD07)
RegistroCorsi	«Façade», Singleton	Ricerca dei Corsi e delle Sezioni, gestione e verifica delle Iscrizioni
RegistroContenuti	«Façade», Singleton	Ricerche, filtri, conteggi e distribuzioni sui Contenuti; gestione delle Notifiche

Tabella 39 — Classi del package entity

Nota UML apposta ai Registri. «I tre Registri realizzano le responsabilità che nel modello di analisi appartenevano alla classe Piattaforma; si veda §4.2.1 della documentazione.» I Registri sono le uniche classi del package entity a conoscere il package database, coerentemente con la prescrizione della lezione 14 (pag. 39).

La gerarchia Utente → {Studente, Docente} è mappata sul database con la strategia a tabella singola e colonna discriminante: una sola tabella UTENTE con una colonna TIPO che vale 'STUDENTE' o 'DOCENTE'. È la strategia più efficiente quando le sottoclassi aggiungono pochi attributi propri, perché evita la giunzione necessaria alle strategie a tabella per classe, e permette a Hibernate di istanziare la sottoclasse corretta leggendo la sola colonna discriminante — che è esattamente ciò che serve all'autenticazione, dove il ruolo si conosce solo dopo aver letto il record.

4.2.6 Package database

Il package database contiene due sole classi, ed è il livello più semplice dell'architettura proprio perché non conosce il dominio.

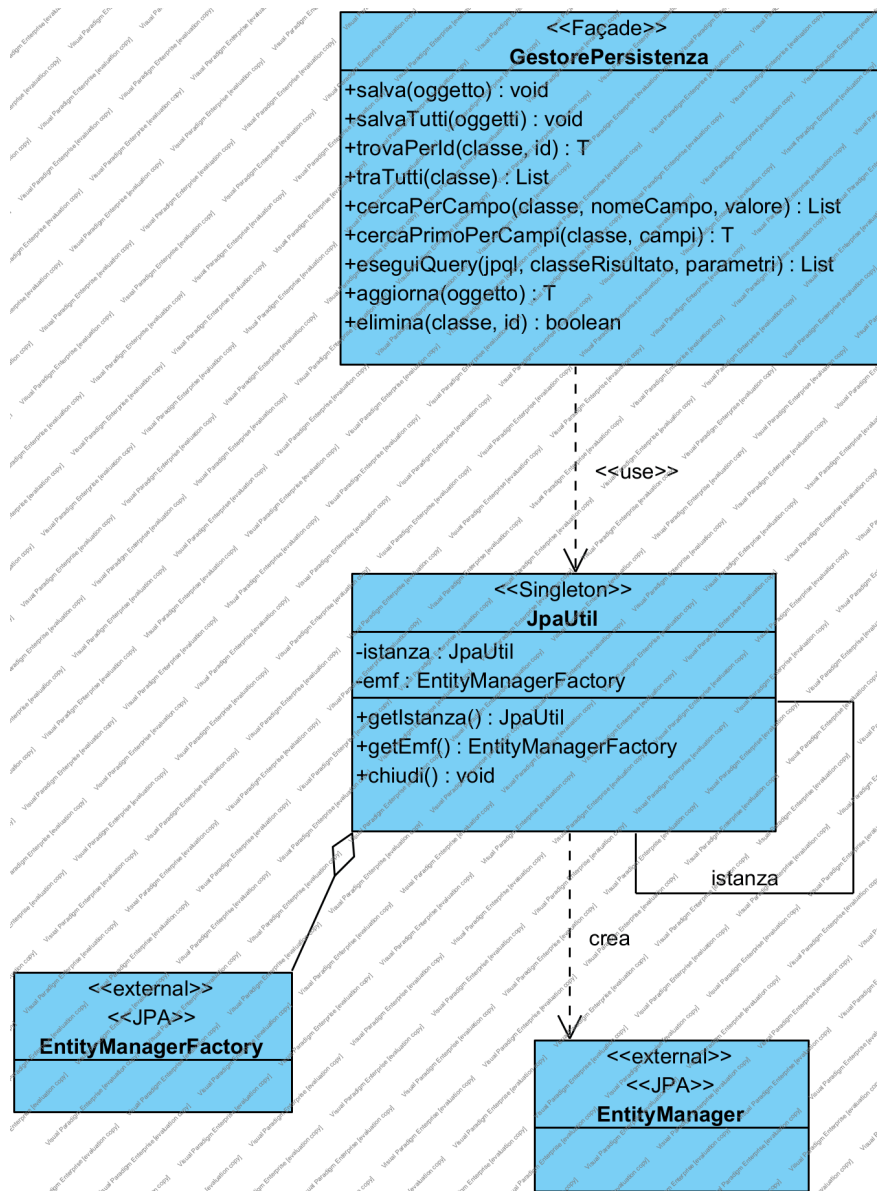


Figura 16 — Diagramma delle classi — package database [\[apri il modello\]](#)

Classe	Ruolo
JpaUtil	Singleton — crea e custodisce l'unica EntityManagerFactory dell'applicazione, la cui costruzione è costosa; espone un metodo per ottenere EntityManager e un hook di chiusura ordinata alla terminazione della JVM
GestorePersistenza	«Facade», Singleton — unico punto di accesso alla persistenza; realizza le operazioni CRUD in forma generica su Class<T>, apre e chiude una transazione per operazione ed esegue il rollback in caso di eccezione

Tabella 40 — Classi del package database

Nota UML apposta a GestorePersistenza. «Sostituisce le classi DAO per entità: con un ORM la traduzione oggetto-tabella è a carico di Hibernate.»

GestorePersistenza non contiene alcun riferimento a Utente, Corso o Contenuto: le sue firme accettano Class<T> e restituiscono T o List<T>. È questa genericità a soddisfare due principi della

lezione 13 contemporaneamente: mantenere alta l'astrazione, perché il livello non conosce il dominio su cui opera, e progettare per la riusabilità, perché la stessa classe è riutilizzabile in un qualunque altro progetto che usi JPA.

La gestione delle transazioni segue il modello «un EntityManager per operazione»: ogni metodo apre il proprio EntityManager, avvia la transazione, esegue, effettua il commit e chiude in un blocco finally, con rollback esplicito in caso di eccezione. È la scelta più semplice e più robusta per un'applicazione desktop monoutente, e ha una conseguenza progettuale importante che viene discussa al §5.3: gli oggetti restituiti sono detached, e le loro collezioni a caricamento pigro non sono più navigabili fuori dal metodo.

4.2.7 Schema relazionale

Lo schema logico generato da Hibernate a partire dalle annotazioni delle classi entity è riportato di seguito. Le chiavi primarie sono surrogate e generate dal database; i vincoli di unicità traducono i requisiti sui dati.

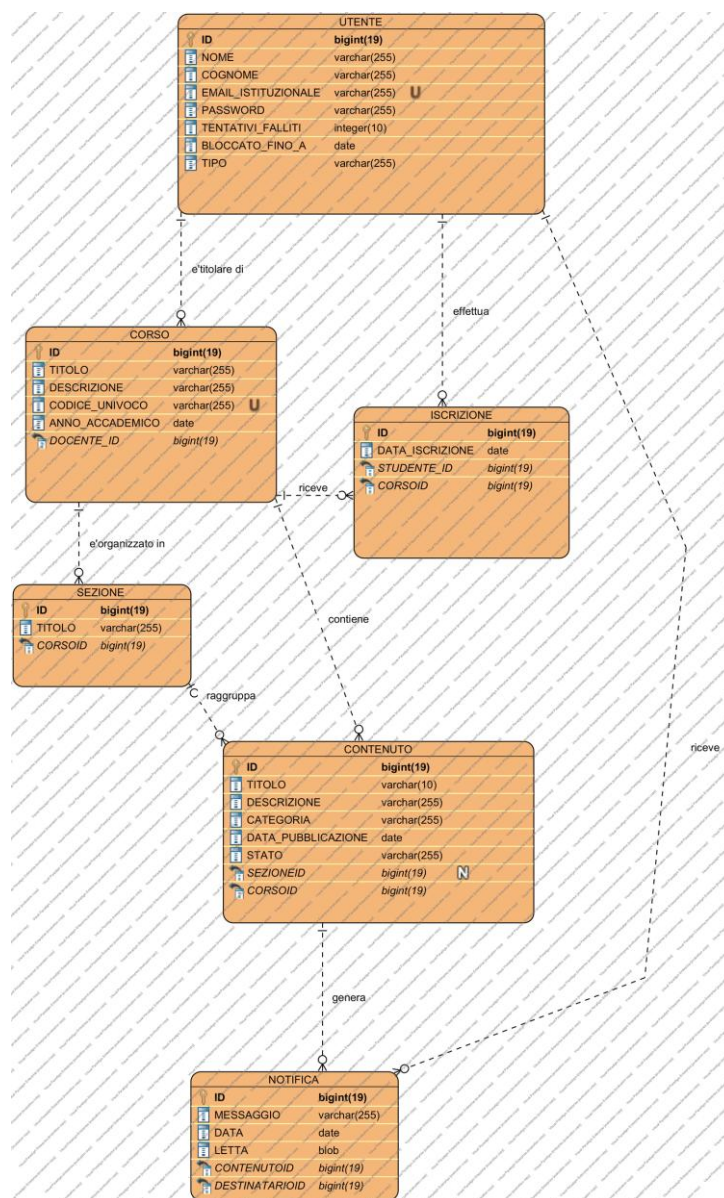


Figura 17 — Schema relazionale del database [\[apri il modello\]](#)

Tabella	Origine	Vincoli notevoli
UTENTE	Utente, Studente, Docente (tabella singola)	EMAIL_ISTITUZIONALE univoca (RD07); colonna discriminante TIPO
CORSO	Corso	CODICE_UNIVOCO univoco (RD02); chiave esterna verso UTENTE (titolare)
SEZIONE	Sezione	Chiave esterna verso CORSO (RD06)
CONTENUTO	Contenuto	Colonna STATO con i valori dell'enumerazione; chiavi esterne verso CORSO e SEZIONE
ISCRIZIONE	Iscrizione	Coppia (STUDENTE, CORSO) univoca: impedisce la doppia iscrizione (RD03)
NOTIFICA	Notifica	Chiave esterna verso UTENTE (destinatario); flag di lettura (RD09)

Tabella 41 — Tabelle dello schema relazionale e loro origine

Che cosa è cambiato nello schema. Rispetto alla prima versione sono scomparse la colonna *USERNAME*, resa superflua dall'uso dell'email come identificativo di accesso (RD07), e la colonna booleana *PUBBLICATO*, sostituita dalla colonna *STATO* che assume i tre valori del diagramma di stato del §2.7.7. Sono state aggiunte le colonne *TENTATIVI_FALLITI* e *BLOCCATO_FINO_A* richieste da RD08.

4.3 Design pattern applicati

Sono stati applicati sei design pattern fra quelli presentati alla lezione 15. Per ciascuno si indicano il problema che risolve nel nostro sistema, il requisito o vincolo che lo motiva, e le classi che vi partecipano. Nessuno è stato introdotto per esibizione: dove un pattern non risolveva un problema reale del progetto non è stato usato.

Pattern	Categoria	Problema risolto nel progetto	Motivazione
Singleton	Creazionale	Unicità della factory di persistenza, della Façade, dei Gestori, dei Registri e della sessione	Costo di costruzione e necessità di un punto di accesso unico
Façade	Strutturale	Riduzione dell'accoppiamento fra boundary e control, e fra entity e persistenza	Principio di riduzione dell'accoppiamento (L13)
Factory Method	Creazionale	Creazione dell'Utente del ruolo corretto senza che il chiamante conosca le sottoclassi	RF01: il ruolo è un dato di input
State	Comportamentale	Ciclo di vita del Contenuto con transizioni ammesse e vietate	RF10, RF12 e diagramma di stato §2.7.7
Observer	Comportamentale	Notifica automatica agli Studenti alla pubblicazione di materiale	RF22, RF23
Adapter	Strutturale	Uso di un servizio di messaggistica esterno con firma incompatibile	V04

Tabella 42 — Design pattern applicati e loro motivazione

4.3.1 Singleton

Il pattern Singleton garantisce che di una classe esista una sola istanza, accessibile da un punto noto. È applicato a JpaUtil, GestorePersistenza, PiattaformaFacade, ai quattro Gestori, ai tre Registri, alle tre classi di stato e a SessioneUtente.

Le ragioni sono di due tipi. Per JpaUtil la ragione è il costo: la costruzione dell'EntityManagerFactory comporta la lettura del file di configurazione, la validazione delle mappature e la creazione del pool di connessioni, ed è un'operazione che va eseguita una sola volta per esecuzione dell'applicazione. Per le altre classi la ragione è la coerenza dello stato: SessioneUtente deve rappresentare una e una sola sessione di lavoro, e i Registri devono essere lo stesso punto di accesso per chiunque interroghi la persistenza.

L'implementazione adottata è quella con inizializzazione pigra e metodo di accesso sincronizzato, che è la forma presentata alla lezione 15 (pag. 25). Le tre classi di stato sono Singleton per una ragione ulteriore: sono prive di dati propri, e istanziarne una per ogni Contenuto sarebbe uno spreco senza contropartita.

4.3.2 Façade

Il pattern Façade fornisce un'interfaccia unificata a un insieme di interfacce di un sottosistema, rendendolo più semplice da usare. È applicato tre volte, a tre livelli diversi dell'architettura.

Façade	Sottosistema nascosto	Clienti
PiattaformaFacade	I quattro Gestori del control e la loro cooperazione	Le dieci schermate del boundary
I tre Registri	Le interrogazioni JPQL e la loro traduzione in oggetti del dominio	I Gestori del control
GestorePersistenza	L'EntityManager, le transazioni, il ciclo di vita degli oggetti persistenti	I Registri del package entity

Tabella 43 — Applicazioni del pattern Façade

L'effetto sull'accoppiamento è misurabile. Senza `PiattaformaFacade` ogni schermata dovrebbe conoscere il Gestore competente per la propria funzione, e alcune ne conoscerebbero più d'uno: `FormPubblicaContenuto` avrebbe bisogno di `GestoreContenuti` per creare il `Contenuto` e di `GestoreCorsi` per elencare le Sezioni. Con la Façade, ogni schermata conosce una sola classe, e l'aggiunta di un nuovo Gestore non comporta alcuna modifica al boundary.

La Façade non contiene logica: ogni suo metodo verifica la sessione, se necessario, e delega al Gestore competente. È una scelta deliberata, perché una Façade che contenesse logica diventerebbe la classe più grande del sistema e ne concentrerebbe tutte le ragioni di cambiamento.

4.3.3 Factory Method

La registrazione (RF01) riceve il ruolo come dato di input e deve creare un'istanza di `Studente` o di `Docente`. Senza il pattern, `GestoreUtenti` conterrebbe un costrutto condizionale sul ruolo e conoscerebbe entrambe le sottoclassi concrete; ogni nuovo ruolo comporterebbe la modifica di quel codice.

Con il Factory Method, la classe astratta `UtenteFactory` dichiara il metodo di creazione, e le due sottoclassi `StudenteFactory` e `DocenteFactory` lo realizzano restituendo l'istanza del tipo corretto. `GestoreUtenti` seleziona la factory in base al ruolo e le chiede l'oggetto, senza mai nominare `Studente` né `Docente` nel proprio codice di creazione.

Perché la factory sta nel control e non nell'entity. La factory non è un concetto del dominio: nessuno degli scenari di §2.5.3 la menziona. È un meccanismo di costruzione che serve al coordinatore del caso d'uso, e appartiene quindi al livello che quel caso d'uso coordina. Le dipendenze «create» dalla factory verso `Studente` e `Docente` rispettano la direzione ammessa, dal control verso l'entity.

4.3.4 State

Il diagramma di stato del §2.7.7 stabilisce che un `Contenuto` attraversa tre stati e che alcune transizioni sono vietate. Rappresentare questo con un attributo booleano, come nella prima versione dell'elaborato, comporta due problemi: due valori non bastano a rappresentare tre stati, e le transizioni vietate diventano controlli condizionali sparsi nel codice, che è facile dimenticare in un punto.

Il pattern State risolve entrambi i problemi facendo del comportamento dipendente dallo stato una responsabilità dell'oggetto stato. L'interfaccia `StatoContenuto` dichiara le operazioni `pubblica()`, `rimuovi()` e `isVisibile()`; le tre classi concrete la realizzano ciascuna secondo le regole del proprio stato.

Stato	pubblica()	rimuovi()	isVisibile()
StatoBozza	Transizione verso StatoPubblicato; notifica gli osservatori	Transizione verso StatoRimosso	false
StatoPubblicato	Operazione non ammessa: solleva eccezione	Transizione verso StatoRimosso	true
StatoRimosso	Operazione non ammessa: solleva eccezione	Operazione non ammessa: solleva eccezione	false

Tabella 44 — Tabella delle transizioni realizzata dal pattern State

La corrispondenza con il diagramma di stato è biunivoca: ogni cella della tabella è un arco del diagramma, oppure la sua assenza. Aggiungere un quarto stato — per esempio «archiviato» — significherebbe aggiungere una classe, e non modificare codice esistente in più punti.

Il problema della persistenza dello stato. Un oggetto stato è un Singleton privo di dati e non è persistibile. Sul database viene quindi memorizzato il valore dell'enumerazione `StatoContenutoEnum`, che è un dato; l'oggetto stato corrispondente viene risolto in memoria a partire da quel valore, ed è marcato come campo transiente perché non deve essere mappato su alcuna colonna. È il punto in cui il pattern e l'ORM si incontrano, ed è una domanda naturale all'esame.

4.3.5 Observer

I requisiti RF22 e RF23 impongono che gli Studenti iscritti siano notificati quando nuovo materiale diventa visibile. Il problema progettuale è che l'evento nasce nel livello entity — è il Contenuto a cambiare stato — mentre la reazione appartiene al livello control, e il livello entity non può conoscere il control.

Il pattern Observer risolve l'inversione della dipendenza. L'interfaccia Observer è dichiarata nel package entity; la classe Contenuto estende Subject e, quando entra nello stato pubblicato, avvisa i propri osservatori attraverso quell'interfaccia. `GestoreNotifiche`, che sta nel control, realizza Observer e si registra come osservatore. Il livello entity conosce quindi soltanto un'interfaccia definita al proprio livello, e ignora chi la realizzi.

Il beneficio è concreto e non teorico: nella prima versione `GestoreContenuti` chiamava direttamente `GestoreNotifiche` dopo aver creato il Contenuto, e quella chiamata andava ripetuta identica in ogni punto in cui un Contenuto potesse diventare visibile. Con l'Observer la notifica è agganciata alla transizione di stato, e vale automaticamente sia per la pubblicazione immediata (UC7) sia per l'attivazione differita (UC9), che è precisamente lo scenario che la prima versione lasciava scoperto.

4.3.6 Adapter

Il vincolo V04 prevede che le Notifiche viaggino su un servizio di messaggistica. Un servizio esterno ha una propria interfaccia, che non coincide con quella di cui il nostro sistema ha bisogno: nel nostro caso il gateway espone un metodo che riceve un identificativo di destinatario e un corpo del messaggio già formattato, mentre il control ha bisogno di inviare una Notifica a uno Studente.

Il pattern Adapter interpone una classe che traduce fra le due interfacce. Il control dichiara l'interfaccia `ServizioNotifiche` nei termini che gli servono; `AdapterServizioNotifiche`, nel boundary, la realizza

traducendo ogni chiamata in una o più chiamate al gateway. Il collegamento fra i due avviene una sola volta, nel metodo main, che è il punto in cui l'applicazione viene composta.

Beneficio verificabile. *Sostituire il servizio di messaggistica con un fornitore diverso comporta la scrittura di un nuovo adattatore e una riga di modifica nel main. Nessuna classe del control e nessuna classe dell'entity cambia. È lo stesso beneficio che si otterrebbe da un'iniezione di dipendenza, ottenuto qui con i soli strumenti presentati a lezione.*

4.4 Verifica dei principi di progettazione

La lezione 13 elenca sei principi di progettazione. La tabella seguente riporta, per ciascuno, come il progetto lo soddisfa e dove la conformità è verificabile.

Principio	Come è soddisfatto	Dove si verifica
Massimizzare la coesione	Coesione di livello: ogni package ha una sola ragione di esistere; ogni Gestore raccoglie i casi d'uso di una sola area funzionale; ogni Registro le interrogazioni su un solo aggregato	§4.2.3 – §4.2.6
Ridurre l'accoppiamento	Il boundary dipende da una sola classe del control; i Registri da una sola classe del database; il package database non dipende da nulla	§4.3.2 e verifica meccanica §6.4
Mantenere alta l'astrazione	CRUD generico su <code>Class<T></code> ; quattro interfacce di dominio (Observer, Subject, StatoContenuto, ServizioNotifiche); Utente e UtenteFactory astratte	§4.2.6 e §4.3
Progettare per la riusabilità	GestorePersistenza è riusabile in qualunque progetto JPA senza modifiche; StileGUI è riusabile in qualunque applicazione Swing	§4.2.6
Progettare in modo difensivo	Validazione dei dati in ingresso nel control prima di ogni operazione; rollback esplicito su ogni transazione; blocco dell'account dopo tentativi ripetuti; transizioni di stato vietate che sollevano eccezione	§4.3.4, §5.4, RQ-SEC-01
Progettare per la testabilità	I Gestori sono invocabili senza interfaccia grafica: la suite JUnit attacca direttamente la Façade, senza istanziare alcuna finestra Swing	§6.2

Tabella 45 — Verifica dei principi di progettazione della lezione 13

Il principio della progettazione difensiva merita una precisazione sulla collocazione della validazione. I controlli di formato — lunghezza dei campi, struttura dell'indirizzo email, composizione della password, formato del codice del Corso — sono eseguiti nel livello control e non nel boundary. La ragione è che non si tratta di formattazione ma di regole legate a requisiti espliciti: la struttura dell'email traduce RD07, la composizione della password traduce RQ-SEC-01. Collocarle nel boundary significherebbe renderle aggirabili da qualunque altro cliente del control — per esempio dalla suite di test, che infatti le verifica proprio perché stanno nel control.

4.5 Diagrammi di sequenza di progettazione

Per ciascun caso d'uso sviluppato si riporta il diagramma di sequenza di progettazione, che mostra gli oggetti software effettivamente coinvolti e i messaggi scambiati con le firme reali dei metodi. Il confronto con il corrispondente diagramma di analisi del §2.7 rende visibile la trasformazione descritta al §4.2.1.

Tutti i diagrammi seguono la stessa struttura, che è la traduzione diretta della stratificazione BCED: la schermata del boundary invoca `PiattaformaFacade`, che delega al `Gestore` competente, il quale interroga i `Registri`, che a loro volta invocano `GestorePersistenza`. Il valore di ritorno risale la stessa catena, convertito in DTO nel passaggio dal control al boundary.

4.5.1 Registrazione (UC1)

Si osservi il punto in cui interviene il Factory Method: GestoreUtenti seleziona la factory in base alla stringa del ruolo ricevuta dal boundary, e le chiede l'Utente. La verifica di unicità dell'email precede la creazione ed è delegata a RegistroUtenti, che è la classe del livello entity responsabile del reperimento degli Utenti.

4.5.2 Accesso (UC2)

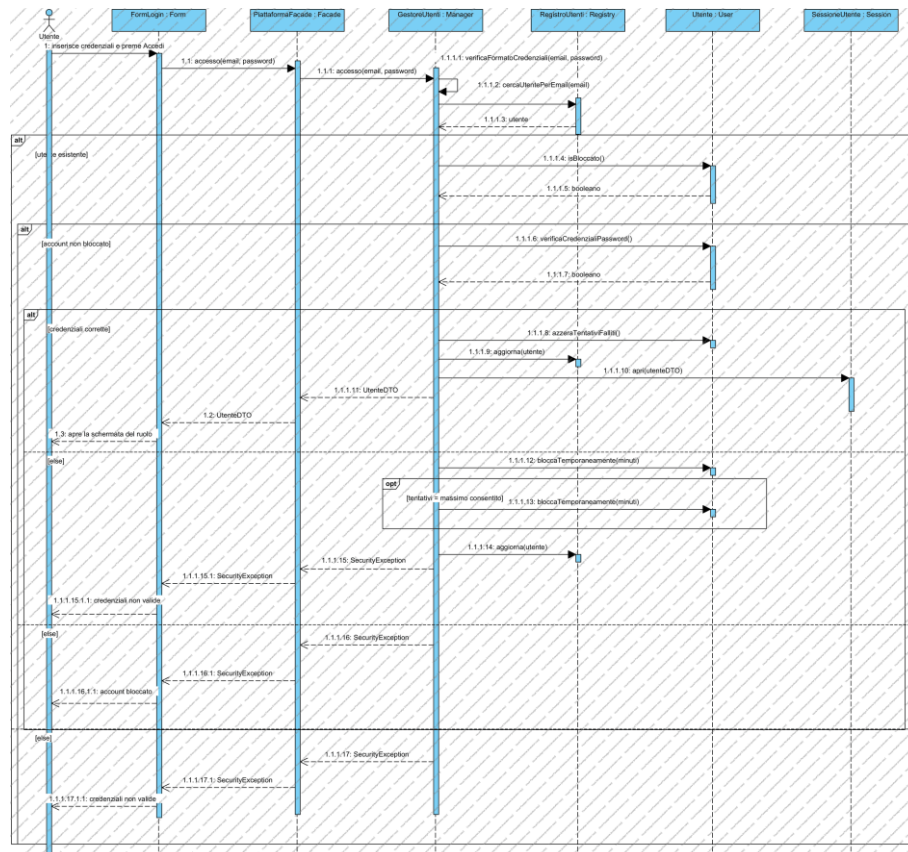


Figura 19 — Diagramma di sequenza di progettazione — Accesso (UC2) [\[apri il modello\]](#)

Il diagramma mostra le tre verifiche in sequenza — formato, blocco dell'account, corrispondenza della password — e i due esiti che aggiornano lo stato dell'Utente: l'azzeramento del contatore in caso di successo, il suo incremento e l'eventuale blocco in caso di fallimento. Si noti che entrambe le operazioni sono invocate sull'oggetto Utente, non eseguite dal Gestore: è la stessa collocazione stabilita in analisi al §2.7.2, conservata invariata in progettazione.

L'ultimo messaggio apre la sessione su SessioneUtente. È l'oggetto che sostituisce il menu di scelta dell'utente che nella prima versione dell'elaborato simulava l'identità del chiamante, e che rende reali tutti i controlli di autorizzazione dei casi d'uso successivi.

4.5.3 IscrivitiCorso (UC6)

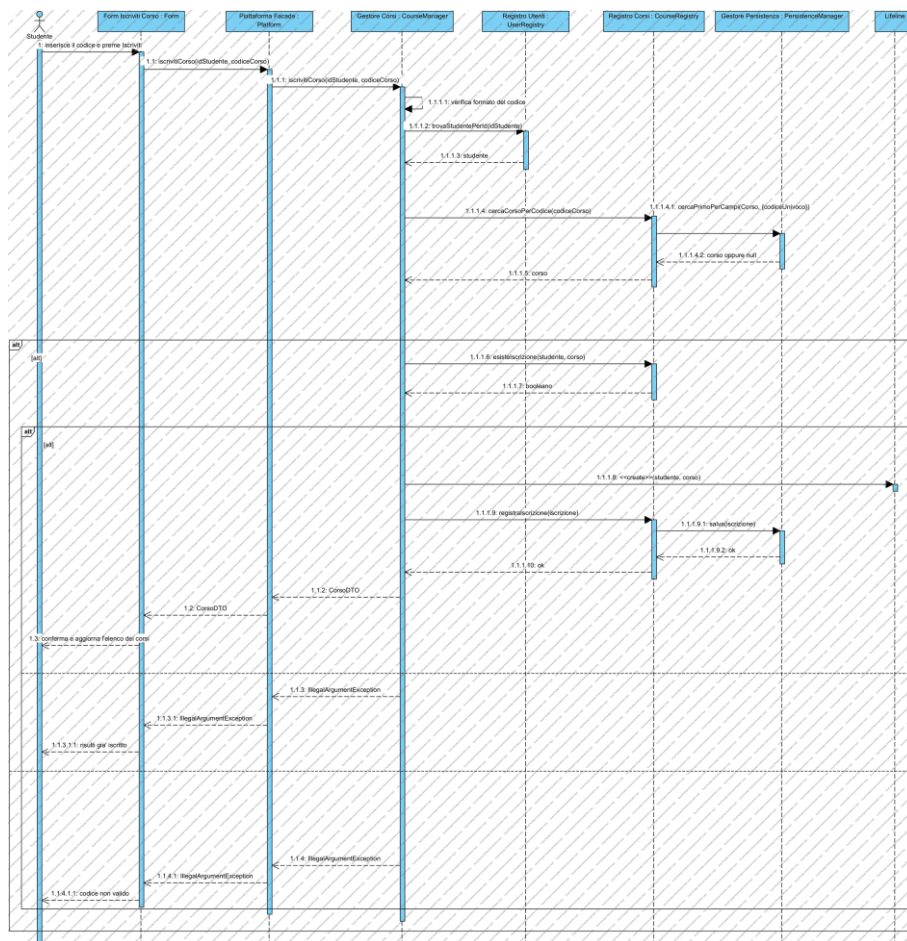


Figura 20 — Diagramma di sequenza di progettazione — IscrivitiCorso (UC6) [\[apri il modello\]](#)

La ricerca del Corso e la verifica di non-duplicazione sono due interrogazioni distinte a RegistroCorsi; la creazione dell'Iscrizione e il suo salvataggio avvengono solo dopo che entrambe hanno dato l'esito atteso. Il valore restituito al boundary è un CorsoDTO, non un Corso: il boundary riceve i dati da mostrare, non l'oggetto del dominio.

4.5.4 PubblicaContenuto (UC7)

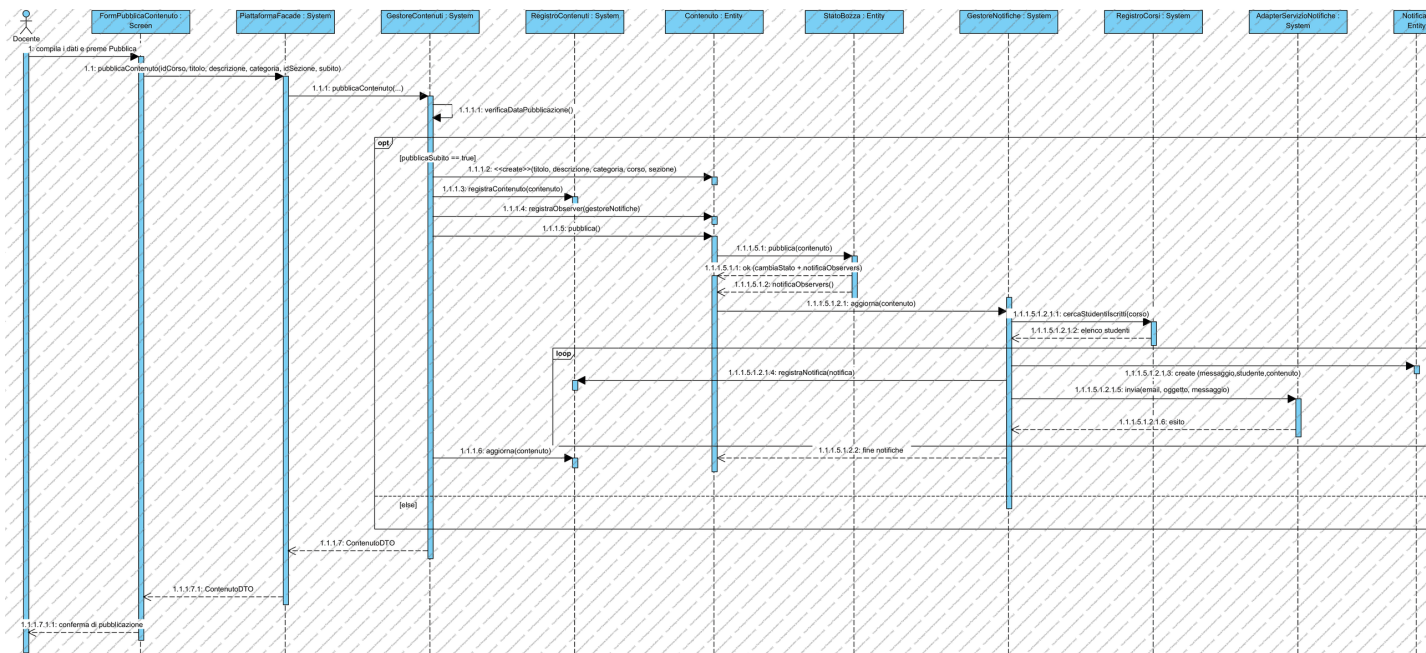


Figura 21 — Diagramma di sequenza di progettazione — PubblicaContenuto (UC7) [\[apri il modello\]](#)

È il diagramma più ricco, perché tre pattern vi compaiono insieme. Il Contenuto viene creato in stato di bozza; se il Docente ha richiesto la pubblicazione immediata, il messaggio pubblica() viene inoltrato all'oggetto stato, che esegue la transizione (State) e fa notificare gli osservatori (Observer); GestoreNotifiche, in quanto osservatore, crea una Notifica per ogni Studente iscritto e ne chiede il recapito a ServizioNotifiche, la cui realizzazione concreta è l'adattatore del boundary (Adapter).

Vale la pena notare quale messaggio non compare: non c'è alcuna chiamata da GestoreContenuti a GestoreNotifiche. La notifica non è un passo della procedura di pubblicazione, è una conseguenza del cambio di stato, e questo è esattamente ciò che l'Observer permette di esprimere.

4.5.5 VisualizzaContenutiCorso (UC13)

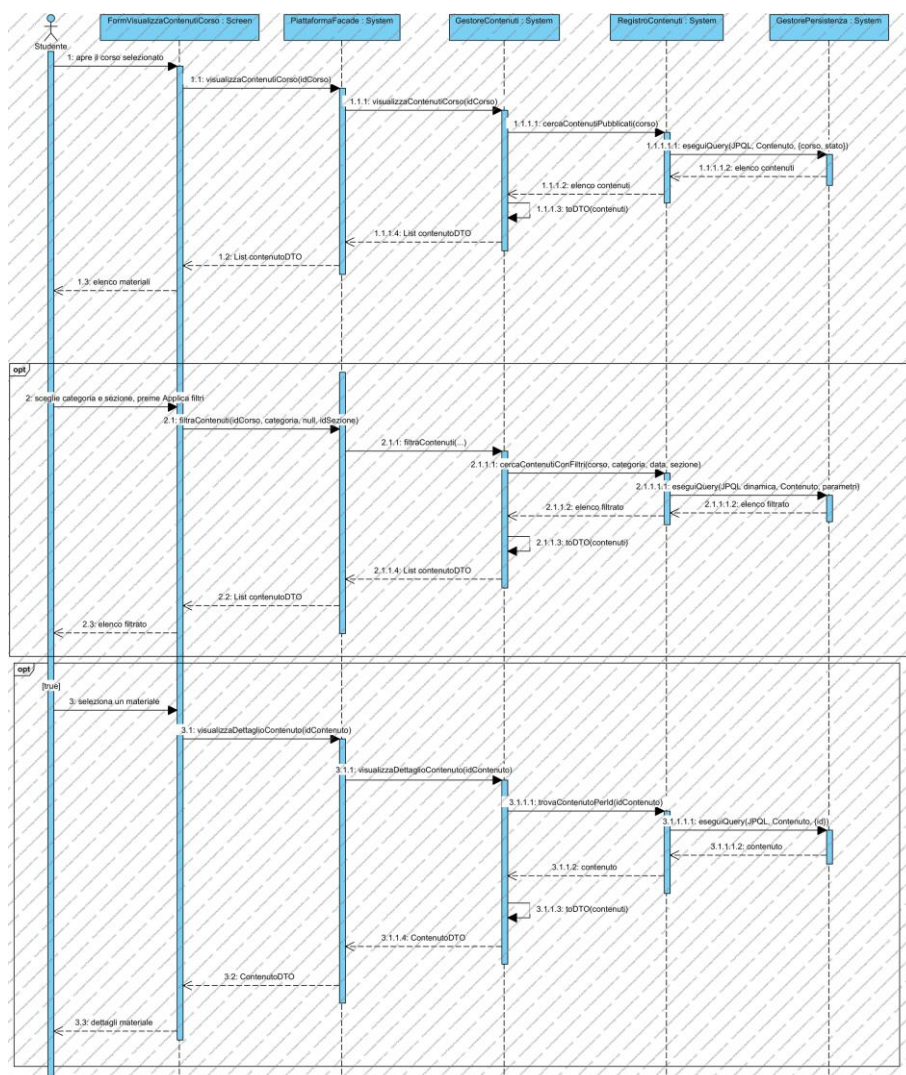


Figura 22 — Diagramma di sequenza di progettazione — VisualizzaContenutiCorso (UC13) [\[apri il modello\]](#)

La verifica dell'iscrizione avviene prima di qualunque lettura, interrogando RegistroCorsi con l'identificativo dello Studente preso dalla sessione. Il filtro è realizzato come interrogazione parametrica su RegistroContenuti e non come selezione in memoria: il filtraggio a valle di una lettura completa funzionerebbe, ma trasferirebbe dal database all'applicazione dati che verrebbero poi scartati.

4.5.6 MonitoraAndamentoCorso (UC15)

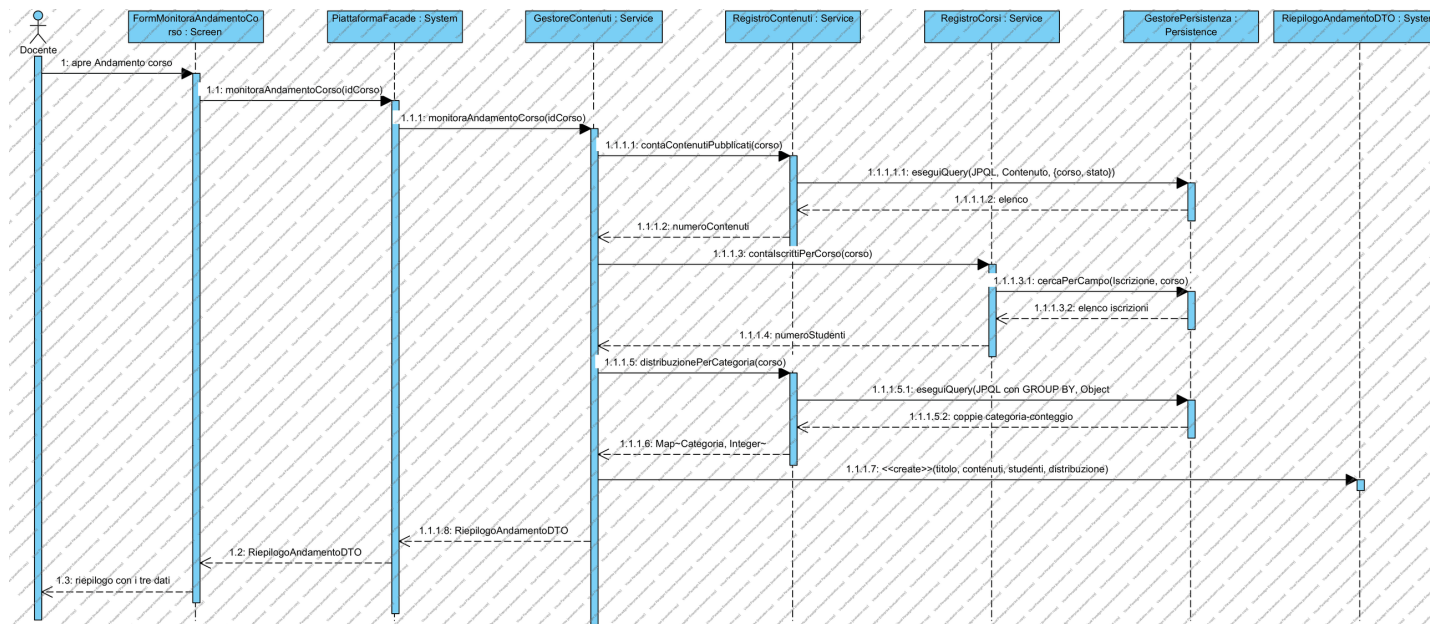


Figura 23 — Diagramma di sequenza di progettazione — MonitoraAndamentoCorso (UC15) [\[apri il modello\]](#)

I tre conteggi sono tre interrogazioni distinte, ciascuna eseguita dal Registro competente, e il risultato è assemblato in un unico `RiepilogoAndamentoDTO` restituito al boundary. È la realizzazione della scelta anticipata in analisi al §2.7.6: un solo oggetto di ritorno invece di tre valori separati, così che l'aggiunta di una nuova statistica non modifichi la firma del metodo.

4.5.7 VisualizzaNotifiche (UC22)

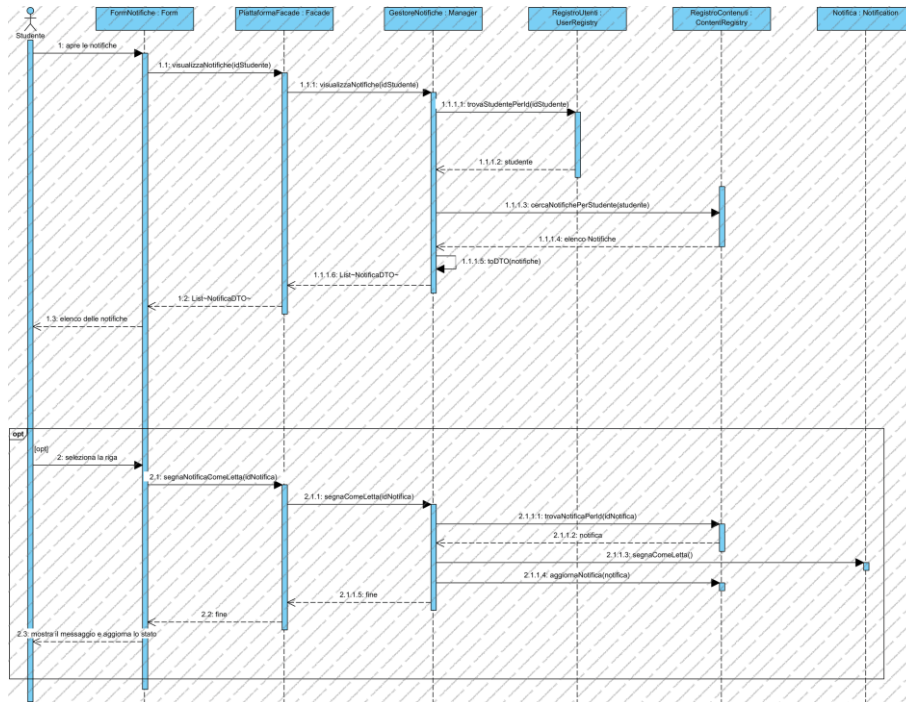


Figura 24 — Diagramma di sequenza di progettazione — VisualizzaNotifiche (UC22) [\[apri il modello\]](#)

Il caso d'uso chiude il ciclo aperto da UC7: le Notifiche prodotte dall'osservatore vengono lette dallo Studente e marcate come lette. La marcatura è un aggiornamento del dominio ed è quindi eseguita invocando il metodo dell'entità Notifica, non modificando direttamente la colonna sul database.

5. Implementazione

Il capitolo descrive la realizzazione del modello di progettazione, procedendo dal livello più basso dell'architettura verso il più alto: database, entity, control, boundary. Per ogni livello si riportano le classi realizzate, i frammenti di codice significativi e la motivazione delle scelte implementative. I frammenti sono estratti dai file sorgente del repository: il collegamento posto sopra ciascuno di essi apre il file corrispondente.

5.1 Struttura del progetto e tecnologie

Elemento	Scelta	Motivazione
Linguaggio	Java 17	Vincolo V01; è la versione con supporto a lungo termine usata a lezione
Build e dipendenze	Apache Maven	Vincolo V06; un solo file pom.xml descrive dipendenze e compilazione
Persistenza	Hibernate 6.4.4 (implementazione di JPA 3.0)	Vincolo V02; è l'ORM indicato dalla lezione 16
DBMS	MySQL 8 con driver mysql-connector-j 8.4.0	Vincolo V02
Interfaccia grafica	Java Swing con look-and-feel FlatLaf 3.4.1	Vincolo V03; FlatLaf sostituisce l'aspetto predefinito, datato, senza modificare il codice dei componenti
Test di unità	JUnit 5 (Jupiter) 5.10.2	È il framework presentato alla lezione 19

Tabella 46 — Tecnologie adottate e loro motivazione

Il progetto è organizzato secondo la struttura standard Maven: i sorgenti in `src/main/java`, le risorse di configurazione in `src/main/resources`, i test in `src/test/java`. I package Java riproducono esattamente i package del diagramma delle classi di progettazione, così che a ogni package del modello corrisponda una cartella del codice.

Package	Classi	Righe di codice
it.unina.materialedidattico.boundary	13	1.481
it.unina.materialedidattico.control (incluso il sotto-package factory)	10	1.076
it.unina.materialedidattico.entity (incluso il sotto-package stato)	19	1.272
it.unina.materialedidattico.database	2	228
it.unina.materialedidattico.dto	6	314
Main (composition root)	1	35
Totale, esclusi i test	51	4.406
Suite di test	6	668

Tabella 47 — Consistenza del codice sorgente per package

5.1.1 Configurazione della persistenza

L'intera configurazione dell'ORM è contenuta nel file META-INF/persistence.xml, che dichiara la persistence unit, l'elenco delle classi mappate, i parametri di connessione al database e il dialetto SQL. Nessun'altra classe del progetto contiene stringhe di connessione: sostituire MySQL con un altro DBMS supportato da Hibernate richiede la modifica di questo solo file.

La proprietà hibernate.hbm2ddl.auto è impostata al valore «update», che fa creare a Hibernate le tabelle mancanti a partire dalle annotazioni delle classi entity. È la scelta comoda in fase di sviluppo, e ha un limite di cui va tenuto conto: aggiunge colonne ma non ne rimuove mai, per cui una colonna eliminata dal modello resta nella tabella. È la ragione per cui il repository include lo script sql/reset_schema.sql, che ricrea lo schema da zero.

5.2 Package database

Il livello più basso dell'architettura è realizzato da due sole classi, nessuna delle quali conosce il dominio applicativo.

5.2.1 JpaUtil

JpaUtil incapsula la EntityManagerFactory. La factory è una risorsa costosa — la sua costruzione comporta la lettura della configurazione, la validazione delle mappature e la creazione del pool di connessioni — e va quindi creata una sola volta per esecuzione: da qui il Singleton. Gli EntityManager, al contrario, sono oggetti leggeri e usa-e-getta, e ne viene creato uno nuovo per ogni operazione.

Da [database/JpaUtil.java](#) (righe 18-46)

```
public class JpaUtil {

    private static JpaUtil istanza;

    private final EntityManagerFactory emf;

    private JpaUtil() {
        this.emf = Persistence.createEntityManagerFactory("materialeDidatticoPU");
        // Chiusura pulita della factory alla terminazione della JVM.
        Runtime.getRuntime().addShutdownHook(new Thread(this::chiudi));
    }

    public static synchronized JpaUtil getIstanza() {
        if (istanza == null) {
            istanza = new JpaUtil();
        }
        return istanza;
    }

    /** Nuovo EntityManager: rappresenta la singola sessione di lavoro col database. */
    public EntityManager getEntityManager() {
        return emf.createEntityManager();
    }

    public void chiudi() {
        if (emf != null && emf.isOpen()) {
            emf.close();
        }
    }
}
```

Lo shutdown hook registrato nel costruttore garantisce che la factory venga chiusa ordinatamente alla terminazione della macchina virtuale, anche se l'applicazione viene chiusa dalla croce della finestra. Senza di esso il pool di connessioni resterebbe aperto fino alla terminazione forzata del processo.

Si noti che JpaUtil è l'unica classe dell'intero progetto che conosce il nome della persistence unit: né l'entity, né il control, né il boundary sanno quale ORM o quale DBMS sia in uso. È la forma più concreta del principio di riduzione dell'accoppiamento verso la tecnologia.

5.2.2 GestorePersistenza

GestorePersistenza è la Façade del livello database. Ogni suo metodo apre il proprio EntityManager, avvia la transazione, esegue l'operazione, effettua il commit e chiude in un blocco finally. In caso di eccezione esegue il rollback e ripropaga: non decide cosa mostrare all'utente, perché non conosce il caso d'uso in corso.

Da [database/GestorePersistenza.java](#) (righe 30-44)

```

public void salva(Object oggetto) {
    EntityManager em = JpaUtil.getIstanza().getEntityManager();
    try {
        em.getTransaction().begin();
        em.persist(oggetto);
        em.getTransaction().commit();
    } catch (RuntimeException e) {
        if (em.getTransaction().isActive()) {
            em.getTransaction().rollback();
        }
        throw e;
    } finally {
        em.close();
    }
}

```

Il metodo restituisce void e non un booleano di esito. È una scelta deliberata: un valore di ritorno booleano invita il chiamante a ignorarlo, mentre un'eccezione lo obbliga a occuparsene o a lasciarla propagare consapevolmente. Nella prima versione del progetto questa firma restituiva un booleano, e il ramo che restituiva false era codice irraggiungibile.

Le operazioni di ricerca sono generiche e non conoscono le classi del dominio: ricevono un `Class<T>` e restituiscono `T` o `List<T>`. Il metodo `cercaPerCampi` costruisce dinamicamente l'interrogazione JPQL a partire da una mappa nome-valore, ed è quello su cui poggiano quasi tutte le ricerche dei Registri.

Da [database/GestorePersistenza.java](#) (righe 90-119)

```

public <T> List<T> cercaPerCampi(Class<T> classe, Map<String, Object> campi) {
    EntityManager em = JpaUtil.getIstanza().getEntityManager();
    try {
        StringBuilder jpql = new StringBuilder();
        jpql.append("SELECT e FROM ").append(classe.getSimpleName()).append(" e");

        if (!campi.isEmpty()) {
            jpql.append(" WHERE ");
            int contatore = 0;
            for (String nomeCampo : campi.keySet()) {
                if (contatore > 0) {
                    jpql.append(" AND ");
                }
                // I punti dei percorsi annidati non sono ammessi nei nomi dei parametri
                JPQL.
                jpql.append("e.").append(nomeCampo).append(" = ").append(nomeCampo.replace(".", "_"));
                contatore++;
            }
        }

        TypedQuery<T> query = em.createQuery(jpql.toString(), classe);
        for (Map.Entry<String, Object> campo : campi.entrySet()) {
            query.setParameter(campo.getKey().replace(".", "_"), campo.getValue());
        }
        return query.getResultList();
    } finally {
        em.close();
    }
}

/** Primo risultato di cercaPerCampi; null se non ce ne sono. */

```

Il modello «un EntityManager per operazione». Ogni metodo chiude il proprio EntityManager prima di restituire il risultato: gli oggetti restituiti sono quindi detached, cioè non più agganciati al contesto di persistenza. Le loro collezioni a caricamento pigro non sono più navigabili, e tentare di farlo produce una `LazyInitializationException`. È una conseguenza architetturale precisa, ed è la

ragione per cui le entità non espongono metodi che navighino le proprie collezioni: le aggregazioni sono calcolate dai Registri con interrogazioni dedicate. Il §5.3.3 riprende il punto.

5.3 Package entity

5.3.1 Mappatura delle entità

Le classi del dominio sono mappate sulle tabelle mediante annotazioni JPA. La classe Utente mostra tutti gli elementi ricorrenti: identificatore generato dal database, colonne con vincoli, e la strategia di ereditarietà.

Da [entity/Utente.java](#) (righe 20-49)

```
@Entity
@Table(name = "UTENTE")
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "TIPO", discriminatorType = DiscriminatorType.STRING, length = 10)
public abstract class Utente {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 50)
    private String nome;

    @Column(nullable = false, length = 50)
    private String cognome;

    /** Identificativo con cui l'Utente si autentica: univoco nel sistema (RD07). */
    @Column(name = "EMAIL_ISTITUZIONALE", nullable = false, unique = true, length = 150)
    private String emailIstituzionale;

    @Column(nullable = false, length = 100)
    private String password;

    /** Tentativi di accesso falliti consecutivi (RD08). */
    @Column(name = "TENTATIVI_FALLITI", nullable = false)
    private int tentativiFalliti;

    /** Istante fino al quale l'account resta bloccato; null se non bloccato (RD08). */
    @Column(name = "BLOCCATO_FINO_A")
    private LocalDateTime bloccatoFinoA;
```

L'annotazione di ereditarietà a tabella singola con colonna discriminante realizza la scelta motivata al §4.2.5: **Studente e Docente** aggiungono pochi attributi propri, per cui una tabella unica evita le giunzioni e permette a Hibernate di istanziare la sottoclasse corretta leggendo la sola colonna TIPO. È ciò che serve all'autenticazione, dove il ruolo si conosce solo dopo aver letto il record.

Il vincolo di unicità sulla colonna dell'email traduce direttamente RD07 ed è il presidio ultimo: anche se il controllo applicativo di RegistroUtenti venisse aggirato, il database rifiuterebbe l'inserimento.

5.3.2 Le regole di business nelle entità

Le entità non sono contenitori di dati con soli metodi di lettura e scrittura: contengono le regole di business che riguardano i propri dati, come prescrive il livello Entity del pattern BCED. I metodi seguenti sono quelli emersi dal diagramma di sequenza di analisi del caso d'uso Accesso (§2.7.2).

Da [entity/Utente.java](#) (righe 70-91)

```
public boolean verificaCredenziali(String passwordInserita) {
    return passwordInserita != null && passwordInserita.equals(this.password);
}

/** True se l'account risulta attualmente sotto blocco temporaneo (RQ-SEC-01). */
```



```

    public boolean isBloccato() {
        return bloccatoFinoA != null &&
LocalDateTime.now(ZoneId.of("Europe/Rome")).isBefore(bloccatoFinoA);
    }

    public void incrementaTentativiFalliti() {
        this.tentativiFalliti++;
    }

    public void bloccaTemporaneamente(int minutiBlocco) {
        this.bloccatoFinoA =
LocalDateTime.now(ZoneId.of("Europe/Rome")).plusMinutes(minutiBlocco);
        this.tentativiFalliti = 0;
    }

    public void azzeriTentativiFalliti() {
        this.tentativiFalliti = 0;
        this.bloccatoFinoA = null;
    }

```

La verifica della password è responsabilità dell'Utente perché è l'unico oggetto che la possiede: nessun altro oggetto legge quel campo, e infatti la classe non espone alcun metodo di lettura per la password. È l'applicazione dell'Information Expert stabilita in analisi e conservata in progettazione.

Domanda prevedibile: le password sono in chiaro. Sì, e ne siamo consapevoli. In un sistema reale il campo conterrebbe l'impronta prodotta da una funzione di derivazione lenta come bcrypt o Argon2, e verificaCredenziali confronterebbe le impronte anziché le stringhe. La modifica sarebbe circoscritta a questo solo metodo e al punto di registrazione, proprio perché la password non è leggibile da nessun altro punto del sistema: è un beneficio dell'incapsulamento adottato.

5.3.3 Il pattern State nel codice

Il Contenuto tiene separati il codice dello stato, che è un dato e viene persistito, e l'oggetto stato, che è un comportamento e viene ricostruito in memoria.

Da [entity/Contenuto.java](#) (righe 46-53)

```

/** Codice dello stato corrente: e' l'unica parte dello stato che viene persistita. */
@Enumerated(EnumType.STRING)
@Column(name = "STATO", nullable = false, length = 20)
private StatoContenutoEnum codiceStato;

/** Oggetto-stato corrispondente al codice: ricostruito in memoria, non persistito. */
@Transient
private StatoContenuto stato;

```

L'annotazione che marca il campo come transiente è ciò che impedisce a Hibernate di cercare una colonna corrispondente: l'oggetto stato è un Singleton privo di dati e non ha nulla da salvare. Alla prima richiesta viene risolto a partire dal codice letto dal database.

Le tre operazioni del ciclo di vita non contengono alcuna logica decisionale: si limitano a inoltrare il messaggio all'oggetto stato corrente, che è l'unico a sapere cosa sia ammesso nel proprio stato.

Da [entity/Contenuto.java](#) (righe 98-131)

```

public void pubblica() {
    getStato().pubblica(this);
}

/**
 * Rimuove il materiale: non sarà più visibile agli Studenti (RF12).
 * @throws IllegalStateException se il materiale risulta già rimosso.

```

```

    */
    public void rimuovi() {
        getStato().rimuovi(this);
    }

    /** True se, nello stato corrente, il materiale e' visibile agli Studenti. */
    public boolean isVisible() {
        return getStato().isVisible();
    }

    /**
     * Esegue la transizione di stato. E' invocato soltanto dalle classi del package
     * entity.stato: sono loro a stabilire quali transizioni siano ammesse, mentre qui
     * si registrano il nuovo stato e la data in cui il materiale e' diventato visibile.
     */
    public void cambiaStato(StatoContenuto nuovoStato) {
        this.stato = nuovoStato;
        this.codiceStato = nuovoStato.getCodice();
        if (nuovoStato.isVisible()) {
            this.dataPubblicazione = LocalDate.now();
        }
    }

    /** Rende pubblica la notifica agli osservatori, invocata dallo stato dopo la transizione. */
    public void notificaObservers() {
        notifica(this);
    }

```

Nessun costrutto condizionale sullo stato compare in questa classe, e non ne compare alcuno in tutto il resto del progetto: la tabella delle transizioni del §4.3.4 è realizzata dalle tre classi concrete, ciascuna delle quali risponde soltanto per sé.

Da [entity/stato/StatoBozza.java](#) (righe 24-45)

```

@Override
public void pubblica(Contenuto contenuto) {
    contenuto.cambiaStato(StatoPubblicato.getIstanza());
    // La transizione verso "pubblicato" e' l'evento che fa scattare le Notifiche
    // agli Studenti iscritti: il Contenuto avvisa i propri osservatori (pattern Observer).
    contenuto.notificaObservers();
}

@Override
public void rimuovi(Contenuto contenuto) {
    contenuto.cambiaStato(StatoRimosso.getIstanza());
}

@Override
public boolean isVisible() {
    return false;
}

@Override
public StatoContenutoEnum getCodice() {
    return StatoContenutoEnum.BOZZA;
}

```

Si osservi la riga in cui lo stato di bozza, dopo aver eseguito la transizione, chiede al Contenuto di avvisare i propri osservatori. È il punto esatto in cui il pattern State e il pattern Observer si incontrano, ed è la ragione per cui la notifica agli Studenti avviene automaticamente sia nella pubblicazione immediata sia nell'attivazione differita, senza che nessuno dei due casi d'uso debba ricordarsene.

Le classi StatoPubblicato e StatoRimosso realizzano le transizioni vietate sollevando un'eccezione di stato illegale, che risale fino al boundary e produce il messaggio mostrato all'utente.

5.3.4 I Registri

I tre Registri sono le uniche classi del package entity che conoscono il package database, e realizzano la prescrizione della lezione 14 secondo cui è il livello Entity a rivolgersi al livello Data. Ciascun Registro raggruppa le interrogazioni su un aggregato del dominio.

Registro	Metodi principali	Casi d'uso serviti
RegistroUtenti	registraUtente, aggiorna, cercaUtentePerEmail, esisteEmailIstituzionale, trovaStudentePerId, trovaDocentePerId	UC1, UC2
RegistroCorsi	cercaCorsoPerCodice, cercaCorsiPerDocente, cercaCorsiPerStudente, esisteIscrizione, registraIscrizione, cercaSezioniPerCorso, contaIscrittiPerCorso, cercaStudentiIscritti	UC6, UC12, UC15, UC21
RegistroContenuti	registraContenuto, cercaContenutiPerCorso, cercaContenutiPubblicati, cercaContenutiConFiltri, contaContenutiPubblicati, distribuzionePerCategoria, registraNotifica, cercaNotifichePerStudente	UC7, UC9, UC11, UC13, UC15, UC22

Tabella 48 — Metodi principali dei tre Registri

I metodi di aggregazione — conteggio degli iscritti, conteggio dei Contenuti pubblicati, distribuzione per categoria — sono realizzati come interrogazioni sul database e non come navigazione delle collezioni delle entità. La ragione è quella spiegata al §5.2.2: gli oggetti restituiti sono detached e le loro collezioni pigre non sono navigabili. La conseguenza è anche un beneficio prestazionale, perché il conteggio viene eseguito dal database e non trasferendo in memoria l'intera collezione.

5.4 Package control

5.4.1 PiattaformaFacade

La Façade espone un metodo pubblico per caso d'uso, con parametri di soli tipi primitivi, stringhe e DTO. Il boundary non può quindi ottenere da essa alcun oggetto del dominio, e la regola di stratificazione è garantita dalla firma stessa dei metodi, non dalla disciplina di chi scrive il codice.

Metodo della Façade	Caso d'uso	Gestore invocato
registrazione(ruolo, nome, cognome, email, password)	UC1	GestoreUtenti
accesso(email, password) / logout() / getUtenteAutenticato()	UC2	GestoreUtenti
iscrivitiCorso(idStudente, codiceCorso)	UC6	GestoreCorsi
visualizzaCorsiIscritti(idStudente) / visualizzaCorsiGestiti(idDocente)	UC12	GestoreCorsi
pubblicaContenuto(idCorso, titolo, descrizione, categoria, idSezione, pubblicaSubito)	UC7	GestoreContenuti
attivaContenuto(idContenuto) / rimuoviContenuto(idContenuto)	UC9, UC11	GestoreContenuti
visualizzaContenutiCorso(idCorso) / filtraContenuti(...) / visualizzaDettaglioContenuto(id)	UC13, UC14, UC18	GestoreContenuti
monitoraAndamentoCorso(idCorso)	UC15, UC19, UC20	GestoreContenuti
visualizzaNotifiche(idStudente) / contaNotificheNonLette(idStudente)	UC22	GestoreNotifiche

Tabella 49 — Corrispondenza fra metodi della Façade e casi d'uso

5.4.2 L'autenticazione

Il metodo di accesso di GestoreUtenti realizza per intero lo scenario del §2.5.3.2, comprese le tre sequenze alternative. È il codice più denso di requisiti dell'intero progetto: vi si leggono RF02, RQ-SEC-01 e RQ-SEC-02.

Da [control/GestoreUtenti.java](#) (righe 83-117)

```
public UtenteDTO accesso(String emailIstituzionale, String password) {
    verificaFormatoCredenziali(emailIstituzionale, password);

    Utente utente = registroUtenti.cercaUtentePerEmail(emailIstituzionale.trim());
    if (utente == null) {
        throw new SecurityException("Credenziali non valide.");
    }

    if (utente.isBloccato()) {
        throw new SecurityException("Account bloccato per troppi tentativi falliti. "
            + "Riprovare tra " + MINUTI_BLOCCO + " minuti.");
    }

    if (!utente.verificaCredenziali(password)) {
        utente.incrementaTentativiFalliti();
        boolean appenaBloccato = utente.getTentativiFalliti() >= MAX_TENTATIVI;
```

```

        if (appenaBloccato) {
            utente.bloccaTemporaneamente(MINUTI_BLOCCO);
        }
        registroUtenti.aggiorna(utente);

        if (appenaBloccato) {
            throw new SecurityException("Credenziali non valide. Account bloccato per "
                + MINUTI_BLOCCO + " minuti.");
        }
        throw new SecurityException("Credenziali non valide.");
    }

    utente.azzeriTentativiFalliti();
    registroUtenti.aggiorna(utente);

    UtenteDTO utenteDTO = toDTO(utente);
    SessioneUtente.getIstanza().apri(utenteDTO);
    return utenteDTO;
}

```

Tre dettagli meritano attenzione. Il messaggio di errore è identico quando l'email non esiste e quando la password è sbagliata: è la realizzazione di RQ-SEC-02, che impedisce a un attaccante di scoprire quali indirizzi siano registrati. Il blocco dell'account viene verificato prima della password, così che un account bloccato resti inaccessibile anche a chi conosca le credenziali corrette. L'azzeramento del contatore avviene solo dopo un accesso riuscito, ed è ciò che rende il conteggio dei tentativi consecutivo e non cumulativo.

Si osservi infine che il Gestore non legge mai il campo della password: chiede all'oggetto Utente di verificarla. La logica di autenticazione sta nel control, la conoscenza della credenziale resta nell'entity.

5.4.3 La validazione dei dati

Ogni Gestore valida i propri dati di ingresso prima di qualunque operazione, secondo il principio di progettazione difensiva della lezione 13. I controlli sono espressi come precondizioni che sollevano un'eccezione con un messaggio destinato all'utente finale.

Dato validato	Regola	Requisito
Email istituzionale	Formato conforme all'espressione regolare e dominio unina.it o studenti.unina.it	RD07
Password	Almeno 8 caratteri, almeno una lettera maiuscola e almeno una cifra	RQ-SEC-01
Nome e cognome	Non vuoti, lunghezza massima 50 caratteri	RD01
Codice del Corso	Alfanumerico, lunghezza compresa fra 8 e 12 caratteri	RD02
Titolo del Contenuto	Non vuoto, lunghezza massima 100 caratteri	RD04
Descrizione del Contenuto	Non vuota, lunghezza massima 500 caratteri	RD04
Categoria	Valore appartenente all'enumerazione Categoria	RD05
Sezione	Se indicata, deve appartenere al Corso su cui si opera	RD06

Tabella 50 — Regole di validazione realizzate nel livello control

La collocazione nel control, e non nel boundary, è la scelta argomentata al §4.4: queste non sono regole di formattazione ma traduzioni di requisiti, e devono valere per qualunque cliente del control, compresa la suite di test che infatti le verifica.

5.4.4 Observer e Adapter nel codice

GestoreNotifiche realizza l'interfaccia Observer dichiarata nel package entity. Il suo metodo di aggiornamento viene invocato dal Contenuto al momento della transizione verso lo stato pubblicato, e crea una Notifica per ogni Studente iscritto al Corso.

Da [control/GestoreNotifiche.java](#) (righe 52-64)

```
public void aggiorna(Contenuto contenuto) {
    Corso corso = contenuto.getCorso();
    String messaggio = componiMessaggio(corso, contenuto);

    for (Studente iscritto : registroCorsi.cercaStudentiIscritti(corso)) {
        registroContenuti.registraNotifica(new Notifica(messaggio, iscritto, contenuto));

        if (servizioNotifiche != null) {
            servizioNotifiche.invia(iscritto.getEmailIstituzionale(),
                                    "Nuovo materiale in " + corso.getTitolo(), messaggio);
        }
    }
}
```

Il recapito effettivo è delegato all'astrazione ServizioNotifiche, di cui il Gestore non conosce la realizzazione. Quest'ultima è l'adattatore che sta nel boundary e che traduce la chiamata nella firma offerta dal gateway esterno, convertendone il codice numerico di ritorno nel booleano atteso.

Da [boundary/AdapterServizioNotifiche.java](#) (righe 17-26)

```
public class AdapterServizioNotifiche implements ServizioNotifiche {

    private final GatewayNotificheEsterno gateway = new GatewayNotificheEsterno();

    @Override
    public boolean invia(String destinatario, String oggetto, String messaggio) {
        int esito = gateway.send(destinatario, oggetto, messaggio);
        return esito == GatewayNotificheEsterno.ESITO_ACCETTATO;
    }
}
```

Il collegamento fra l'astrazione e la sua realizzazione avviene una sola volta, all'avvio dell'applicazione. È il punto in cui il sistema viene composto, ed è deliberatamente l'unico punto del progetto in cui una classe del control riceve un oggetto costruito dal boundary.

Da [Main.java](#) (righe 21-34)

```
public class Main {

    public static void main(String[] args) {
        // Composizione delle dipendenze: il control dichiara l'astrazione
        // ServizioNotifiche, il boundary ne fornisce la realizzazione concreta
        // adattando il servizio esterno di messaggistica (pattern Adapter).
        GestoreNotifiche.getIstanza().setServizioNotifiche(new AdapterServizioNotifiche());

        // Aspetto uniforme di tutte le schermate: va impostato prima di costruirne una.
        StileGUI.appllica();

        // Swing non e' thread-safe: la GUI va costruita sull'Event Dispatch Thread.
        SwingUtilities.invokeLater(() -> new FormLogin().setVisible(true));
    }
}
```

5.5 Package boundary

Le schermate sono realizzate con Java Swing. Ciascuna raccoglie l'input, invoca il metodo della Façade corrispondente al caso d'uso e presenta il risultato o il messaggio d'errore. Nessuna contiene logica di business, e nessuna importa classi dei package entity o database.

Il gestore dell'evento di accesso mostra la struttura ricorrente in tutte le schermate: chiamata alla Façade in un blocco protetto, e tre categorie di esito distinte.

Da [boundary/FormLogin.java](#) (righe 83-101)

```
private void gestisciAccesso() {
    try {
        UtenteDTO utente = facade.accesso(txtEmail.getText(), new
String(txtPassword.getPassword()));

        if (utente.isDocente()) {
            new DashboardDocente(utente).setVisible(true);
        } else {
            new DashboardStudente(utente).setVisible(true);
        }
        dispose();
    } catch (IllegalArgumentException | SecurityException e) {
        StileGUI.erroro(this, e.getMessage());
        txtPassword.setText("");
        txtPassword.requestFocusInWindow();
    } catch (RuntimeException e) {
        StileGUI.erroroImprevisto(this, e);
    }
}
```

Le eccezioni previste — dati non validi, credenziali rifiutate — producono il messaggio del control, che il boundary mostra senza reinterpretarlo: è il control a sapere cosa sia andato storto. Le eccezioni impreviste sono intercettate separatamente e mostrate come errore di sistema, perché un'eccezione non gestita su un gestore di eventi Swing terminerebbe silenziosamente l'operazione lasciando la finestra apparentemente bloccata.

Una lezione imparata durante lo sviluppo. Il blocco che intercetta le eccezioni impreviste è stato aggiunto dopo un difetto reale: un errore di caricamento pigro sollevava un'eccezione non prevista dai rami precedenti, che risaliva fino all'Event Dispatch Thread senza produrre alcun messaggio. All'utente la finestra appariva semplicemente inerte. Il difetto è descritto per esteso al §6.5.

5.5.1 La classe StileGUI

StileGUI raccoglie la logica di presentazione comune: applicazione del look-and-feel, palette dei colori, tipografia, costruzione dei componenti ricorrenti (pulsanti primari e secondari, campi di testo, intestazioni di tabella) e finestre di dialogo per conferme, avvisi ed errori. Senza di essa ognuna delle dieci schermate ripeterebbe le stesse decine di righe di impostazione.

La classe non realizza alcun caso d'uso: è un servizio di presentazione, e la sua collocazione nel boundary è quella corretta secondo la definizione del livello data dalla lezione 14.

5.5.2 Le schermate realizzate

Si riportano le schermate dell'applicazione in esecuzione, nell'ordine in cui l'utente le incontra.

Registrazione (UC1)

The screenshot shows a web browser window titled 'Piattaforma di Materiale Didattico - Registrazione'. The main heading is 'Crea un nuovo account'. Below it, a note states 'Serve un'email istituzionale del dominio unina.it'. The form includes a 'Ruolo' dropdown menu with 'Studiante' selected, and input fields for 'Nome', 'Cognome', 'Email istituzionale', and 'Password (8-32 caratteri)'. A blue 'Registrati' button is at the bottom, with a link 'Torna all'accesso' below it.

Figura 25 — Schermata di registrazione di un nuovo Utente [\[apri il modello\]](#)

La scelta del ruolo determina quale factory verrà usata dal control per costruire l'oggetto: è il punto in cui il Factory Method del §4.3.3 diventa visibile all'utente. Le indicazioni sul dominio richiesto e sulla lunghezza della password anticipano all'utente le regole di validazione che il control applicherà, ma non le sostituiscono: i controlli restano nel control e vengono eseguiti comunque.

Accesso (UC2)

The screenshot shows a web browser window titled 'Piattaforma di Materiale Didattico'. The main heading is 'Accedi alla piattaforma'. Below it, a note states 'Materiale didattico dei corsi a cui sei iscritto'. The form includes input fields for 'Email istituzionale' and 'Password'. A blue 'Accedi' button is at the bottom, with a link 'Non hai un account? Registrati' below it.

Figura 26 — Schermata di accesso all'avvio dell'applicazione [\[apri il modello\]](#)

È l'unica finestra che si apre all'avvio: nessuna funzionalità è raggiungibile senza un accesso riuscito, come prescrive il vincolo V05. Nella prima versione dell'elaborato l'applicazione si apriva invece su un menu che permetteva di scegliere quale utente impersonare fra quelli presenti nei dati di esempio.

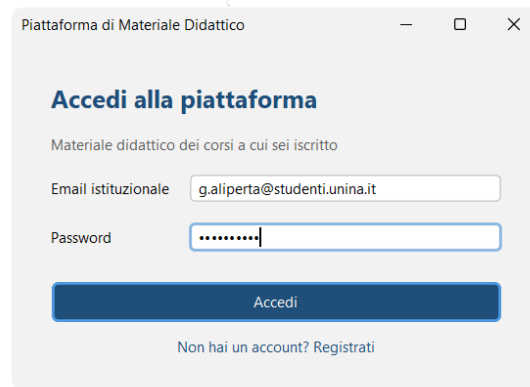


Figura 27 — Schermata di accesso con le credenziali inserite [\[apri il modello\]](#)

Area personale dello Studente

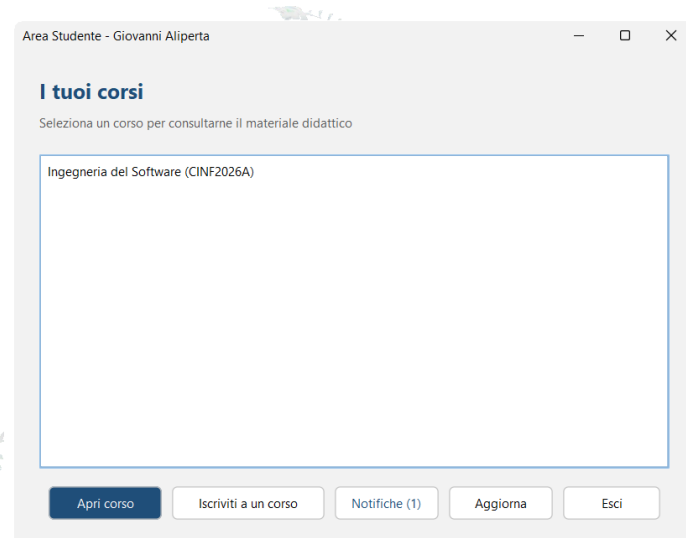


Figura 28 — Area personale dello Studente, con i Corsi seguiti e il contatore delle Notifiche [\[apri il modello\]](#)

La schermata realizza UC12 e costituisce il punto di accesso a UC6, UC13 e UC22. Il contatore accanto al pulsante delle notifiche mostra il numero di messaggi non letti e viene aggiornato a ogni ritorno sulla schermata.

Iscrizione a un Corso (UC6)

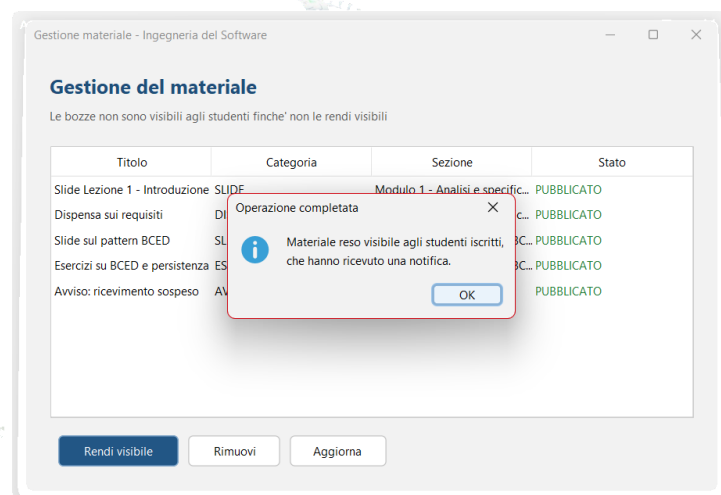


Figura 29 — Messaggio di conferma di un'operazione andata a buon fine [\[apri il modello\]](#)

Tutte le conferme e tutti gli errori sono presentati come finestre di dialogo modali. Nelle prime versioni il riscontro era affidato a un'etichetta all'interno della finestra, che risultava però invisibile perché il ridimensionamento automatico del contenitore ne azzerava l'altezza: un difetto di presentazione che rendeva l'applicazione apparentemente muta.

Consultazione del materiale (UC13, UC14, UC18)

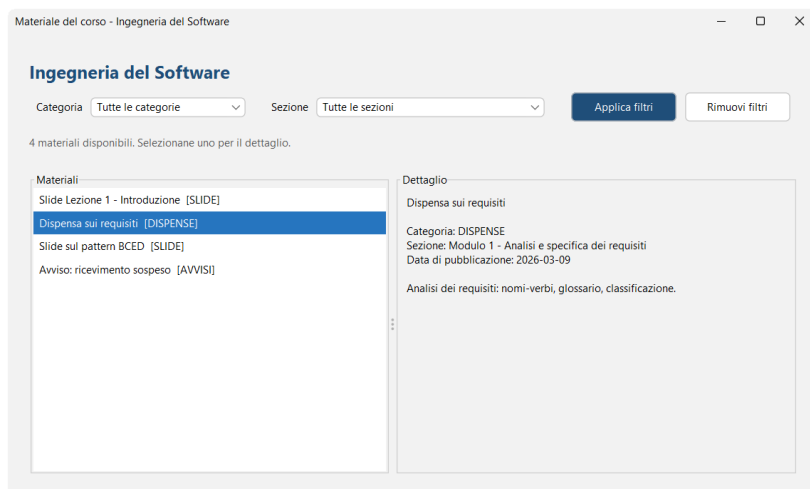


Figura 30 — Elenco dei Contenuti di un Corso, con filtri e pannello di dettaglio [\[apri il modello\]](#)

Allo Studente sono mostrati i soli Contenuti in stato pubblicato: le bozze e i materiali rimossi non compaiono, ed è la verifica visibile del funzionamento del pattern State.

Area personale del Docente

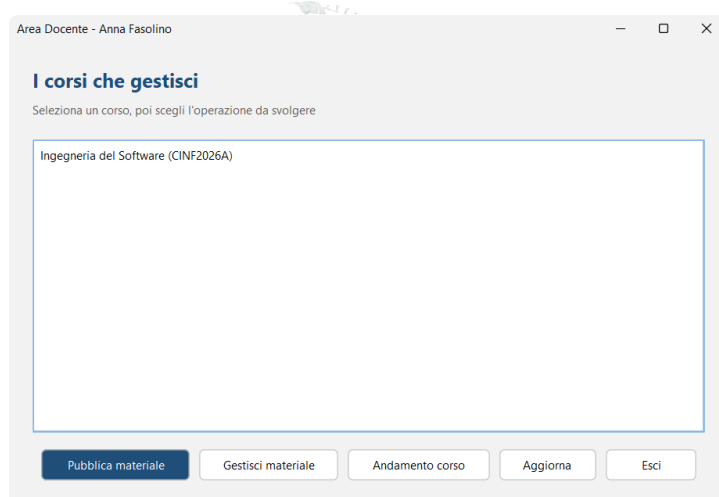


Figura 31 — Area personale del Docente, con i Corsi di cui è titolare [\[apri il modello\]](#)

Pubblicazione del materiale (UC7)

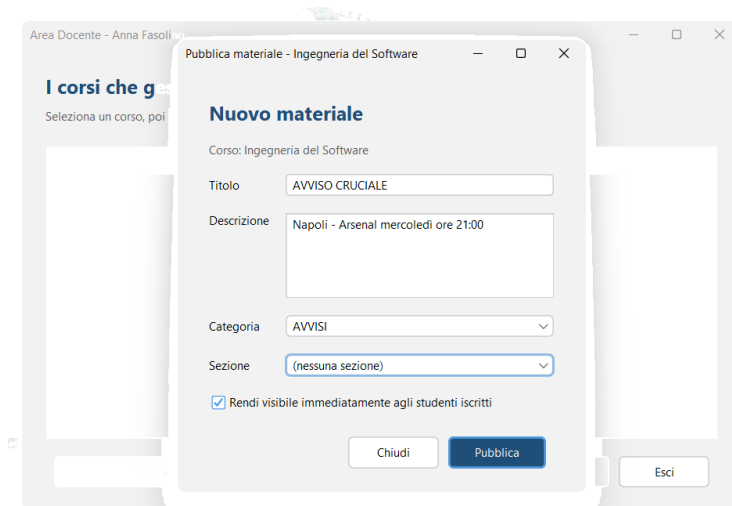


Figura 32 — Schermata di pubblicazione di un nuovo Contenuto [\[apri il modello\]](#)

La casella di scelta in fondo alla finestra determina se il Contenuto nasca già visibile oppure resti in bozza: è il punto di ingresso dei due scenari previsti dalla frase 16 della traccia. L'elenco delle categorie è popolato dai valori dell'enumerazione, e quello delle sezioni dalle sole Sezioni del Corso corrente: due classi di equivalenza di errore del §3.4 sono quindi irraggiungibili dall'interfaccia per costruzione.

Gestione dello stato del materiale (UC9, UC11)

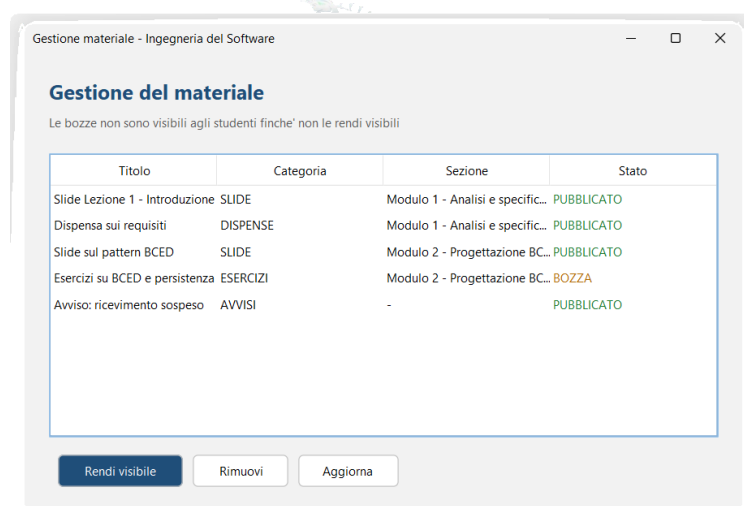


Figura 33 — Elenco dei Contenuti con il loro stato e le azioni ammesse [\[apri il modello\]](#)

È la schermata in cui il pattern State è direttamente osservabile: la colonna dello stato mostra il valore corrente di ciascun Contenuto, e i pulsanti di attivazione e rimozione producono le transizioni. Richiedere una transizione non ammessa — attivare un Contenuto già pubblicato — produce il messaggio di errore generato dall'eccezione sollevata dall'oggetto stato.

Monitoraggio dell'andamento (UC15, UC19, UC20)

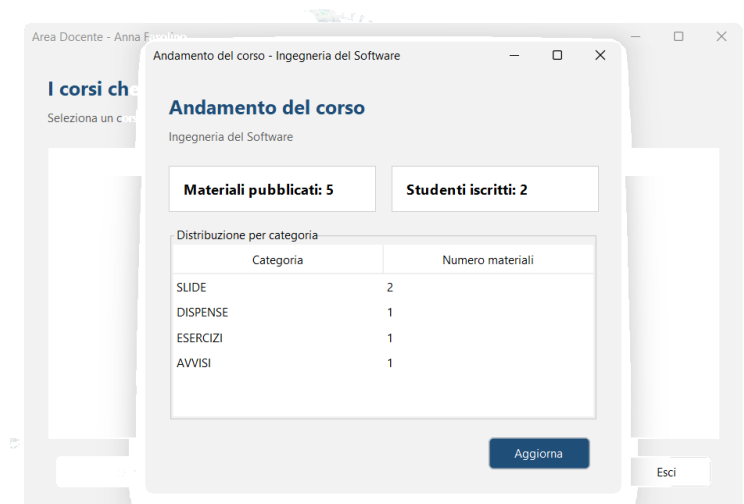


Figura 34 — Riepilogo dell'andamento di un Corso [\[apri il modello\]](#)

Il riepilogo mostra i tre valori calcolati dai Registri e assemblati in un unico DTO: numero di Contenuti pubblicati, numero di Studenti iscritti e distribuzione per categoria.

Notifiche dello Studente (UC22)

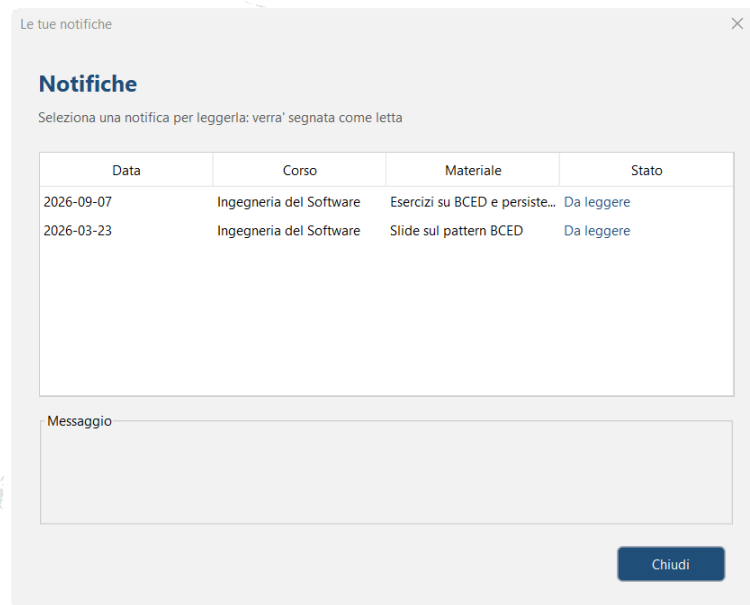


Figura 35 — Elenco delle Notifiche ricevute dallo Studente [\[apri il modello\]](#)

È la schermata che chiude il ciclo aperto dalla pubblicazione: ogni riga corrisponde a una Notifica prodotta da `GestoreNotifiche` in quanto osservatore del `Contenuto`, con il `Corso` di provenienza, il materiale che l'ha generata e lo stato di lettura. Selezionando una riga il messaggio viene mostrato per esteso e la Notifica passa a «letta», e il contatore presente nell'area personale dello `Studente` si aggiorna di conseguenza.

La schermata è la prova che l'Observer funziona da capo a fondo: nessuna delle due notifiche in elenco è stata creata da un'azione dello `Studente` né da una chiamata esplicita del caso d'uso di pubblicazione, ma dalla transizione di stato del `Contenuto`.

5.6 Package dto

I sei DTO trasportano i dati fra control e boundary. Ciascuno contiene soltanto tipi primitivi, stringhe, date e liste di stringhe: nessun riferimento a classi del package entity, nessun comportamento oltre ai metodi di lettura.

DTO	Contenuto	Usato da
UtenteDTO	Identificativo, nome, cognome, email, ruolo come stringa	FormLogin, FormRegistrazione, entrambe le dashboard
CorsoDTO	Identificativo, titolo, descrizione, codice, anno accademico, nome del Docente titolare	Dashboard, FormIscrivitiCorso
SezioneDTO	Identificativo e titolo	FormPubblicaContenuto, FormVisualizzaContenutiCorso
ContenutoDTO	Identificativo, titolo, descrizione, categoria e stato come stringhe, data, titolo della Sezione	FormVisualizzaContenutiCorso, FormGestisciContenuti
NotificaDTO	Identificativo, testo, data, titolo del Corso, flag di lettura	FormNotifiche, DashboardStudente
RiepilogoAndamentoDTO	Numero di Contenuti, numero di iscritti, mappa categoria-conteggio	FormMonitoraAndamentoCorso

Tabella 51 — Data Transfer Object e loro utilizzo

Si osservi che categoria e stato sono trasportati come stringhe e non come valori delle rispettive enumerazioni: le enumerazioni appartengono al package entity, e farle attraversare il confine renderebbe il boundary dipendente da esso. È il dettaglio in cui la regola di stratificazione verrebbe violata senza che nulla lo segnali.

5.7 Diagramma di deployment

Il diagramma di deployment descrive la collocazione fisica dei componenti software sui nodi di elaborazione.

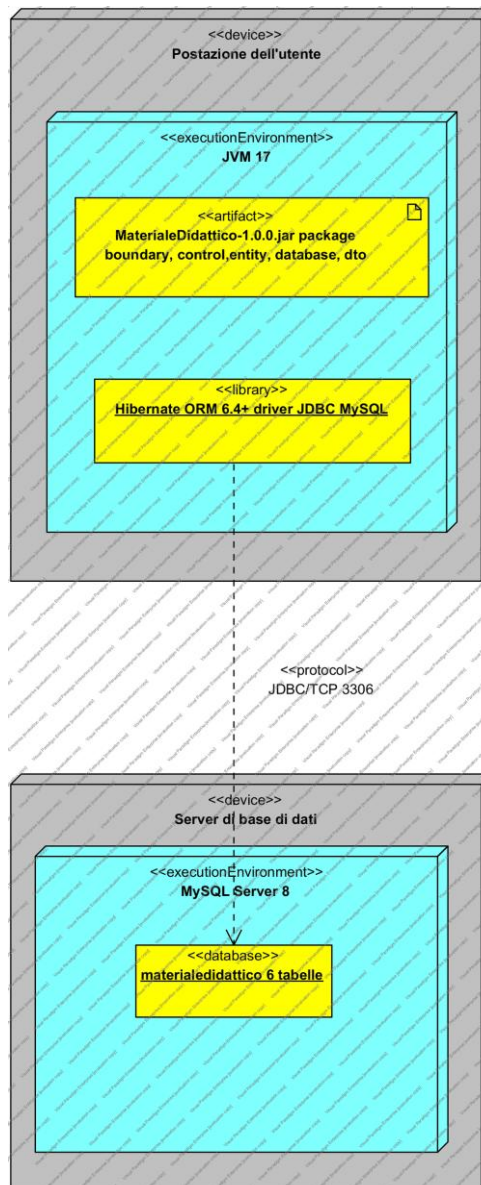


Figura 36 — Diagramma di deployment [\[apri il modello\]](#)

L'applicazione è distribuita come archivio Java eseguibile su una macchina client dotata di macchina virtuale Java 17; il database MySQL risiede su un nodo server, raggiunto attraverso il driver JDBC per il protocollo TCP/IP. Nella configurazione di sviluppo e di dimostrazione i due nodi coincidono, con MySQL in ascolto sulla porta 3306 della stessa macchina.

Che cosa è cambiato rispetto alla prima versione. Il diagramma precedente rappresentava un file di database H2 collocato sul filesystem della macchina client: un'architettura a nodo singolo, senza alcuna comunicazione di rete. La sostituzione di H2 con MySQL, richiesta dal vincolo V02 riscritto, rende il sistema realmente client-server e giustifica la presenza di due nodi distinti.

6. Verifica e validazione

L'attività di verifica è stata condotta su tre fronti complementari: il test strutturale su un metodo rappresentativo, con costruzione del grafo di flusso e calcolo della complessità ciclomatica (§6.1); il test di unità automatizzato con JUnit 5, che attacca il livello control attraverso la Façade (§6.2); l'esecuzione dei casi di test funzionali progettati al capitolo 3 (§6.3). A questi si aggiungono la verifica meccanica del rispetto della stratificazione architetturale (§6.4) e il registro dei difetti individuati e corretti durante lo sviluppo (§6.5).

6.1 Test strutturale

Il metodo scelto per l'analisi strutturale è il metodo di accesso della classe `GestoreUtenti`. La scelta è motivata: è il metodo con il maggior numero di punti di decisione dell'intero progetto, realizza da solo tre requisiti (RF02, RQ-SEC-01, RQ-SEC-02) ed è il presupposto di tutti gli altri casi d'uso, per cui un suo difetto si propagherebbe a tutto il sistema.

6.1.1 Grafo di flusso di controllo

Il grafo è stato costruito a partire dal codice riportato al §5.4.2: ogni blocco di istruzioni consecutive senza salti costituisce un nodo, ogni punto di decisione un nodo a rombo con due archi uscenti.

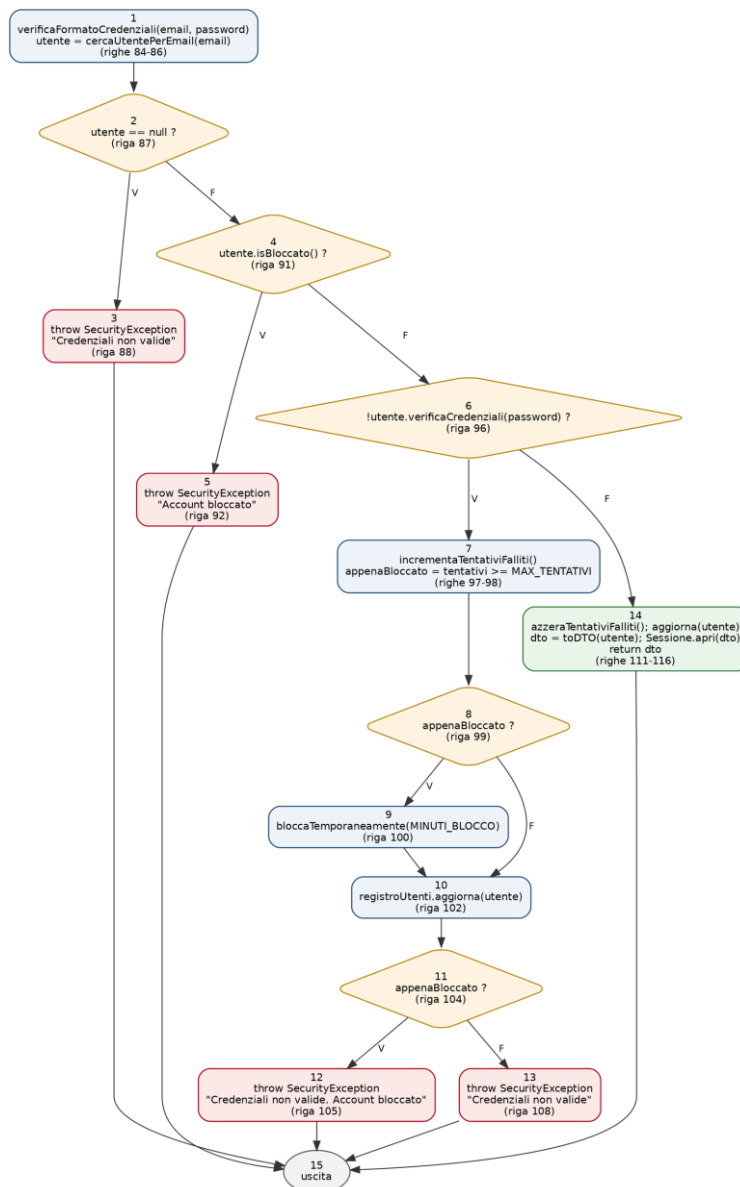


Figura 37 — Grafo di flusso di controllo del metodo `accesso()` di `GestoreUtenti` [\[apri il modello\]](#)

6.1.2 Complessità ciclomatica

La complessità ciclomatica del grafo è stata calcolata con i tre metodi equivalenti presentati alla lezione 19, ottenendo lo stesso valore.

Metodo di calcolo	Elementi	Risultato
$V(G) = E - N + 2$	19 archi, 15 nodi	$19 - 15 + 2 = 6$
$V(G) = \text{numero di predicati} + 1$	5 predicati (righe 87, 91, 96, 99, 104)	$5 + 1 = 6$
$V(G) = \text{numero di regioni del grafo piano}$	6 regioni	6

Tabella 52 — Calcolo della complessità ciclomatica del metodo `accesso()`

Il valore 6 è ampiamente al di sotto della soglia di 10 oltre la quale la letteratura consiglia la scomposizione del metodo. La densità di decisioni è però significativa, e riflette il fatto che il metodo

realizza tre requisiti distinti; una eventuale evoluzione — per esempio l'aggiunta di un secondo fattore di autenticazione — richiederebbe di estrarre la politica di blocco in una classe dedicata.

6.1.3 Cammini indipendenti e corrispondenza con i casi di test

Dal valore della complessità ciclomatica discende che il grafo ammette un insieme base di sei cammini indipendenti. La tabella li elenca, ciascuno con la condizione che lo attiva e il caso di test funzionale del §3.2 che lo copre.

Cammino	Condizione	Esito	Caso di test
1-2-3-15	L'email non corrisponde ad alcun Utente registrato	SecurityException «Credenziali non valide»	TC5
1-2-4-5-15	L'account risulta temporaneamente bloccato	SecurityException «Account bloccato»	TC8
1-2-4-6-14-15	Credenziali corrette e account non bloccato	Sessione aperta, UtenteDTO restituito	TC1, TC2
1-2-4-6-7-8-10-11-13-15	Password errata, tentativi ancora sotto la soglia	SecurityException «Credenziali non valide»	TC6
1-2-4-6-7-8-9-10-11-12-15	Password errata, soglia dei tentativi raggiunta	Account bloccato e SecurityException	TC7
1-2-4-6-7-8-9-10-11-13-15	appenaBloccato vero alla riga 99 e falso alla riga 104	— cammino non percorribile —	nessuno

Tabella 53 — Cammini indipendenti del metodo `accesso()` e casi di test corrispondenti

Sul sesto cammino. Il sesto cammino dell'insieme base non è percorribile. Le decisioni delle righe 99 e 104 valutano la stessa variabile locale, che non viene modificata fra le due valutazioni: sono predicati correlati, e la combinazione «vero poi falso» non può verificarsi. È un esempio del fatto che la complessità ciclomatica fornisce un limite superiore al numero di cammini da provare, non il numero esatto: cinque casi di test sono sufficienti a coprire tutti i cammini realmente percorribili, e la copertura dei rami risulta comunque completa.

Si osservi inoltre che i cammini che terminano ai nodi 3 e 13 producono lo stesso identico messaggio pur seguendo percorsi diversi: è la realizzazione di RQ-SEC-02, che impedisce di distinguere dall'esterno il caso «email non registrata» dal caso «password errata». Il test di unità dedicato verifica proprio l'uguaglianza dei due messaggi.

6.2 Test di unità con JUnit

La suite automatizzata è composta da 37 test raccolti in cinque classi, una per area funzionale. Tutti i test attaccano `PiattaformaFacade`, cioè il livello control, senza istanziare alcun componente Swing: è la verifica pratica del principio di progettazione per la testabilità enunciato al §4.4. Se la logica di business fosse stata scritta dentro le schermate, questa suite non sarebbe stata scrivibile.

Classe di test	Casi d'uso verificati	N. test
<code>RegistrazioneTest</code>	UC1	9
<code>AccessoTest</code>	UC2	8
<code>IscrivitiCorsoTest</code>	UC6	5
<code>PubblicaContenutoTest</code>	UC7, UC9, UC11	9
<code>MonitoraAndamentoCorsoTest</code>	UC15, UC13, UC18, UC22	6
Totale	9 casi d'uso	37

Tabella 54 — Composizione della suite di test di unità

6.2.1 Isolamento dell'ambiente di test

I test operano su uno schema di database separato, dichiarato in una persistence unit distinta presente solo fra le risorse di test. La classe base comune svuota le tabelle prima di ogni test e mette a disposizione i metodi di preparazione dei dati — registrazione di un Docente, registrazione di uno Studente, creazione di un Corso — così che ogni test parta da uno stato noto e sia indipendente dall'ordine di esecuzione.

La separazione degli schemi è una precauzione necessaria: i test cancellano ripetutamente il contenuto delle tabelle, e farlo sullo schema di lavoro distruggerebbe i dati di dimostrazione a ogni esecuzione.

6.2.2 Elenco dei test e loro esito

Di seguito l'elenco completo dei test, con l'indicazione del requisito o della regola di progettazione che ciascuno verifica. Tutti i test risultano superati.

Classe	Test	Verifica	Esito
<code>RegistrazioneTest</code>	Registrazione di uno Studente con dati validi	RF01, Factory Method	PASS
	Registrazione di un Docente con dati validi	RF01, Factory Method	PASS
	Il ruolo deve essere Studente o Docente	Validazione, RF01	PASS
	Il nome non può essere vuoto	Validazione, RD01	PASS
	Il nome non può contenere cifre	Validazione, RD01	PASS
	L'email deve appartenere al dominio unina.it	Validazione, RD07	PASS
	La password deve avere almeno 8 caratteri	RQ-SEC-01	PASS
	La password non può superare i 32 caratteri	Validazione	PASS

Classe	Test	Verifica	Esito
	L'email istituzionale deve essere univoca nel sistema	RD07	PASS
AccessoTest	Accesso con credenziali corrette e apertura della sessione	RF02, V05	PASS
	Password errata: accesso rifiutato	RF02	PASS
	Email inesistente e password errata producono lo stesso messaggio	RQ-SEC-02	PASS
	Formato email non valido: rifiutato prima di interrogare il registro	Progettazione difensiva	PASS
	Password vuota: rifiutata come formato non valido	Validazione	PASS
	Dopo cinque tentativi falliti l'account viene bloccato	RQ-SEC-01	PASS
	Un accesso riuscito azzerà i tentativi falliti precedenti	RQ-SEC-01	PASS
	Il logout chiude la sessione	V05	PASS
IscrivitiCorsoTest	Iscrizione con codice valido	RF06	PASS
	Codice che non corrisponde a nessun corso	RF06, scenario alternativo	PASS
	Codice di formato non valido: troppo corto	Validazione, RD02	PASS
	Codice di formato non valido: caratteri non alfanumerici	Validazione, RD02	PASS
	Un secondo tentativo di iscrizione allo stesso corso viene rifiutato	RD03, unicità	PASS
PubblicaContenutoTest	Pubblicazione immediata: materiale visibile e iscritti notificati	RF07, RF22, Observer	PASS
	Salvataggio come bozza: non visibile e nessuna notifica	RF10, State	PASS
	L'attivazione di una bozza la rende visibile e notifica gli iscritti	RF10, RF22, State + Observer	PASS
	Un materiale già pubblicato non può essere ripubblicato	State, transizione vietata	PASS
	La rimozione toglie il materiale dalla vista degli studenti	RF12, State	PASS
	Un materiale rimosso non può essere rimosso né ripubblicato	State, transizioni vietate	PASS
	Il titolo è obbligatorio	Validazione, RD04	PASS
	La descrizione è obbligatoria	Validazione, RD04	PASS
	La categoria deve essere una di quelle ammesse	RD05	PASS

Classe	Test	Verifica	Esito
MonitoraAndamentoCorsoTest	Corso senza materiali né iscritti: riepilogo a zero	RF19, scenario alternativo	PASS
	Il riepilogo conta materiali, iscritti e distribuzione	RF17, RF18, RF19	PASS
	Il filtro per categoria restituisce solo i materiali richiesti	RF15	PASS
	Senza filtri vengono restituiti tutti i materiali visibili	RF14	PASS
	La notifica selezionata viene segnata come letta	RF24, RD09	PASS
	Ogni iscritto riceve la propria notifica	RF22, Observer	PASS

Tabella 55 — Esito della suite di test di unità (37 test, 37 superati)

L'esecuzione completa della suite richiede circa quindici secondi, comprensivi dell'inizializzazione della EntityManagerFactory, e si avvia con il comando `mvn test`.

Che cosa verificano davvero questi test. Nove test su trentasette non verificano un requisito funzionale ma una scelta di progettazione: i quattro test sulle transizioni di stato vietate verificano il pattern State, i due test sulle notifiche verificano il pattern Observer, il test sull'uguaglianza dei messaggi verifica RQ-SEC-02, i due test sui contatori verificano la politica di blocco. Sono i test che sarebbe stato impossibile scrivere sulla versione precedente del progetto, perché quei comportamenti non esistevano.

6.3 Esiti del test funzionale

I casi di test progettati al capitolo 3 sono stati eseguiti sull'applicazione completa attraverso l'interfaccia grafica. La tabella riporta, per ciascun caso d'uso, l'esito complessivo e le note rilevanti.

Caso d'uso	Casi di test	Esito	Note
UC1 Registrazione	TC1-TC8 (8)	8 PASS	TC6 verificato registrando due volte la stessa email: il secondo tentativo è respinto dal controllo applicativo prima di raggiungere il vincolo del database
UC2 Accesso	TC1-TC8 (8)	8 PASS	TC7 e TC8 verificati sbagliando cinque volte la password e riprovando poi con quella corretta: l'accesso resta rifiutato
UC6 IscrivitiCorso	TC1-TC6 (6)	6 PASS	TC1 eseguito sul Corso CINF2024B, perché lo Studente dei dati di esempio è già iscritto a CINF2024A
UC7 PubblicaContenuto	TC1-TC9 (9)	9 PASS	TC8 e TC9 non raggiungibili dall'interfaccia per costruzione (elenchi popolati dai soli valori validi); verificati a livello di control dalla suite JUnit
UC9 / UC11 Attiva e Rimuovi	TC1-TC7 (7)	7 PASS	TC4, TC5 e TC6 producono il messaggio generato dall'eccezione sollevata dall'oggetto stato
UC13 VisualizzaContenutiCorso	TC1-TC6 (6)	6 PASS	TC5 verificato su un Corso contenente solo bozze: l'elenco risulta correttamente vuoto
UC15 MonitoraAndamentoCorso	TC1-TC4 (4)	4 PASS	TC4 verificabile per la prima volta in questa versione: senza autenticazione il controllo di titolarità non era eseguibile
UC22 VisualizzaNotifiche	TC1-TC3 (3)	3 PASS	TC3 verificato riaprendo la schermata: il contatore delle non lette torna a zero
Totale	51 casi di test	51 PASS	—

Tabella 56 — Esiti dell'esecuzione dei casi di test funzionali

Due categorie di casi meritano una precisazione. I casi marcati «non raggiungibili dall'interfaccia per costruzione» corrispondono a classi di equivalenza di errore che l'interfaccia grafica rende impossibili: l'elenco a discesa delle categorie contiene solo i valori dell'enumerazione, e quello delle sezioni solo le sezioni del Corso corrente. Il caso di test resta valido e viene verificato al livello del control, dove il controllo esiste ed è necessario, perché il control deve difendersi da qualunque cliente, non solo dalla propria interfaccia grafica.

Nella prima versione dell'elaborato quattro casi di test erano stati classificati come «non applicabili» perché richiedevano di sapere chi fosse l'utente collegato. Tutti e quattro sono oggi eseguibili e superati: è la conseguenza più immediata dell'introduzione dell'autenticazione.

6.4 Verifica del rispetto dell'architettura

Il rispetto della stratificazione BCED non è verificabile con i test funzionali: un sistema che violasse la direzione delle dipendenze funzionerebbe ugualmente. La verifica è stata quindi condotta ispezionando le direttive di importazione di tutte le classi del progetto, controllo che è ripetibile in pochi secondi e il cui esito è riportato di seguito.

Regola verificata	Violazioni trovate
Nessuna classe di boundary importa classi di entity	0
Nessuna classe di boundary importa classi di database	0
Nessuna classe di entity importa classi di control	0
Nessuna classe di entity importa classi di boundary	0
Nessuna classe di database importa classi di boundary, control o entity	0
Nessuna classe di control importa classi di boundary	0
Solo i tre Registri, fra le classi di entity, importano classi di database	0
Solo PiattaformaFacade, fra le classi di control, è importata dal boundary	0

Tabella 57 — Esito della verifica delle dipendenze fra i livelli architetturali

L'unica eccezione apparente è il metodo `main`, che importa sia una classe del boundary sia una del control per collegare l'adattatore al gestore delle notifiche. Non è una violazione: il `main` non appartiene ad alcuno dei quattro livelli, è il punto in cui l'applicazione viene composta, ed è per definizione l'unico luogo in cui i livelli possono essere messi in contatto.

6.5 Registro dei difetti individuati e corretti

Si riportano i difetti individuati durante lo sviluppo e la verifica, con la causa e la correzione applicata. Sono documentati perché due di essi hanno prodotto una modifica di progettazione, e non soltanto di codice.

Difetto osservato	Causa	Correzione
Nessun messaggio di conferma dopo un'operazione riuscita	Il riscontro era affidato a un'etichetta interna alla finestra, la cui altezza veniva azzerata dal ridimensionamento automatico del contenitore	Sostituzione di tutti i riscontri con finestre di dialogo modali, centralizzate in <code>StileGUI</code>
La pubblicazione di un Contenuto non produceva alcun effetto né alcun messaggio	Eccezione di caricamento pigro sollevata navigando la collezione di un oggetto detached; non essendo del tipo intercettato, risaliva fino all'Event Dispatch Thread senza produrre alcun avviso	Rimozione dei metodi che navigavano le collezioni delle entità, sostituiti da interrogazioni sui Registri; aggiunta in tutte le schermate di un blocco che intercetta le eccezioni impreviste
Errore «colonna STATO sconosciuta» all'avvio	La generazione automatica dello schema aggiunge colonne ma non ne rimuove: la vecchia colonna booleana era rimasta in tabella	Aggiunta dello script <code>sql/reset_schema.sql</code> che ricrea lo schema da zero, e della relativa procedura in appendice

Difetto osservato	Causa	Correzione
Codice irraggiungibile nel metodo di salvataggio	Il metodo restituiva un booleano, ma il ramo che restituiva falso era irraggiungibile perché l'eccezione veniva ripropagata	Modifica della firma: il metodo restituisce void e comunica l'esito negativo unicamente con l'eccezione

Tabella 58 — Difetti individuati durante lo sviluppo e correzioni applicate

Il secondo difetto è quello di maggior valore didattico. La sua causa non era un errore di battitura ma una conseguenza dell'architettura: avendo scelto il modello «un EntityManager per operazione», gli oggetti restituiti sono detached, e le loro collezioni pigre non sono navigabili. La correzione non è stata aggiungere un controllo, ma togliere dalle entità i metodi che navigavano le collezioni, riportando quelle interrogazioni sui Registri, dove l'EntityManager è aperto. È il tipo di difetto che si previene con la progettazione, non con il collaudo.

Appendice A — Indice dei diagrammi e collegamento con il modello

Il modello Visual Paradigm e questo documento costituiscono un unico elaborato. Ogni figura del documento riporta sotto la didascalia un collegamento che apre il diagramma corrispondente nel repository; specularmente, ogni diagramma del modello riporta nelle proprie proprietà il riferimento al paragrafo che lo descrive. La tabella seguente è l'indice completo di questa corrispondenza.

#	Diagramma	Tipo	Paragrafo
1	Use Case — Analisi	Use Case Diagram	§2.5.2
2	Class Diagram — Analisi	Class Diagram	§2.6
3	Sequence Diagram UC1 Registrazione — Analisi	Sequence Diagram	§2.7.1
4	Sequence Diagram UC2 Accesso — Analisi	Sequence Diagram	§2.7.2
5	Sequence Diagram UC6 IscrivitiCorso — Analisi	Sequence Diagram	§2.7.3
6	Sequence Diagram UC7 PubblicaContenuto — Analisi	Sequence Diagram	§2.7.4
7	Sequence Diagram UC13 VisualizzaContenutiCorso — Analisi	Sequence Diagram	§2.7.5
8	Sequence Diagram UC15 MonitoraAndamentoCorso — Analisi	Sequence Diagram	§2.7.6
9	StateChart Diagram Contenuto — Analisi	State Machine Diagram	§2.7.7
10	Class Diagram — Analisi Raffinato	Class Diagram	§2.9
11	Distribuzione delle responsabilità di Piattaforma	Class Diagram (sotto-diagramma di Piattaforma)	§2.9.1
12	CD Progettazione — Vista d'insieme	Class Diagram	§4.2.2
13	CD Progettazione — Package Boundary	Class Diagram	§4.2.3
14	CD Progettazione — Package Control	Class Diagram	§4.2.4
15	CD Progettazione — Package Entity	Class Diagram	§4.2.5
16	CD Progettazione — Package Database	Class Diagram	§4.2.6
17	Schema relazionale	Entity Relationship Diagram	§4.2.7
18	UC1 Registrazione — Progetto	Sequence Diagram	§4.5.1
19	UC2 Accesso — Progetto	Sequence Diagram	§4.5.2
20	UC6 IscrivitiCorso — Progetto	Sequence Diagram	§4.5.3
21	UC7 PubblicaContenuto — Progetto	Sequence Diagram	§4.5.4
22	UC13 VisualizzaContenutiCorso — Progetto	Sequence Diagram	§4.5.5
23	UC15 MonitoraAndamentoCorso — Progetto	Sequence Diagram	§4.5.6
24	UC22 VisualizzaNotifiche — Progetto	Sequence Diagram	§4.5.7
25	Deployment Diagram	Deployment Diagram	§5.7
26	CFG del metodo accesso()	Grafo di flusso (generato)	§6.1.1

Tabella 59 — Indice dei diagrammi e corrispondenza con i paragrafi del documento

I diagrammi da 1 a 25 sono contenuti nel file di progetto Visual Paradigm allegato alla consegna; le esportazioni in formato PNG a 300 punti per pollice si trovano nella cartella dei diagrammi del repository.

Repository del progetto: <https://github.com/AntoClem/Ingegneria-del-Software-PROGETTO>

Modello Visual Paradigm: https://github.com/AntoClem/Ingegneria-del-Software-PROGETTO/blob/main/VisualParadigm/Progetto_IS_Gruppo18.vpp

Diagrammi esportati: <https://github.com/AntoClem/Ingegneria-del-Software-PROGETTO/tree/main/VisualParadigm/Diagrammi>

Codice sorgente: <https://github.com/AntoClem/Ingegneria-del-Software-PROGETTO/tree/main/JavaProject/src/main/java/it/unina/materiale didattico>

Appendice B — Installazione ed esecuzione

Si riportano i passi necessari a mettere in esecuzione l'applicazione a partire dal contenuto del repository.

B.1 Prerequisiti

Componente	Versione	Note
Java Development Kit	17 o successiva	Il progetto è compilato con release 17
Apache Maven	3.8 o successiva	Incluso nelle distribuzioni recenti di IntelliJ IDEA
MySQL Server	8.0 o successiva	In ascolto sulla porta 3306
MySQL Workbench	8.0 (facoltativo)	Usato per eseguire gli script di preparazione del database

Tabella 60 — Prerequisiti per l'esecuzione

B.2 Preparazione del database

Le credenziali di accesso al database sono dichiarate nel file di configurazione della persistenza e vanno allineate a quelle del proprio server MySQL. La procedura completa è la seguente.

- Avviare il server MySQL e aprire una connessione come utente root.
- Eseguire lo script `sql/reset_schema.sql`, che elimina e ricrea lo schema `materialeDidatticoDB`. È necessario solo alla prima esecuzione o dopo una modifica delle classi entity.
- Avviare l'applicazione una prima volta: Hibernate crea automaticamente le tabelle a partire dalle annotazioni delle classi entity.
- Chiudere l'applicazione ed eseguire lo script `sql/seed_data.sql`, che popola le tabelle con i dati di dimostrazione. Lo script è autopulente: elimina i dati esistenti prima di inserire i propri, e può quindi essere rieseguito quante volte si vuole.

Perché l'ordine è questo. Lo script dei dati di esempio inserisce righe nelle tabelle, che devono quindi esistere; ma le tabelle sono create da Hibernate al primo avvio, a partire dalle annotazioni. Eseguire lo script prima del primo avvio produce l'errore di tabella inesistente.

B.3 Compilazione ed esecuzione

Operazione	Comando
Compilazione	<code>mvn clean compile</code>
Esecuzione della suite di test	<code>mvn test</code>
Avvio dell'applicazione	<code>mvn exec:java -Dexec.mainClass="it.unina.materialeDidattico.Main"</code>
Avvio da ambiente di sviluppo	Eseguire la classe Main

Tabella 61 — Comandi Maven per la compilazione e l'esecuzione

La suite di test opera su uno schema separato, `materiale didattico db_test`, che viene creato automaticamente alla prima esecuzione: eseguire i test non altera i dati di dimostrazione.

B.4 Credenziali di dimostrazione

Ruolo	Email istituzionale	Password
Docente	a.fasolino@unina.it	Password01
Studente	g.aliperta@studenti.unina.it	Password01

Tabella 62 — Credenziali degli utenti presenti nei dati di dimostrazione

Appendice C — Registro delle revisioni

Questo elaborato è la seconda versione del progetto. La prima è stata presentata nel luglio 2026 e ha ricevuto una serie di osservazioni, tutte recepite. La tabella seguente riporta, per ciascuna osservazione, la modifica apportata e il paragrafo in cui è documentata: è pensata per rendere immediatamente verificabile il lavoro di revisione.

Osservazione ricevuta	Modifica apportata	Dove
L'autenticazione non era realizzata: l'applicazione si apriva su un menu che permetteva di scegliere quale utente impersonare	Introdotti i casi d'uso Registrazione (UC1) e Accesso (UC2), modellati per intero e realizzati nel codice, con gestione della sessione, politica di blocco dell'account e messaggi di errore non informativi. L'applicazione si apre sulla sola schermata di accesso.	§2.5.3.1, §2.5.3.2, §4.5.1, §4.5.2, §5.4.2
Il passaggio dal modello di analisi a quello di progettazione non era tracciabile: mancavano i passi intermedi	Aggiunti il diagramma delle classi di analisi raffinato, la tabella delle responsabilità prima e dopo i diagrammi di sequenza, la tabella di traduzione delle responsabilità in progettazione e il diagramma dedicato alla trasformazione della classe Piattaforma, allegato in Visual Paradigm alla classe stessa. Apposte note UML sui diagrammi nei punti di discontinuità.	§2.6.1, §2.8, §2.9, §2.9.1, §4.2.1
I diagrammi non erano collegati alla documentazione né al modello Visual Paradigm	Ogni figura riporta un collegamento ipertestuale al diagramma nel repository; ogni diagramma del modello riporta nelle proprie proprietà il riferimento al paragrafo corrispondente; l'indice completo della corrispondenza è in appendice A.	Appendice A
I package aggiuntivi e il livello DAO rendevano l'architettura una variante del BCED anziché il BCED	Rimosse le classi DAO, sostituite dalla Façade di persistenza. Il package dto è stato mantenuto e argomentato come tipo di dato di scambio fra livelli e non come livello aggiuntivo; il package di utilità è stato eliminato e la validazione riportata nel control, dove traduce requisiti espliciti.	§4.1, §4.1.1, §5.4.3
Il Singleton di accesso al database e JDBC scritto a mano non corrispondevano a quanto indicato a lezione	Sostituiti da Hibernate (JPA) su MySQL, con la Façade GestorePersistenza come unico punto di accesso alla persistenza. Riscritto il vincolo V02, che nella prima versione imponeva il pattern DAO su H2.	§2.4.4, §4.2.6, §5.2
L'indice del documento conteneva voci non pertinenti al progetto	Documento riscritto integralmente; l'indice è generato dai titoli effettivi e non contiene voci residue.	Indice

Tabella 63 — Osservazioni ricevute sulla prima versione e modifiche apportate

Oltre a quanto richiesto sono stati introdotti tre design pattern che il modello di analisi giustificava e che la prima versione non applicava: State per il ciclo di vita del Contenuto, Observer per la notifica automatica agli Studenti e Adapter per il servizio di messaggistica esterno. Ciascuno risponde a un requisito già presente nella traccia, ed è documentato con la propria motivazione al §4.3.

Indicatore	Prima versione (luglio 2026)	Versione attuale
Casi d'uso realizzati nel codice	4	9
Classi Java	34	51

Indicatore	Prima versione (luglio 2026)	Versione attuale
Righe di codice (esclusi i test)	circa 2.900	4.406
Design pattern applicati	2 (Singleton, DAO)	6 (Singleton, Façade, Factory Method, State, Observer, Adapter)
Interfacce nel modello	0	4
Test di unità automatizzati	0	37
Casi di test funzionali eseguiti	23, di cui 4 non applicabili	51, tutti eseguiti
Diagrammi UML	13	25 (più il grafo di flusso)

Tabella 64 — Confronto sintetico fra le due versioni dell'elaborato